



in28minutes

Getting Started with

Generative AI with Open AI



Hello and Welcome to in28minutes

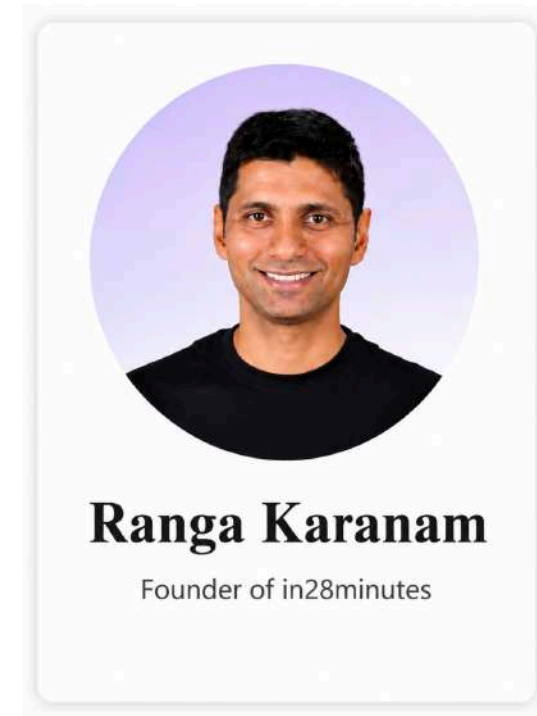
Who We Are: in28minutes, founded by Ranga Karanam

Our Reach: Millions of learners taught worldwide

- But our focus is not the numbers
- **Our focus is your progress**

Our Style: Practical. Story-driven. Step-by-step.

Our Goal: Help you move from where you are to where you want to be





Getting Started with Generative AI

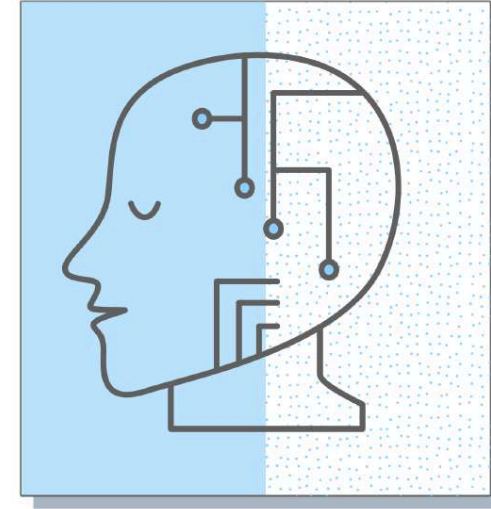
Generative AI: A constantly evolving space

Course Focus: Four key areas

- **Fundamentals:** Understand AI, ML and Generative AI
- **OpenAI:** Understand available tools and API
- **Improving Model Output:** How to get better results?
- **Use OpenAI API:** Integrate generative AI to your apps

Course Design: Learn concepts first, then reinforce them with practical examples

Course Goal: Help you understand generative AI, and integrate generative AI into your applications





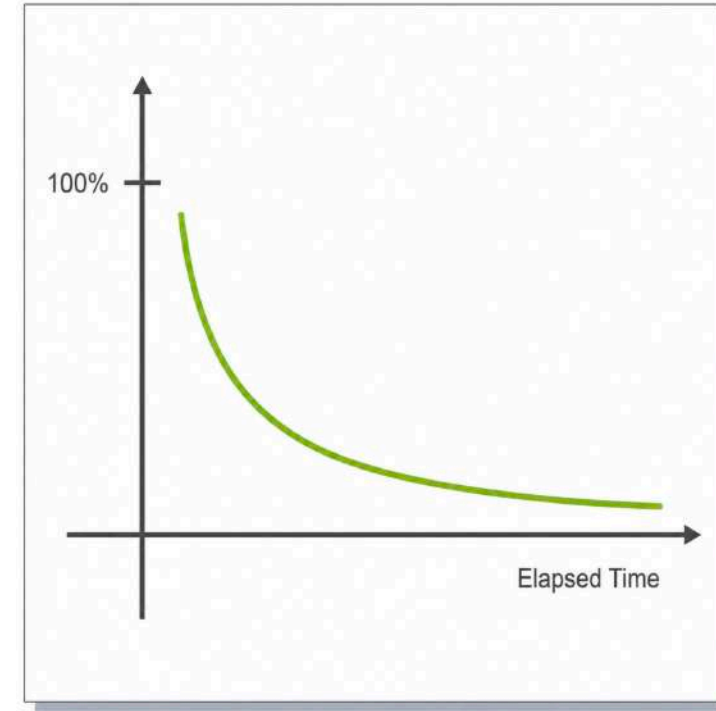
How Do You Put Your Best Foot Forward?

Challenging Certification: You are expected to understand and remember many services

Human Memory Fades: We naturally forget things over time

How Can You Remember More?

- **Active Learning:** Think, practice, and take notes as you learn
- **Regular Review:** Revisit presentations periodically to reinforce your memory





Our Approach to a Great Learning Experience

Three-Pronged Approach to reinforce concepts:

- **Presentations:** Build conceptual understanding
- **Demos:** See concepts in action
- **Quizzes:** Reinforce and test your learning
 - Text quizzes
 - Video quizzes

Recommended:

- Take your time and replay videos whenever needed
- Have fun while learning!



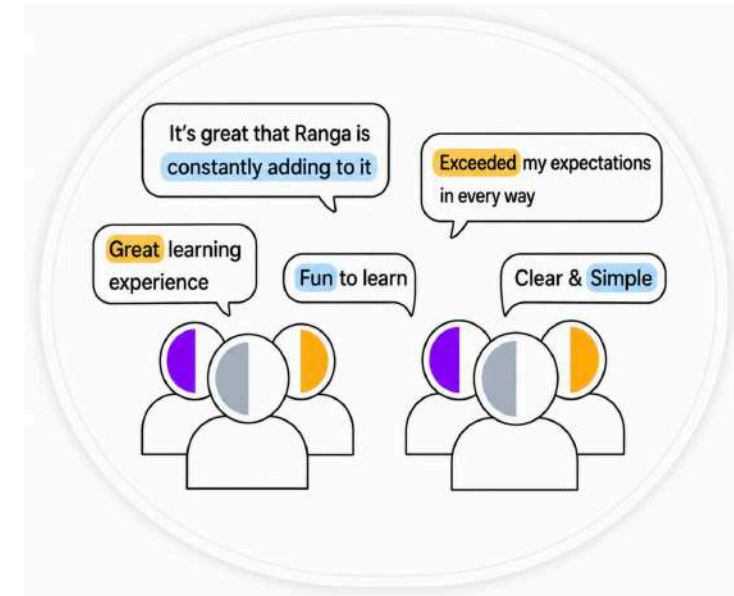


What Learners Are Saying

in28minutes: Trusted by millions of learners worldwide

What Learners Say:

- ★★★★★ - "Hello Ranga! thank you for the amazing weekend course. I am one step closer to my goal."
- ★★★★★ - "Great course! The instructor explains everything clearly and in a fun, engaging way."
- ★★★★★ - "Very interesting material, especially deep dive into Generative AI, LangChain. and ChatGPT."





in28minutes

Getting Started with Artificial Intelligence

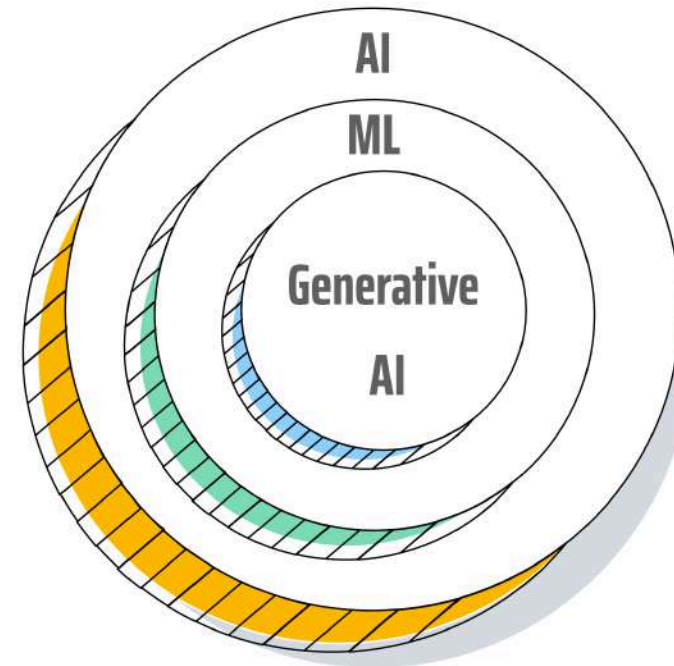
Section Overview - Introduction to AI, ML and Generative AI



Focus: What will we learn?

- What is Artificial Intelligence?
- What is Machine Learning?
- How does Machine Learning work?
- Machine Learning vs Traditional Programming
- What is Generative AI?
- How is Generative AI different from Traditional AI and ML?

Foundation: Build the core concepts needed for the rest of the course





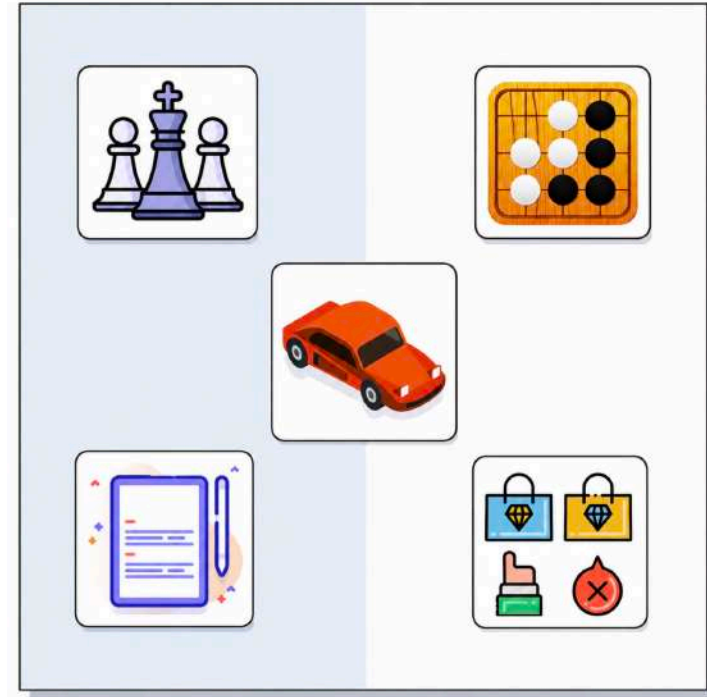
What is Artificial Intelligence?

Humans are great at:

- **Decision-Making:** Choose from multiple options
- **Visual Perception:** Recognize faces and objects
- **Language Understanding:** Understand written text
- **Speech Recognition:** Understand spoken language
- And a lot more..

Artificial Intelligence: Theory & development of computer systems able to perform tasks normally requiring human intelligence

- **Simplified Goal:** Make computers do things that normally require human intelligence





Are All AI Systems Equally Intelligent?

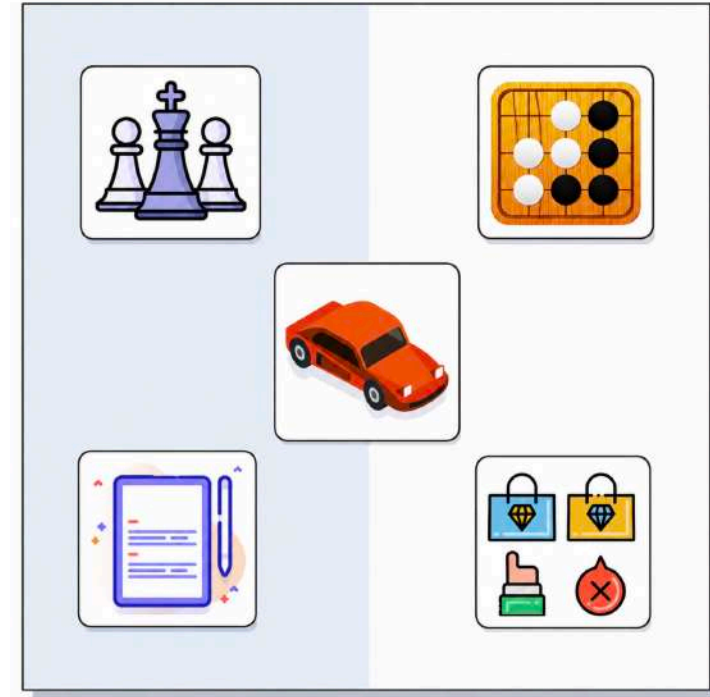
AI can have different levels of capability

Some AI systems are designed to:

- Perform **one specific task**
- Perform **many different tasks**

This leads to two broad types of AI:

- Strong AI
- Narrow AI





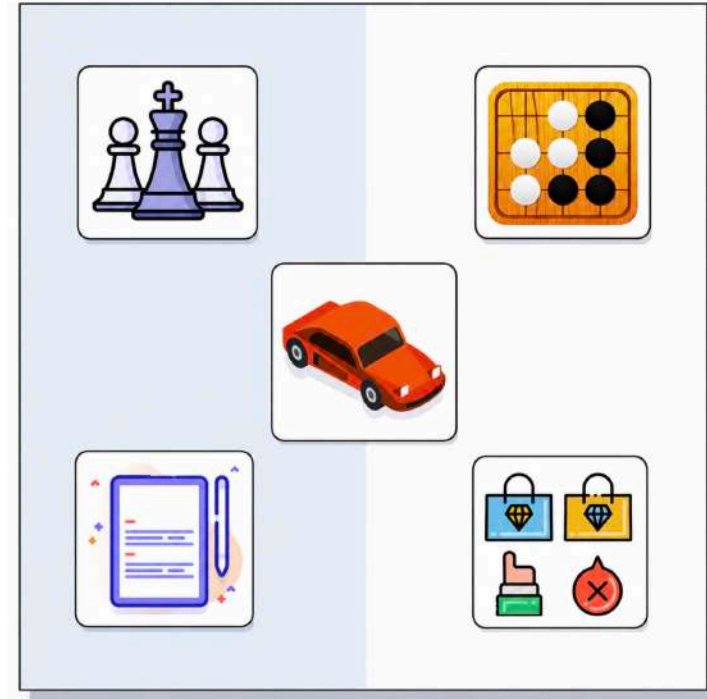
Understanding Types of AI - Strong AI

Strong AI: Goal is to create machines that are as intelligent as humans

Capabilities: Solve problems, learn, and plan for the future

- Expert at everything
- Make complex decisions
- Learns like a child

Also Known As: General AI or Artificial General Intelligence (AGI)





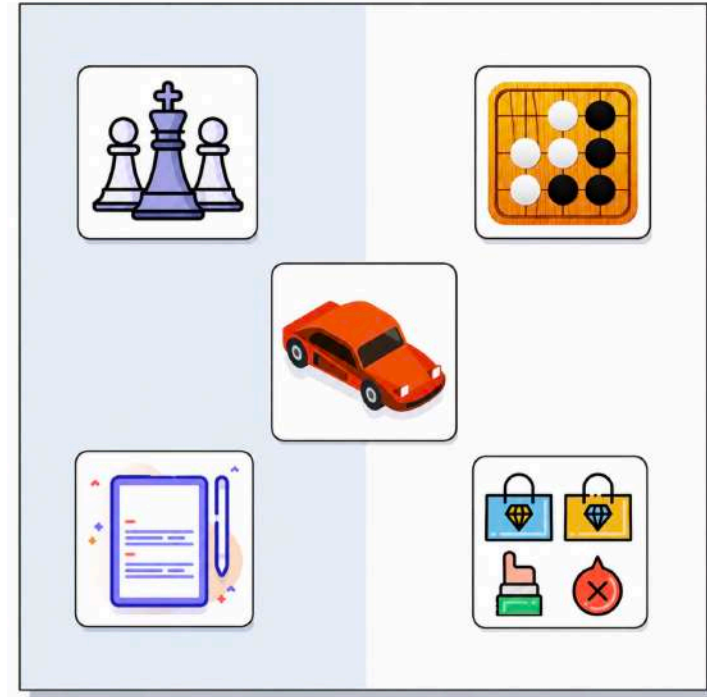
Understanding Types of AI - Narrow AI

Narrow AI: AI focused on a specific task

Examples:

- **Self-driving Car:** Can drive but cannot play chess
- **Chess App:** Can beat champions but cannot drive
- **Weather Predictor:** Knows weather but cannot write poetry

Also Known As: Weak AI





Getting Started with AI - Scenarios

Scenario	Solution
What is the Goal of AI?	Make computers do things that normally require human intelligence
Strong or Narrow AI: Build a system that can learn any game, write poetry, and drive a car like a human	Strong AI (General AI)
Build a computer system that focuses on a specific task such as self-driving, virtual assistants, or object detection	Narrow AI (or Weak AI)
The AI used in your smartphone to recognize your face	Narrow AI



Why are we talking a lot more about AI now?

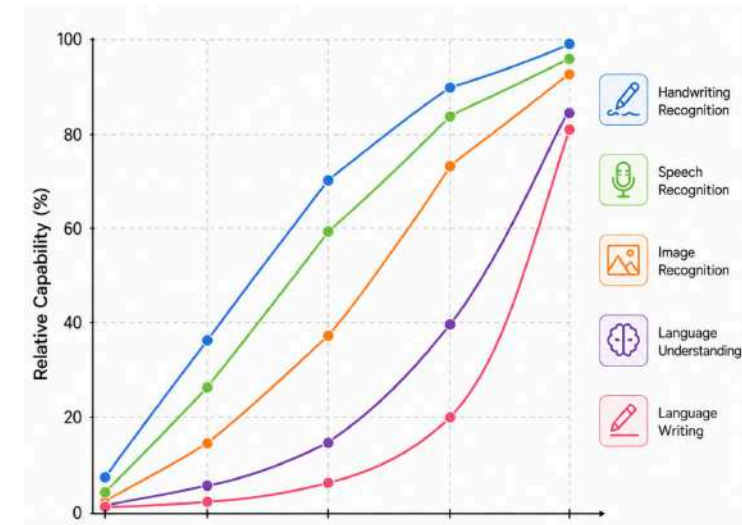
AI is Not New: Researchers have been working on AI since 1950s

Question: Why are we talking a lot more about AI now?

- **Answer:** AI capabilities improved dramatically in recent decades

Question: How can we measure progress in Artificial Intelligence?

- **Early Answer:** The **Turing Test**





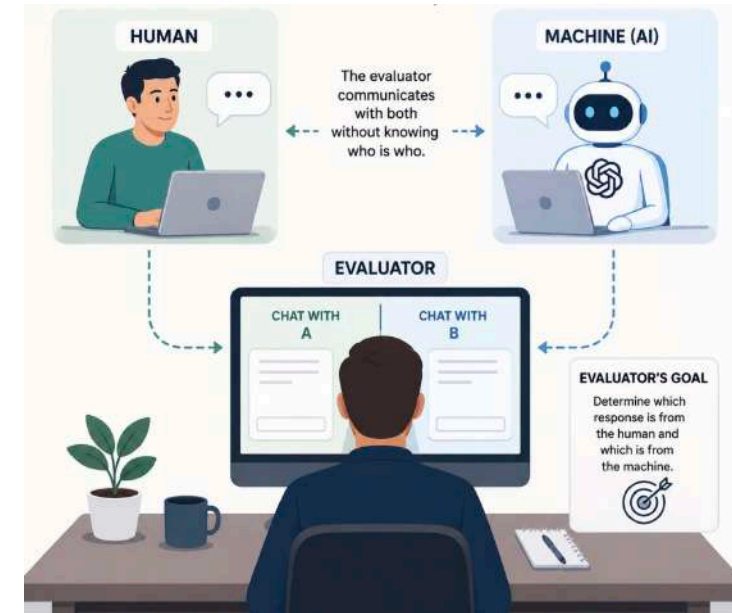
What is Turing Test?

Goal: Check if a AI system can communicate like a human

- 1: An evaluator chats with a human and a AI system
- 2: The evaluator asks questions through text
- 3: Both respond to questions
- 4: Evaluator tries to identify the machine

Early AI: AI systems struggled to imitate human conversation

Today: Advanced AI systems can often produce human-like responses





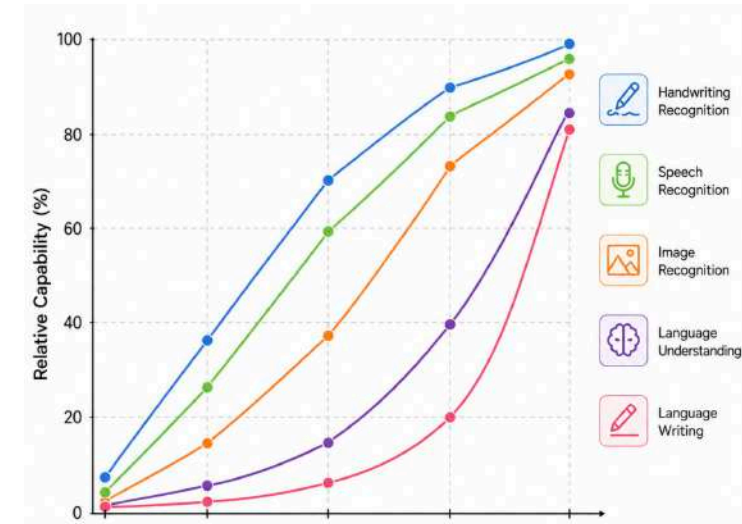
AI Has Made a Lot of Progress

AI Improved Rapidly: Across many areas

- **Image Recognition:** Recognizing object in images
- **Speech Recognition:** Convert spoken words into written text
- **Language Generation:** Large language models are capable of fluent writing
- And a number of others

How: How did this progress happen?

- Coming up!





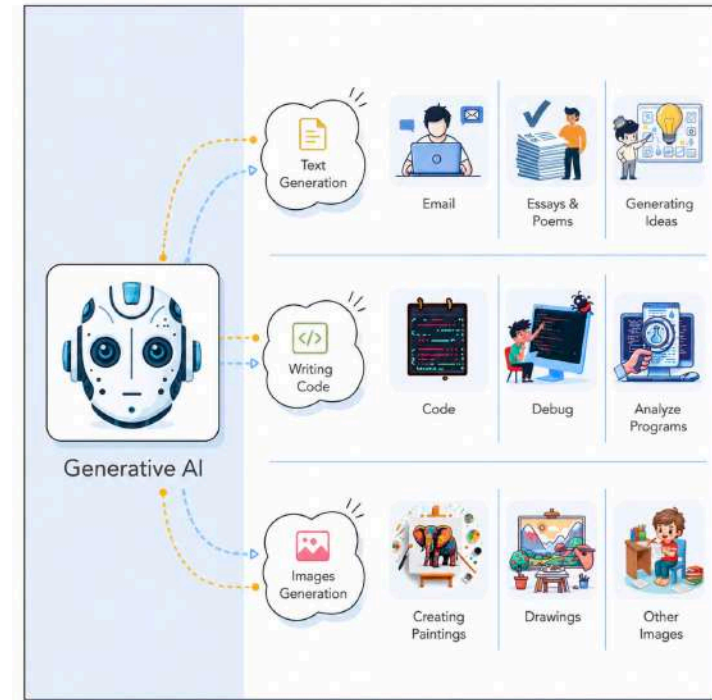
Playing with Generative AI

Checklist: Generate a list of items I need for a 15-day Everest Base Camp trek

- Update the list assuming I will stay at a tea house as part of a guided group

Multimodal: Let's create an image showing the atmosphere at a cricket match

- A vibrant, high-energy night scene at an Indian Premier League (IPL) cricket match. Focus on the electric atmosphere. Show fans wearing team jerseys, waving flags, and cheering loudly. Include digital scoreboards and a sense of celebration.



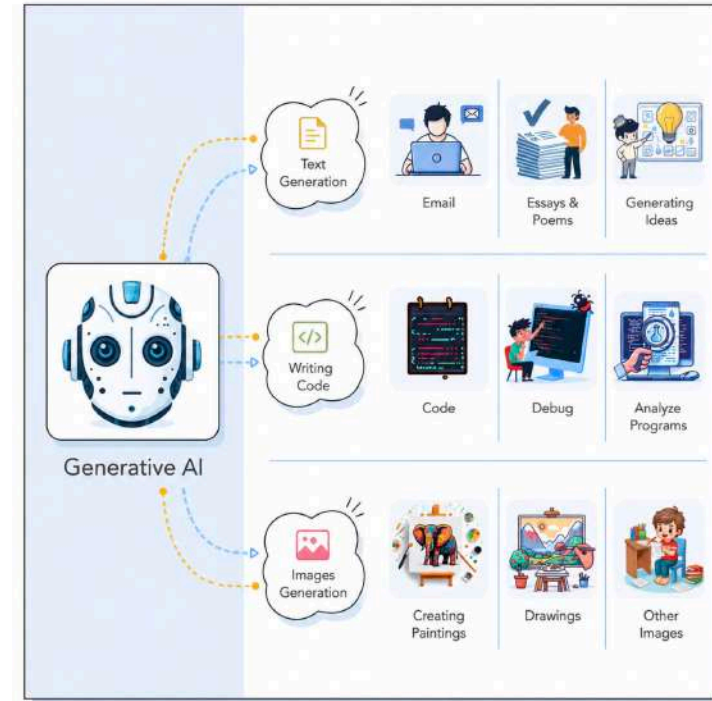


Playing with Generative AI - Observations

Powerful, Not Perfect: Can produce incorrect, biased, or inappropriate responses

- **Use with Judgment:** Outputs should be reviewed and verified
- **Still Valuable:** When used with the right expectations, Generative AI can be extremely useful

How does it work?: Let's start with Machine Learning and build our way to Generative AI





in28minutes

Getting Started with Machine Learning



How is Traditional Programming Done? [1/2]

Approach: Rule-based logic

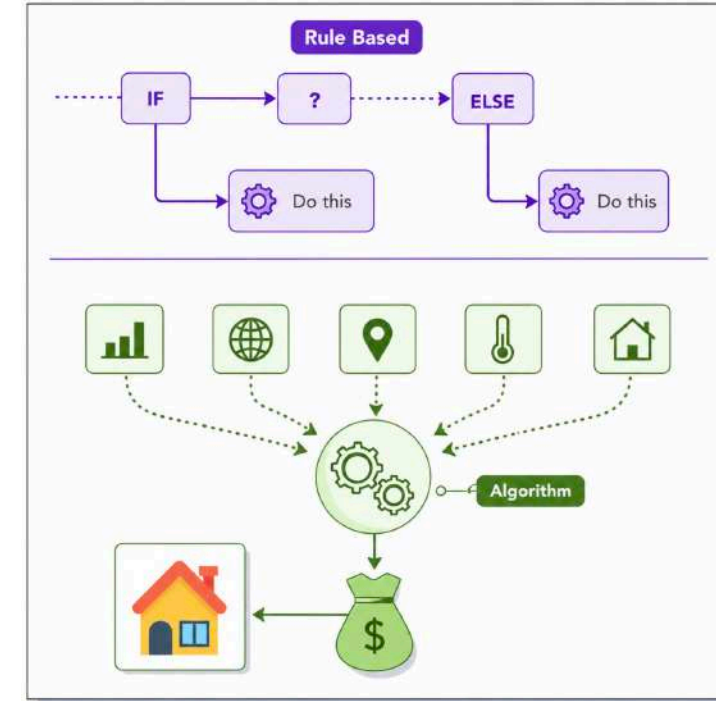
Mechanism: You explicitly tell the computer what to do

Flow:

- **Input:** Data
- **Program:** Rules such as IF this DO that
- **Output:** Result

Example: Calculating tax

- **Rule:** If income > 50k, tax is 20%





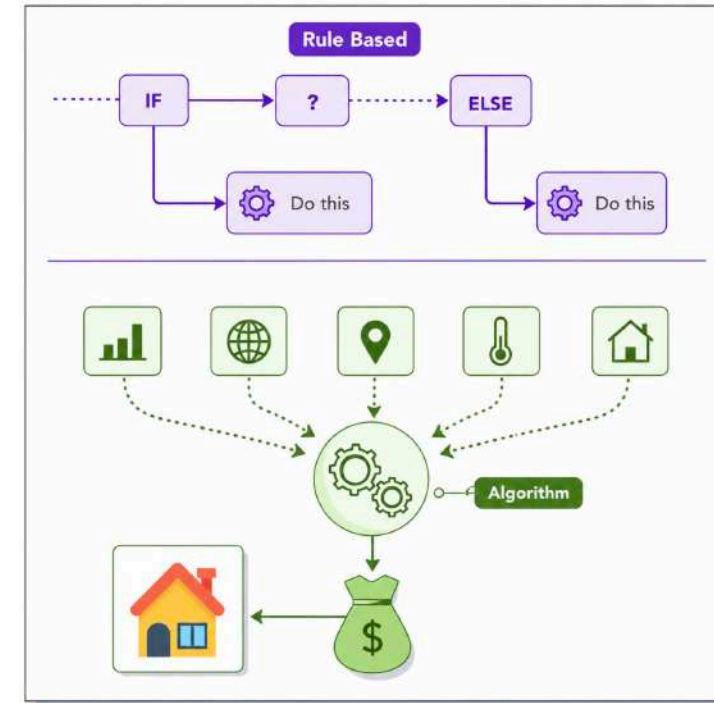
How is Traditional Programming Done? [2/2]

Example: Predicting house price

Approach: You design an algorithm that considers all key factors

- Location
- Home size
- Age
- Condition
- Market

Result: The computer does exactly what you tell it, nothing more and nothing less





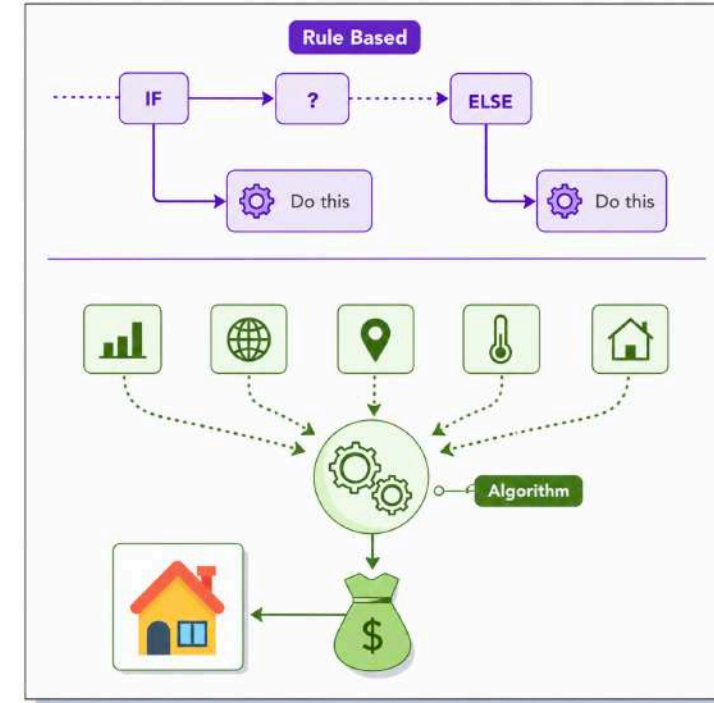
What is the Limitation of Traditional Programming?

Traditional Programming: Input --> Program
(Rules or Algorithm) --> Output

Think About This: Can you write the rules to recognize a cat?

- **Example:** If pixel 1 is light and pixel 2 is dark, and so on
- **Reality:** Almost impossible to hand-code such kind of rules

Result: We hit a wall with traditional coding





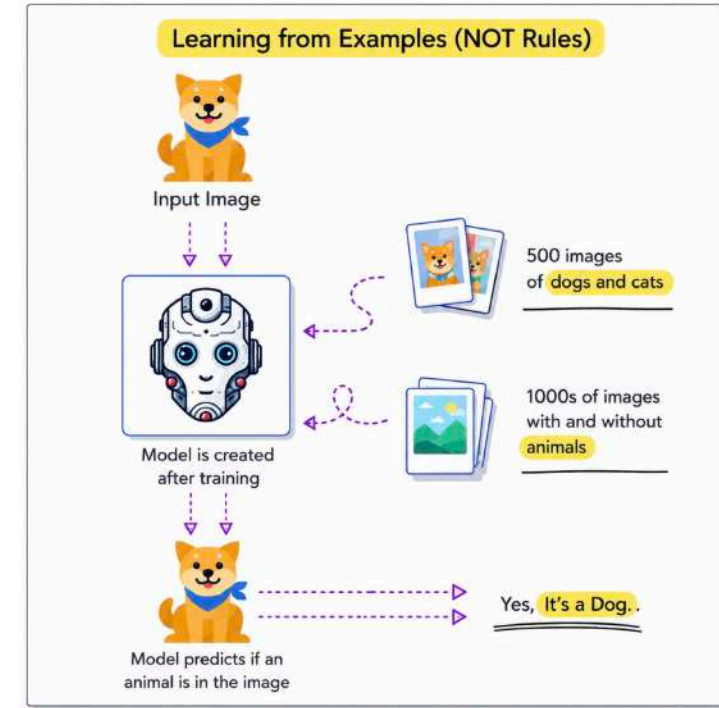
How Does Machine Learning Work?

What Happens?: ML learns from data

Machine Learning: Give examples, ML finds patterns, then predicts the outcome

Example:

- **House Price Prediction:** Learn from past sales data





Machine Learning - Simplified Workflow

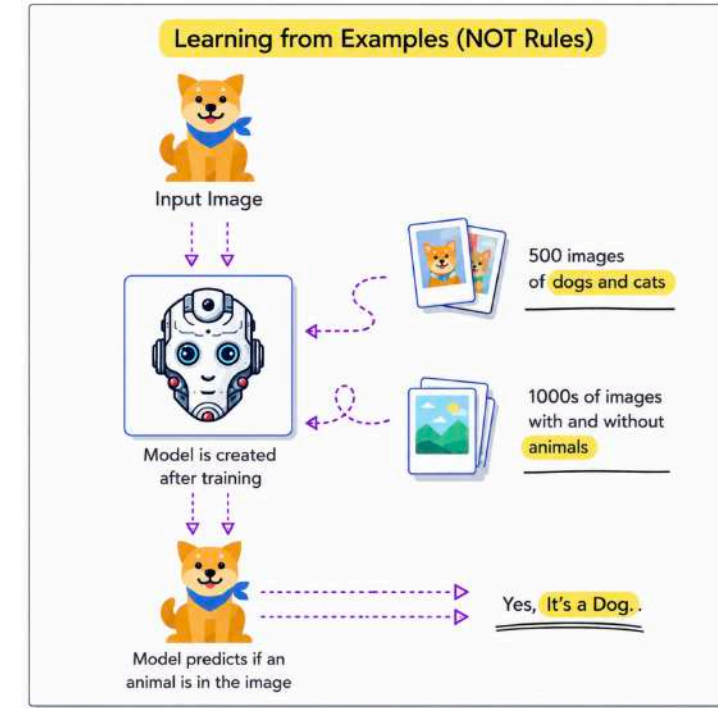
Step 1 - Obtain Data: Provide examples

- **Example:** Photos of cats, dogs and other animals with labels

Step 2 - Train Model: The system analyzes images to find patterns

- **Learned Patterns:** Shapes, edges, and textures

Step 3 - Predict: Give it a new photo and get a prediction





Traditional Programming vs Machine Learning

Traditional Programming

Programmer writes the rules (program)

Input → Program (Rules or Algorithm) → Output

Works well when rules are known

Example: Tax calculation

Example: Payroll processing

Machine Learning

Model learns the rules from data

Input + Expected Outputs → Training → Model

Works well when rules are difficult to define

Example: House price prediction

Example: Spam email detection



Machine Learning Fundamentals - Scenarios

Scenario	Solution
Developer writes logic to tell the computer what to do	Traditional Programming
System finds patterns from data that humans cannot explicitly define	Machine Learning
Category of AI that focuses on learning from data	Machine Learning
How is ML different from traditional programming?	Traditional Programming uses rules, ML uses examples



in28minutes

Getting Started with

Machine Learning Stages



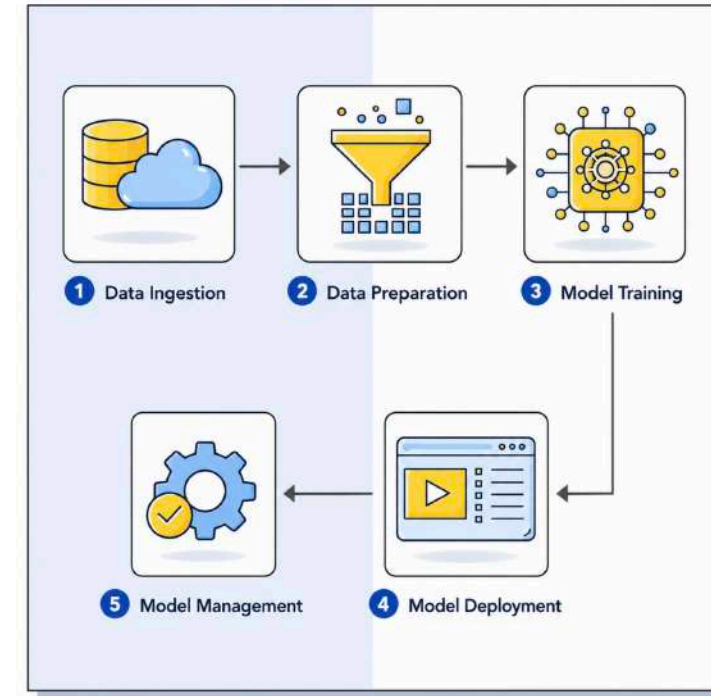
ML Lifecycle: Data Related Stages

Data Ingestion: Collect raw data from different sources

- **Example:** Streaming user activity from a mobile app
- **Example:** Pulling transaction logs from a sales system

Data Preparation: Clean and format data for training

- **Example:** Remove duplicates
- **Example:** Handle missing values in patient health data





ML Lifecycle: Model Related Stages

Model Training: Use data to train the model

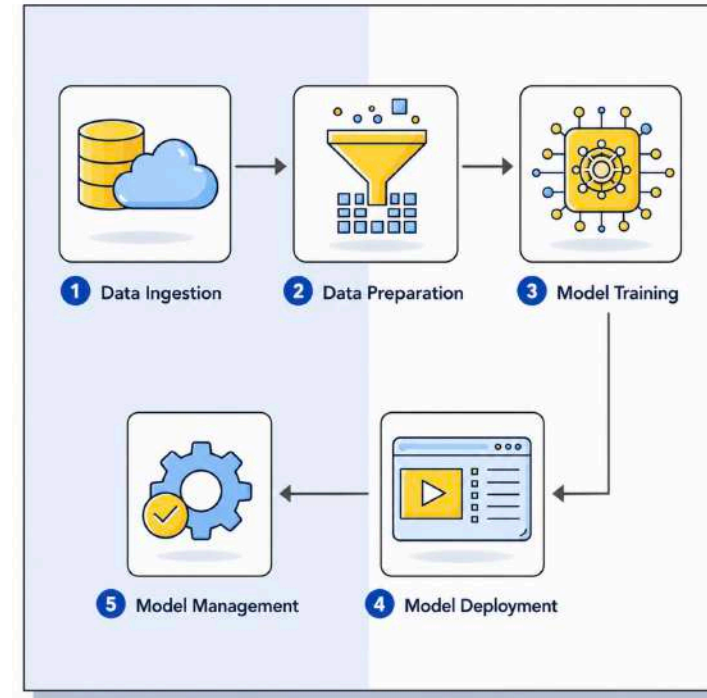
- **Example:** Train a fraud detection model with transactions
- **Example:** Train a price prediction model for used cars

Model Deployment: Make the trained model available to users

- **Example:** Deploy a chatbot in production (Inference)

Model Management: Monitor and maintain the model after deployment

- **Example:** Update the model as new financial data comes in (Re-Training)





ML Lifecycle Recap: End-to-End Example

Use Case: ML model to detect spam in email

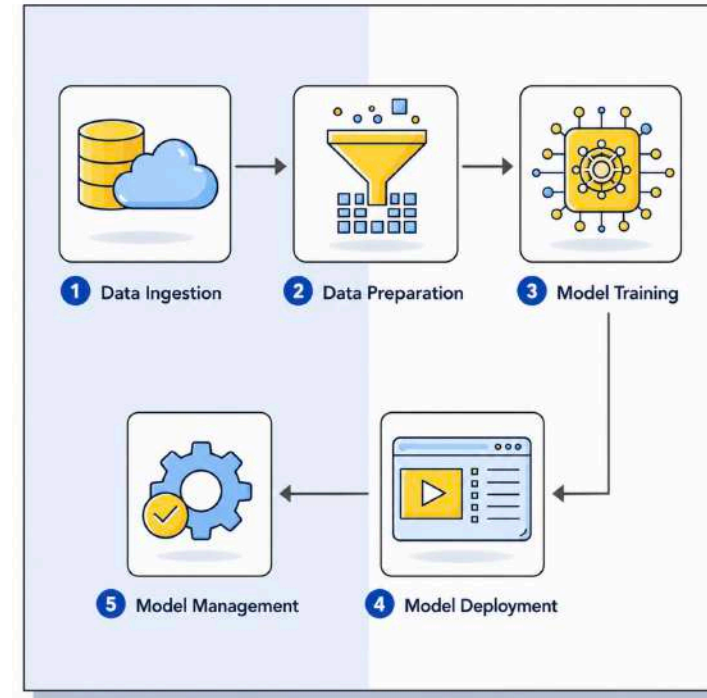
1: Collect email history

2: Prepare labels

3: Train with examples

4: Deploy into email platform

5: Manage performance





Identify the Stage in ML Lifecycle - 1

Activity	Stage
Pulling product transaction logs from a retail store's log store	Data Ingestion
Streaming real-time user activity from a mobile app	Data Ingestion
Removing duplicate customer records from the training dataset	Data Preparation
Filling in missing values in patient data using averages	Data Preparation
Labeling emails as spam or not spam for training	Data Preparation
Using thousands of labeled housing records to teach a model to predict price	Model Training
Training a model to detect credit card fraud using past transaction data	Model Training



Identify the Stage in ML Lifecycle - 2

Activity	Stage
Making your trained model available via an API to a web application	Model Deployment
Deploying an image classification model into a mobile app	Model Deployment
Monitoring how accurate your chatbot is after deployment	Model Management
Updating your model with recent data after noticing performance drop	Model Management
Creating a dashboard that displays model inference accuracy trends	Model Management



in28minutes

Getting Started with Generative AI



Generative AI - Generates New Content

Goal: Generate New Content

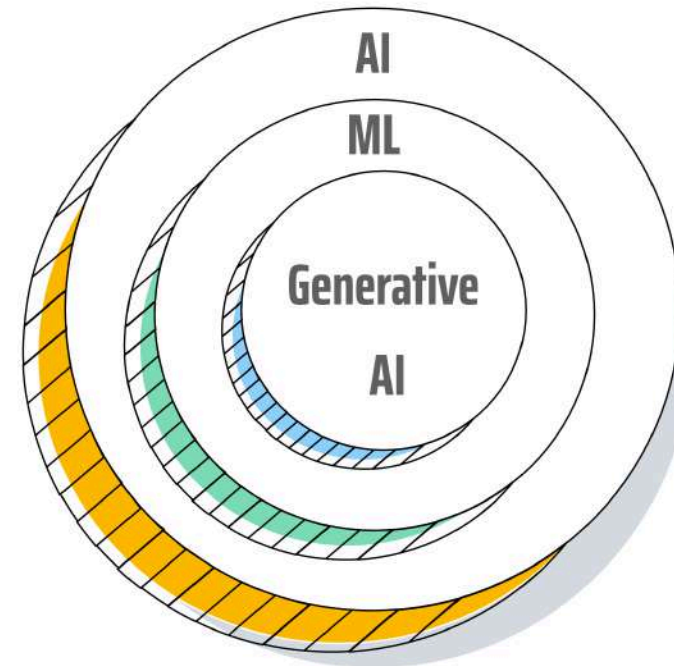
- Instead of making predictions, Generative AI focuses on creating new content

Text Generation: Writing e-mails, essays & poems. Generating ideas.

Code Generation: Generate code snippets. Create unit tests.

Media Creation: Design images. Create instructional videos.

And a lot more features!



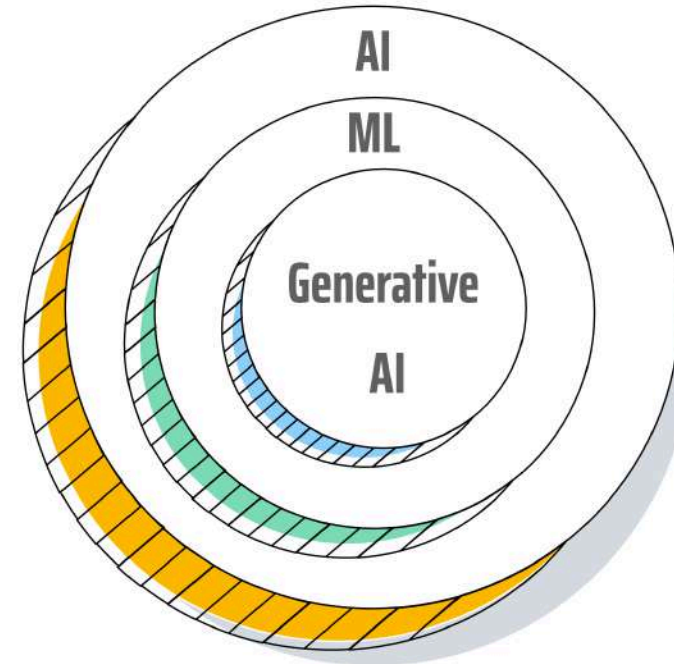


Generative AI - How is it different?

Artificial intelligence (AI): Create machines that simulate human-like intelligence and behavior

Machine learning (ML): A subset of AI where machines learn from data

Generative AI: AI that learns patterns from data to **generate new content**





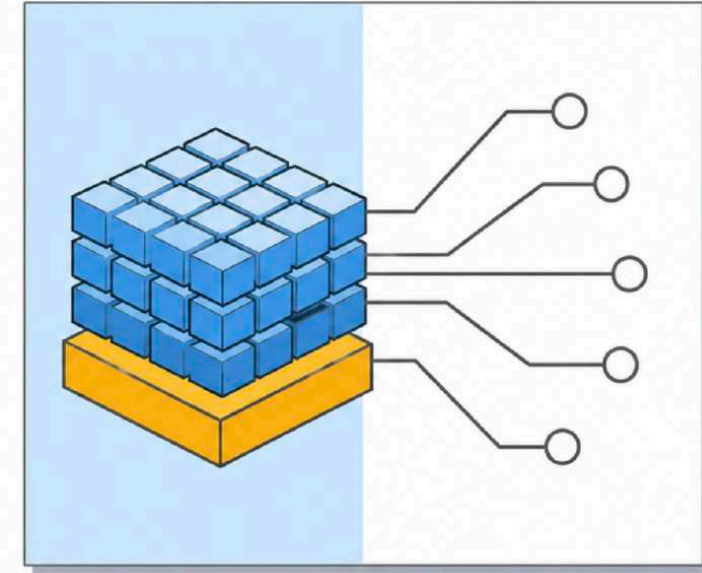
Foundation of Generative AI - The Foundation Models

Foundation Model: The core model powering many generative AI applications

- **One Model, Many Tasks:** Answer questions, summarize, generate code, create content
- **Examples:** GPT, Gemini, Claude, Llama

Pre-trained: Trained on massive amounts of text, code, images, and other data

- **Expensive to Train:** Requires significant compute, time, and data
- **Easy to Reuse:** Build applications without training a model from scratch



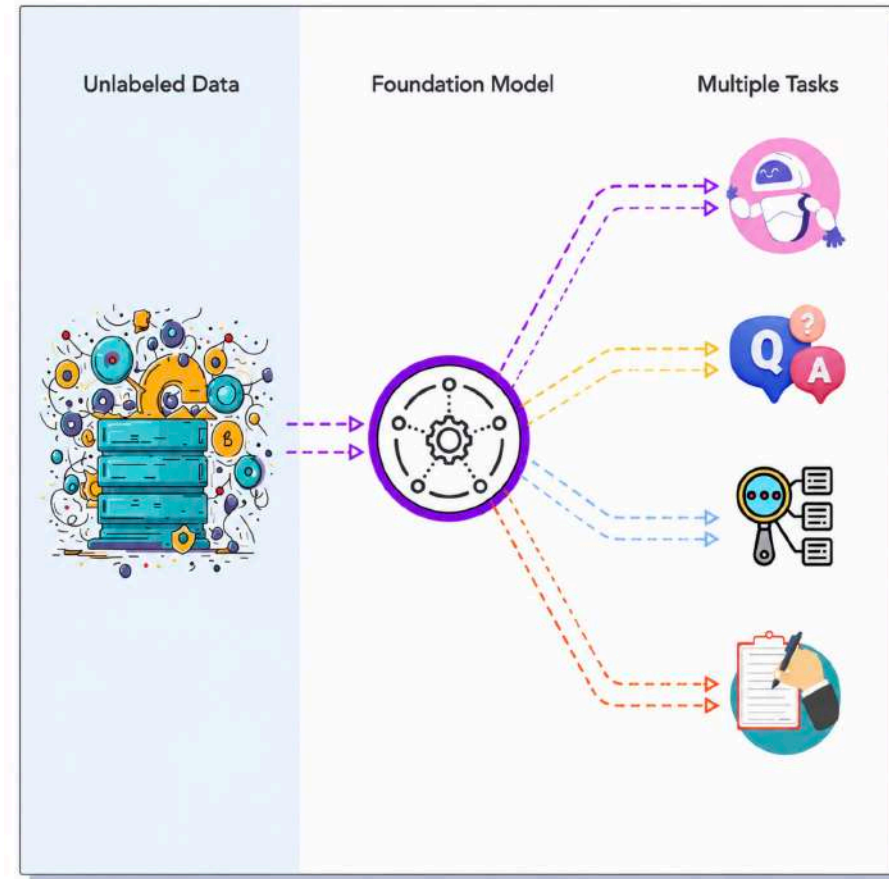


Training Foundation Models is Complex

Complex Training Process: Training is computationally intensive and complex

- **Massive Datasets:** Books, code, web pages, images, captions, and more
- **Specialized Hardware:** GPUs, TPUs, distributed storage, and parallel processing
- **Takes Time:** Training runs for weeks or months

Outcome: Broad general knowledge - understands how humans write, speak and code





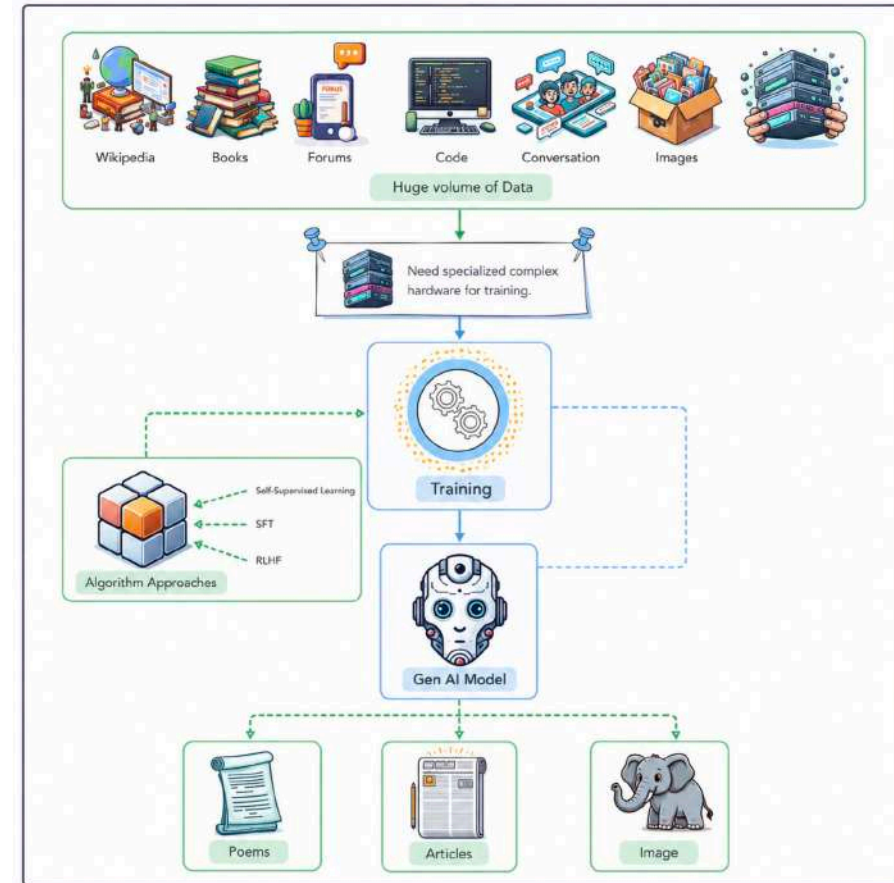
Foundation Models Are Multi Task Models

Traditional ML Models:

- Needed task specific training
- Multiple tasks => Multiple trainings => Multiple models

Foundation Models: Multi Task Models

- Trained once (called pre-training)
- Same model can be used for multiple tasks
 - Chatbot, Classification, Summarization, etc.
- Some models are multi modal as well: Text, video, audio, image...





Text Foundation Models are called Large Language Models

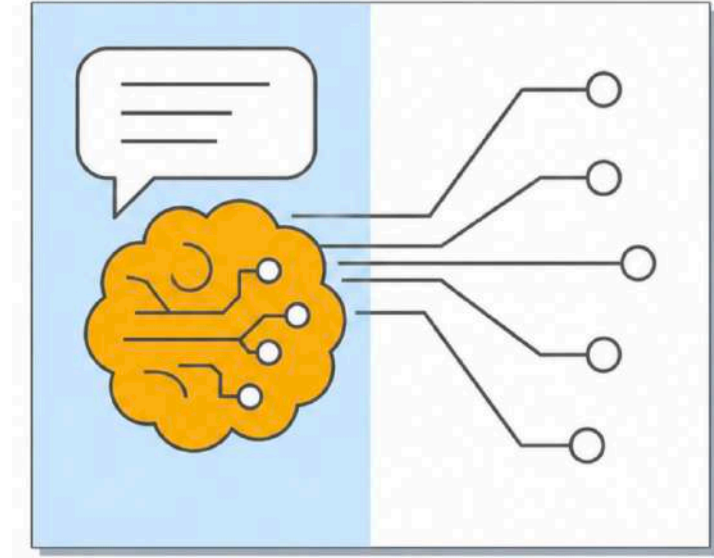
Trained on **huge volumes** of diverse text data

- Books, articles, websites, code, and more

Learn to **predict the next token** in a sequence

- Example: "The cat sat on the ..."
- Model gives probabilities:
 - "mat" (40%), "table" (20%), "chair" (20%), "moon" (10%)
- It usually picks the most likely token — but we can tune settings to make it more creative
 - e.g., using the temperature parameter

Use cases: Chat, summarization, translation, Q&A, ..





Diffusion Models: Turning Noise into Art

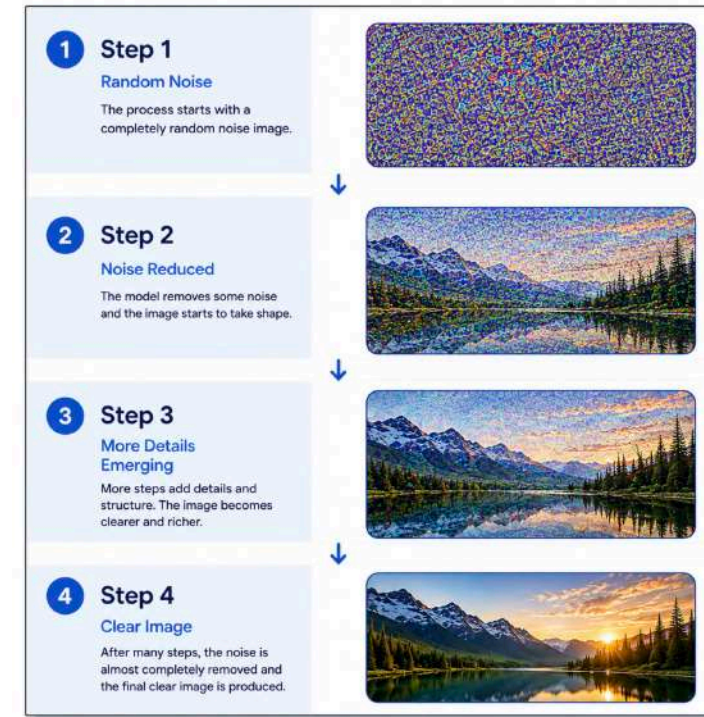
Diffusion models: Great at generating images, audio, and video

How they work:

- Start with **random noise**
- Iteratively **refine it** step-by-step
- Result: **Structured, realistic output**

Used for:

- Creating AI art and realistic pictures from text prompts
- Generating life-like human voices or music
- Producing short AI-generated videos





Foundation Models - Scenarios

Scenario	Solution
Models trained on large, diverse datasets across domains. Adaptable to a wide variety of tasks.	Foundation Models
Text foundation models trained on huge volumes of diverse text data.	Large Language Models
Models that generate images, audio, or video starting with random noise and refining step-by-step	Diffusion Models



in28minutes

Going Deeper into Machine Learning



Section Overview - Getting Deeper into Machine Learning

Focus: What will we learn?

- What are features and labels?
- Why do we need three types of datasets?
- What is the difference between regression, classification, and clustering?
- What is the difference between supervised, unsupervised, and reinforcement learning?
- How do neural networks and deep learning enable machines to learn?
- What enabled the Deep Learning revolution?

Foundation: Build a deeper understanding of how Machine Learning works





Machine Learning - Features and Label

Input: Give inputs to a model

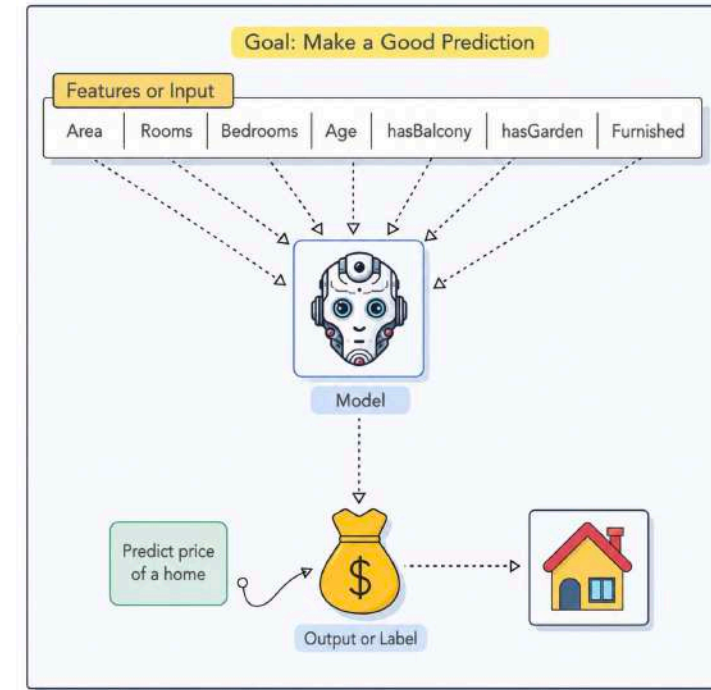
- **Inputs:** Called features

Output: Model returns the prediction

- **Prediction:** Called label

Example: House price prediction model

- **Label:** price
- **Features:** area, rooms, furniture, floor, age, garden





Machine Learning - Features and Labels - Examples

Used Car Price Prediction:

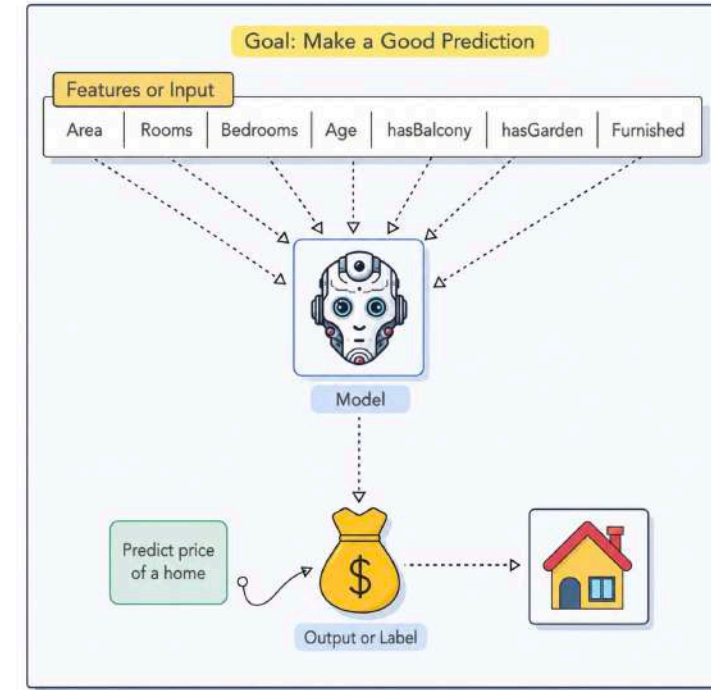
- **Label:** price
- **Features:** manufacturer, year, model, age, condition, cylinders, location

Spam Email Classification:

- **Label:** isSpam
- **Features:** sender, subject, content

Grant a Loan Model:

- **Label:** shouldWeGrantALoan
- **Features:** doesOwnCar, creditScore, isMarried, doesHaveChildren, totalIncome, totalCredit



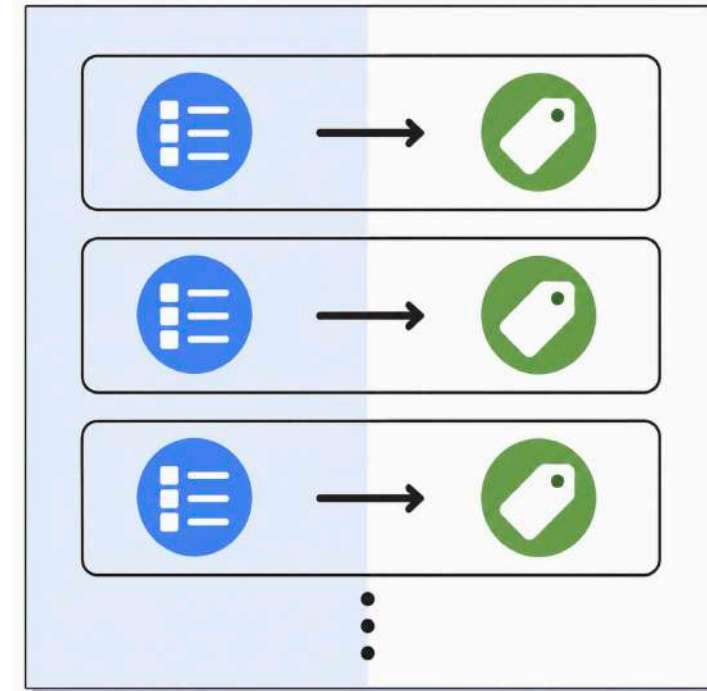


Understanding Labeled Data

Labeled Data: Data tagged with labels

Examples:

- House sales data (area, rooms, furniture, floor, age, garden > price)
- Images tagged as cat or dog
- Customer reviews labeled positive, negative, neutral
- Emails marked as spam or not spam



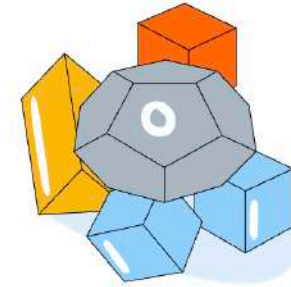


Understanding Unlabeled Data

Unlabeled Data: Raw data with no predefined tags

Examples:

- Text files
- Documents
- Music files
- Web traffic logs
- A folder of random photos with no captions or tags



Unstructured



Video



Image



Audio etc..



Identify Labeled or Unlabeled Data

Scenario	Labeled or Unlabeled
You have a dataset of customer reviews marked as positive, neutral, or negative	Labeled
You're analyzing a folder of selfies, but none are labeled with names	Unlabeled
An e-commerce platform tags products as electronics, clothing, or home	Labeled
You receive thousands of support emails with no categorization	Unlabeled
A medical dataset marks X-ray images as normal or abnormal	Labeled
Network logs are collected but not tagged with user or activity type	Unlabeled



Machine Learning Helps with Variety of Problems

Predict the price of a used car

Predict the no of vehicles using a highway

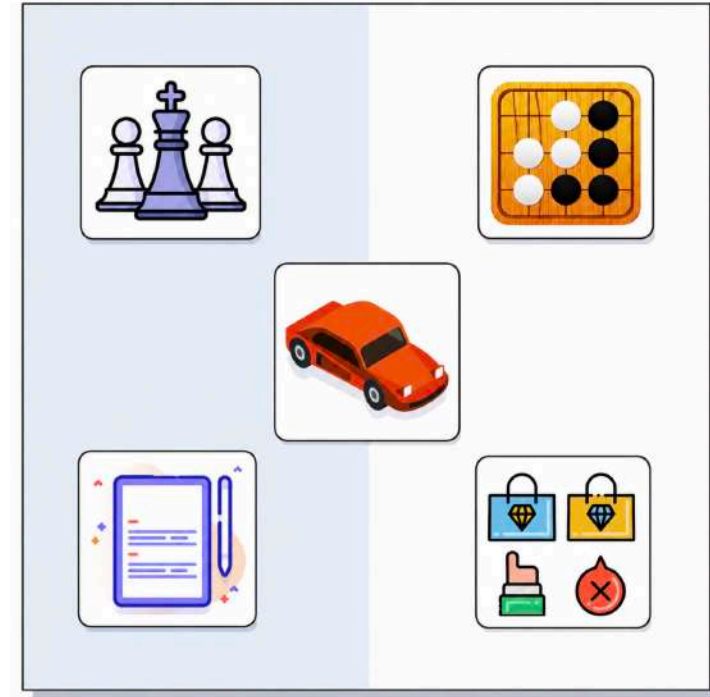
Forecast tomorrow's rainfall

Detect whether an email is spam

Decide whether to grant a loan

Divide customers into groups based on purchasing behavior

Detect anomalies





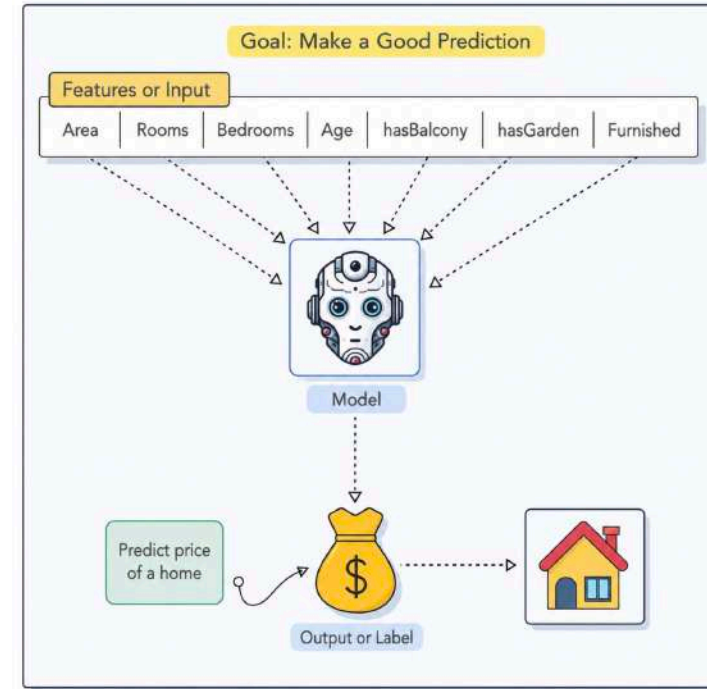
Machine Learning Problem Type 1 - Regression

Regression: Label is a numeric value

- The label can take a wide range of values

Problem Examples:

- Predict the price of a used car
- Estimate the price of a house
- Predict sea level
- Predict the number of vehicles using a highway
- Forecast tomorrow's rainfall





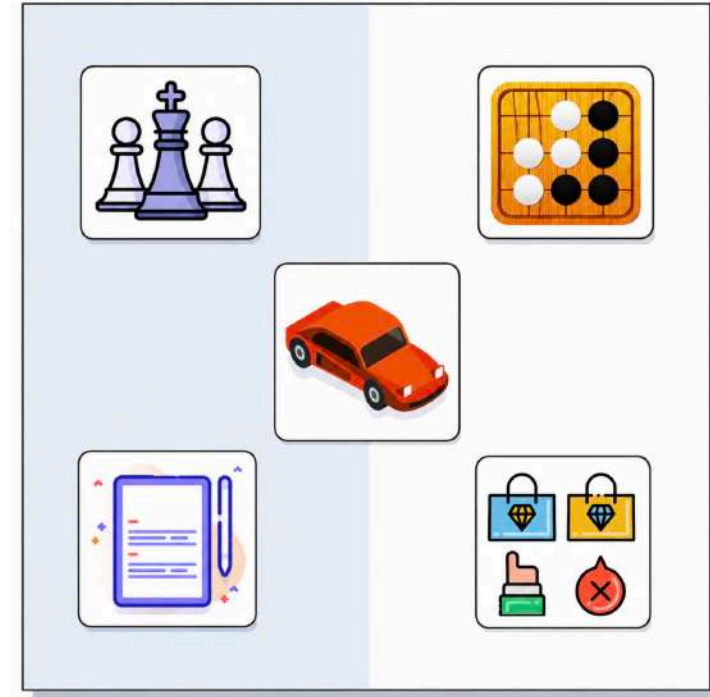
Machine Learning Problem Type 2 - Classification

Classification: Label belongs to a limited set of categories

- **Examples:** Yes or no, 0 or 1, Approved or rejected, Spam or not spam, Type 1, type 2, or type 3

Problem Examples:

- Detect whether an email is spam
- Decide whether to grant a loan
- Determine the type of cloud
- Predict whether it will rain today
- Identify if a medical image is normal or abnormal





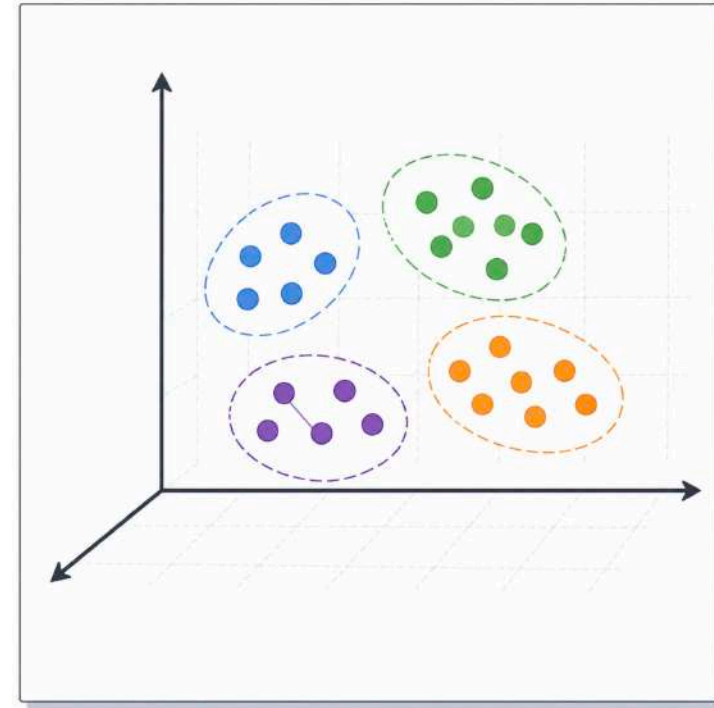
Machine Learning Problem Type 3 - Clustering

Clustering: Group similar items based on their features

- **No Labels:** Groups are not provided beforehand (Unlabeled Data)

Examples:

- Group customers based on purchasing behavior
- Group similar products based on their features
- Group similar users based on their interests





Machine Learning Fundamentals - Scenarios

Scenario	Solution
Categorize into features and labels for house price prediction: price, area, rooms, age	price is label, others can be features
Categorize into features and label for used vehicle price prediction: manufacturer, year, model, age, condition, cylinders, location, price	price is label, others can be features
Categorize: Used Car Price Prediction	Regression
Categorize: Spam Email Identification	Classification
Categorize: Predict amount of rainfall in the next year	Regression
Categorize: Predict if it will rain tomorrow?	Classification
Categorize: Should we grant a loan?	Classification
Categorize: Divide customers into groups	Clustering



in28minutes

Machine Learning

Learning Types



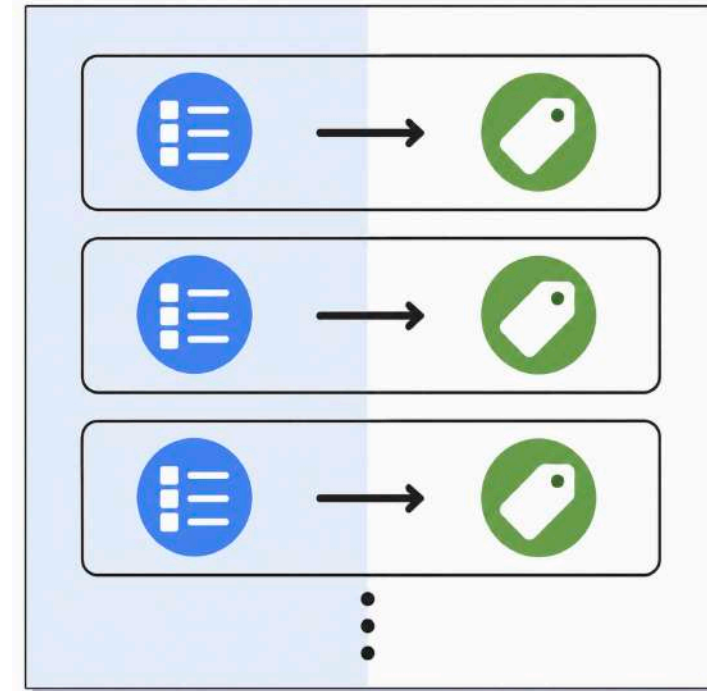
What is Supervised Learning?

What It Learns: Patterns from historical data so it can predict new data

- **What It Uses:** Labeled data
- **Labeled Data:** Inputs (Features) + Output (or label)

Types: Regression and Classification

- **House Price Prediction:** Based on area, rooms, furniture, floor, age, garden
- **Loan Approval:** Based on doesOwnCar, creditScore, isMarried, doesHaveChildren, totalIncome, totalCredit
- **Detect Spam Email:** Based on sender, subject, content





What is Unsupervised Learning?

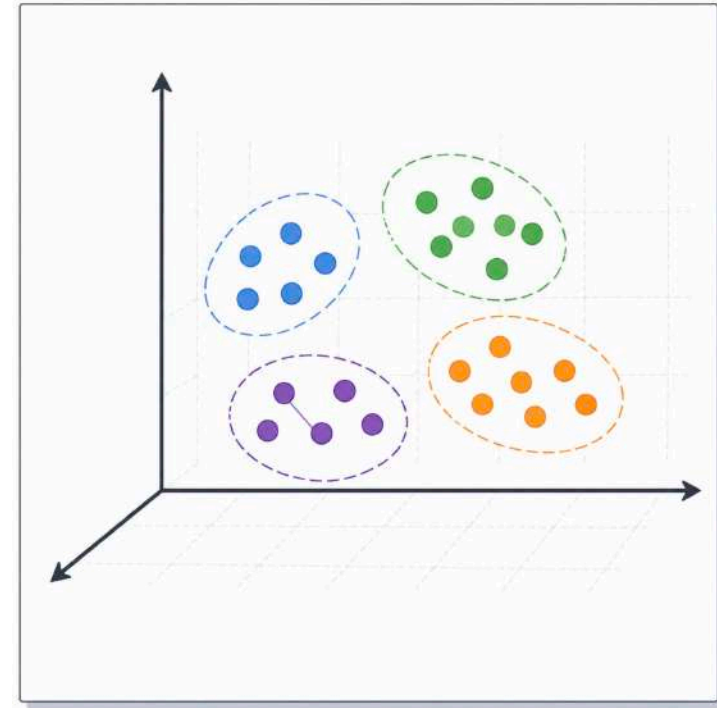
What It Does: Organizes data into clusters based on similarities

- **What It Uses:** Unlabeled data
- **Unlabeled Data:** No predefined tags or labels

Example Problem Type: Clustering

- **Photo Libraries:** Organize images by similar faces
- **Customer Groups:** Group customers based on purchase history

Think: Here is a mess, can you make sense of it?





Supervised vs Unsupervised Learning

Aspect	Supervised Learning	Unsupervised Learning
Data	Labeled	Unlabeled
Goal	Predict known outcomes	Discover hidden patterns
Examples	Email classification, fraud detection	Customer segmentation, anomaly detection
Output	Specific prediction or labels	Grouping or Anamolies



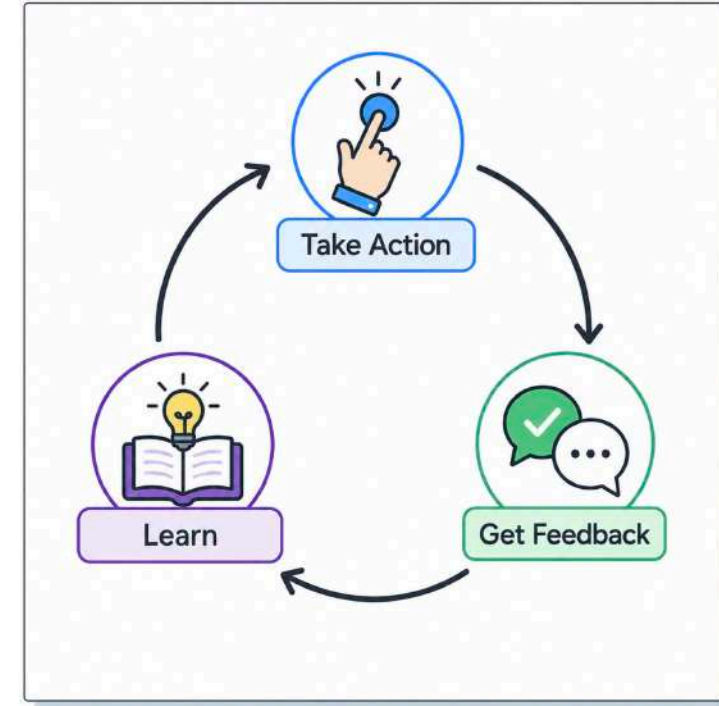
How Does a Child Learn to Walk?

Learning Style: A child learns by doing, not by reading instructions

■ **Process:**

- Try walking
- Stumble
- Get up again
- Try again

Feedback: Applause, encouragement, or falling shapes learning





What is Reinforcement Learning

Definition: Learning by doing and getting feedback

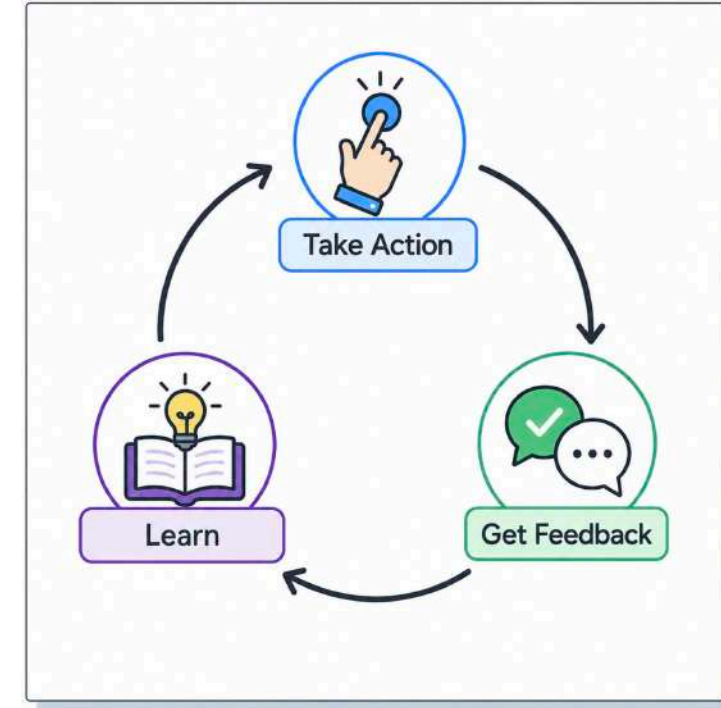
- **Mechanism:** The model takes actions, gets rewards or penalties, and adjusts

Examples:

- **Chess AI:** Learns by playing thousands of games
- **Robot Arm:** Learns by experimenting with motions

Best For: Decision-making in dynamic environments

Think: Try, learn, improve





Supervised vs Unsupervised vs Reinforcement Learning

Scenario	Learning Type	Why?
Predicting house prices using area, location, and number of rooms	Supervised	We have labeled data with known prices to train the model
Grouping customers into segments based on buying behavior	Unsupervised	No predefined labels, the model finds hidden patterns
A robot learning to walk by trial and error	Reinforcement	Learns through feedback, rewards for walking, penalties for falling
Email classification into spam and not spam	Supervised	The model is trained on emails that are already labeled
A drone navigating through an obstacle course by learning from collisions	Reinforcement	Learns a strategy through repeated trials and feedback
A game-playing AI learning how to win through experience	Reinforcement	Learns by receiving points or penalties based on actions taken



in28minutes

Machine Learning

Data is the Key

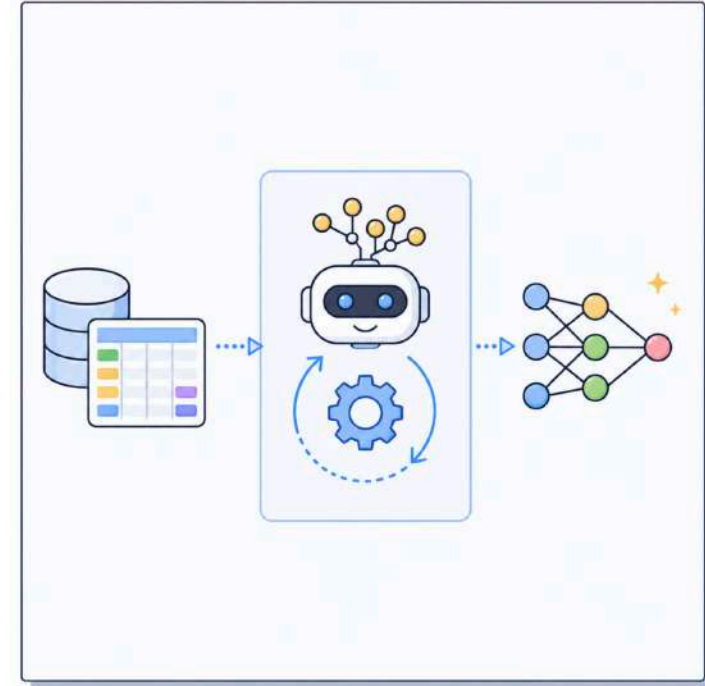


How do you Test the Model?

Question: How do we know whether a model has learned correctly?

Challenge: A model might memorize the training data instead of learning patterns

Solution: Evaluate the model using data it has never seen before





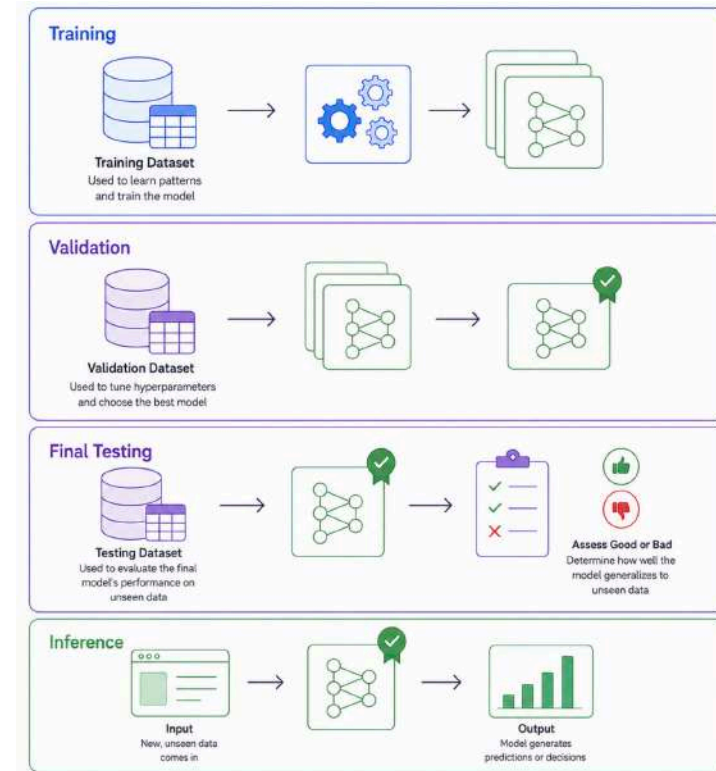
Understanding ML Dataset Types [1/2]

Dataset:

- **Features:** Inputs used to make a prediction
- **Label:** Expected output or prediction

Dataset Types: 3 key types

- **Training Dataset:** Used to train the model
- **Validation Dataset:** Used to compare models and choose the best model
- **Testing Dataset:** Used for final evaluation before deployment





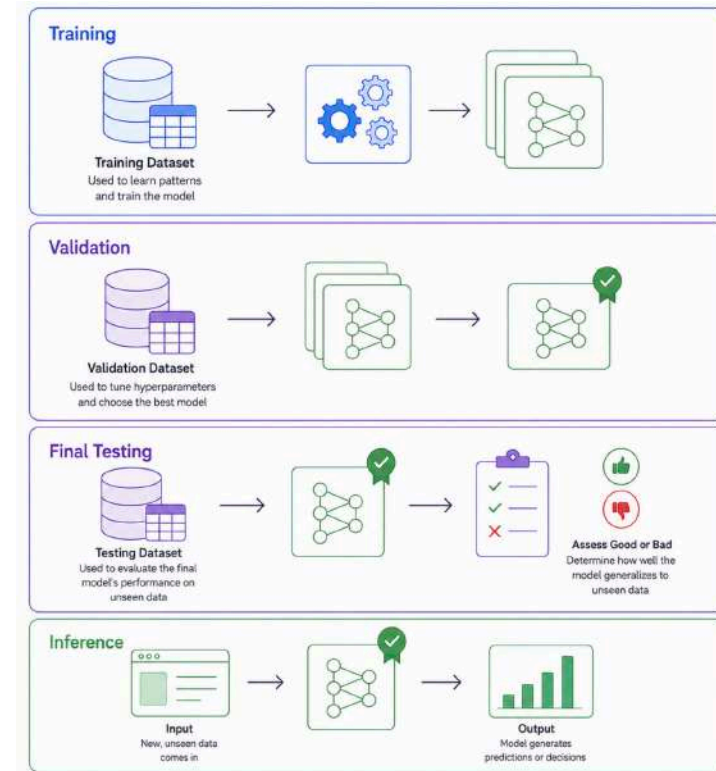
Understanding ML Dataset Types [2/2]

Training: Create a model using the training dataset

Validation: Compare models and choose the best model using the validation dataset

Testing: Evaluate the final model using the testing dataset

Inference: Use the final model to make predictions in production





ML Stages and Terminology - Scenarios

Scenario	Solution
Terminology: Using model to do predictions in production	Inference
Terminology: The process of creating a model	Training
Terminology: Dataset used to (train) or create a model	Training Dataset
Terminology: Dataset used to choose the best model	Validation Dataset
Terminology: Dataset used to do final testing before deployment	Testing Dataset



in28minutes

Machine Learning

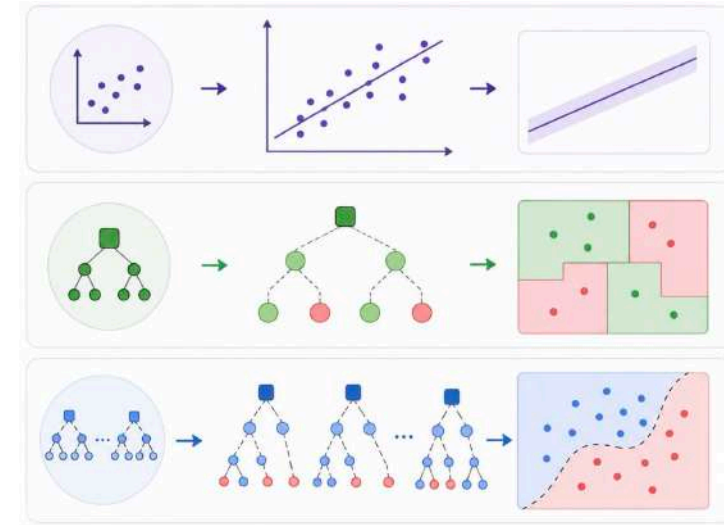
How Does Learning Happen?



Early Machine Learning Used Statistical Techniques

Early Machine Learning: Learned patterns using mathematical and statistical techniques

- **Linear Regression:** Learns the best line that fits the data
- **Decision Trees:** Learn a sequence of simple yes or no questions
- **Random Forests:** Combine many decision trees for better predictions



Challenge: These approaches can struggle with very complex patterns



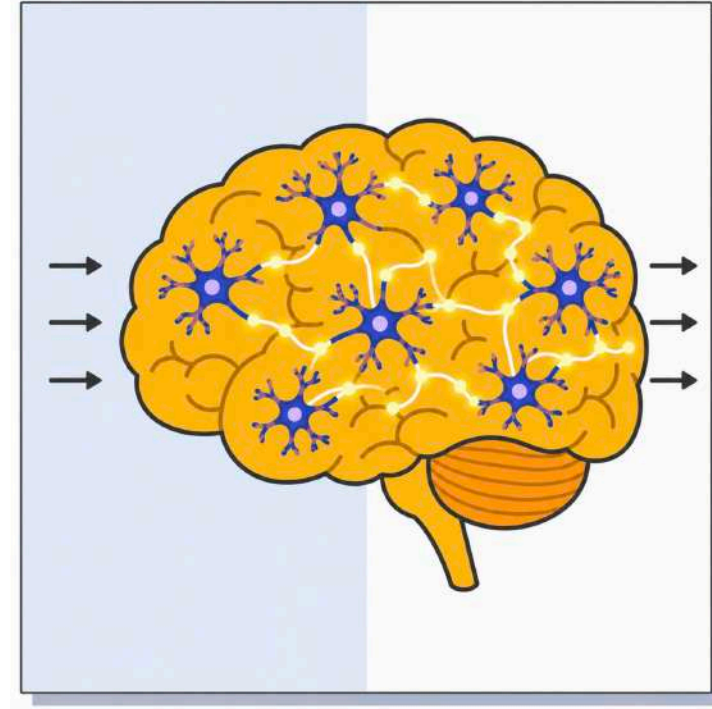
How Does Human Brain Learn?

Brain Structure: Billions of interconnected nerve cells called neurons

Learning: Connections between neurons change with experience

- **Frequently used connections:** Become stronger
- **Rarely used connections:** Become weaker
- **In Short:** Learning involves changing neural connections

Question: Can machines learn using a similar idea?





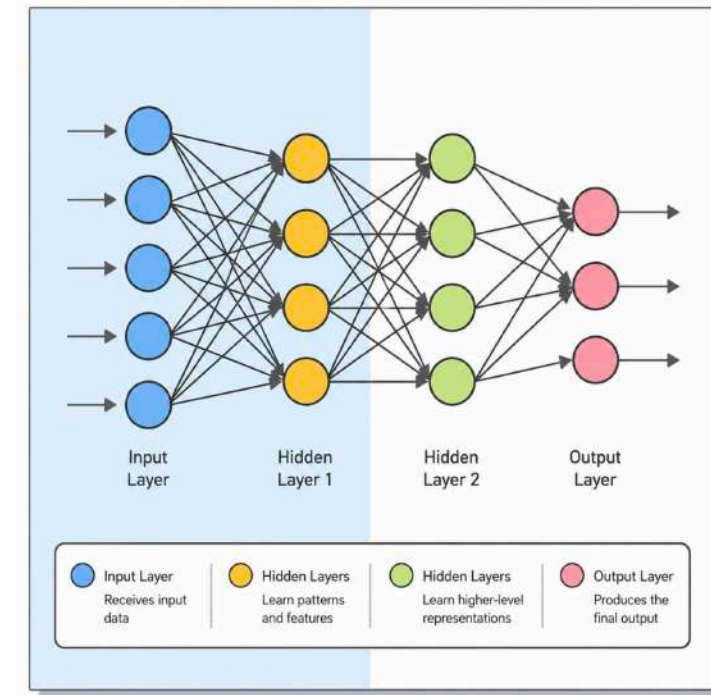
What is a Neural Network?

Neural Network: Interconnected artificial neurons organized into layers

- **Input Layer:** Receives the input data
- **Hidden Layers:** Learn patterns from the data
- **Output Layer:** Produces the prediction
- **Weights:** Represent the strength of connections

Learning: Happens by adjusting weights

- **Early Layers:** Learn simple patterns
- **Deeper Layers:** Learn more complex patterns



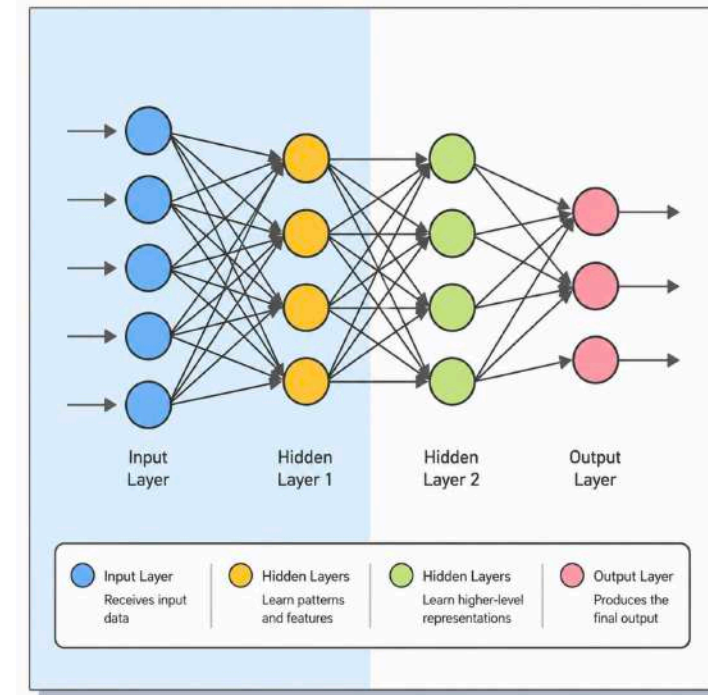


What Does a Neural Network Actually Learn?

Example: Recognizing a cat in an image

- **Early Layers:** Learn simple patterns like edges and lines
- **Middle Layers:** Learn shapes and textures
- **Deeper Layers:** Learn complex patterns like eyes, ears and faces

Key Idea: Each layer builds on patterns learned by earlier layers





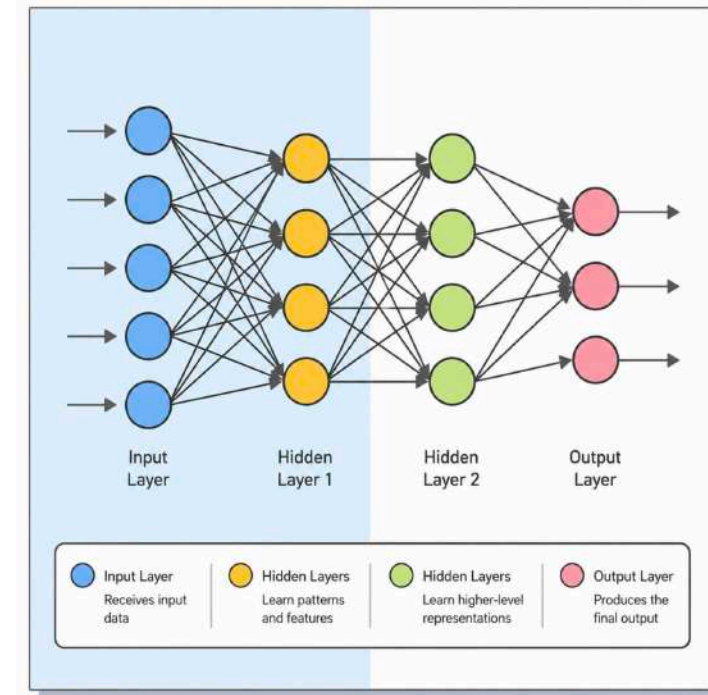
What is Deep Learning?

Deep Learning: Machine learning using neural networks with many layers

Learning Process:

- **Predict:** Make a prediction using current weights
- **Compare:** Measure how far the prediction is from the expected answer
- **Backpropagation:** Update the weights to reduce the error
- **Repeat:** Learn from many examples

Key Idea: Learning happens by repeatedly adjusting weights





Traditional ML vs Deep Learning

	Traditional ML	Deep Learning
Features	Often needs manually selected features	Can learn useful features automatically
Models	Usually smaller and simpler models	Can use very large neural networks
Best For	Simpler problems like forecasting	More complex problems involving language, speech, images and video



Neural Networks are Not New

Not New: Neural networks are NOT new

The Surprise: They became widely successful only in the last couple of decades

The Question: Why?





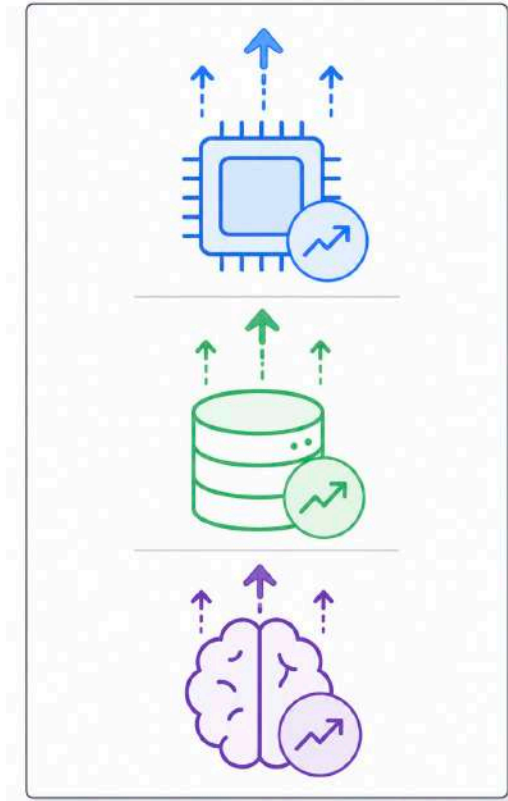
Discovery That GPUs Can Be Used for Deep Learning

Original Purpose: GPUs were designed to render computer graphics

2012 Breakthrough: AI researchers Alex Krizhevsky, Ilya Sutskever & Geoffrey Hinton used GPUs to train AlexNet (a deep neural network) and won ImageNet (a major image recognition competition)

Key Insight: GPUs can be used to efficiently train deep neural networks

- **The Result:** Training deep neural networks became much faster and more practical





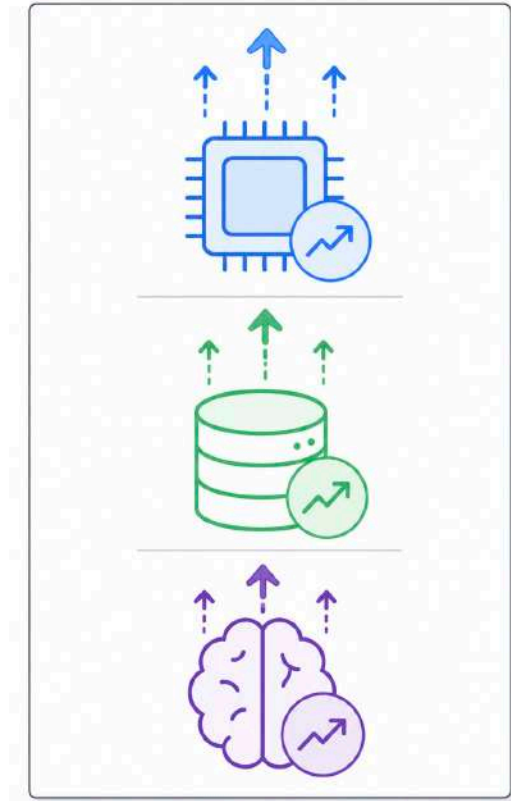
Google's Next Step: TPUs

GPUs: Made neural network training much faster

Google's Next Step: Build hardware specifically for AI workloads

TPU: Tensor Processing Unit

- **Efficient Training:** Optimized for tensor calculations used extensively in neural network training
- **Efficient Inference:** Optimized to run trained neural networks quickly and efficiently





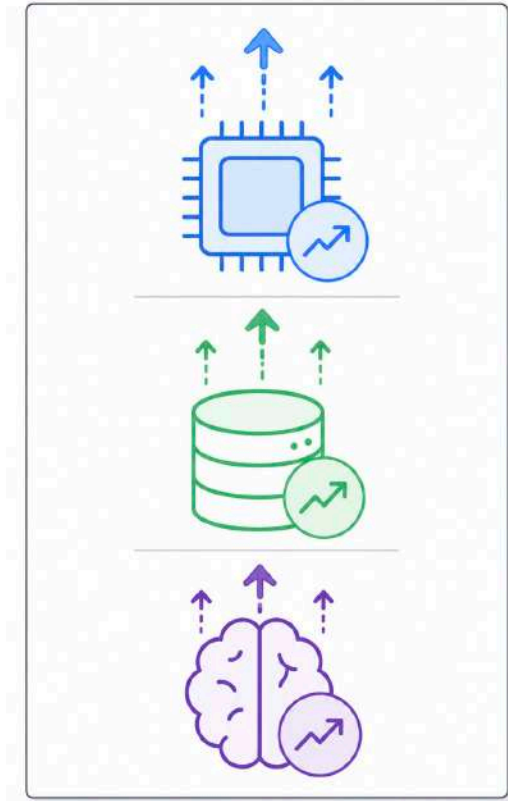
Hardware Alone Wasn't Enough

The Challenge: Building and training neural networks from scratch is complex

The Solution: Frameworks provide reusable building blocks

- **Popular Frameworks:** TensorFlow and PyTorch

The Result: Developers could focus on models instead of low-level implementation





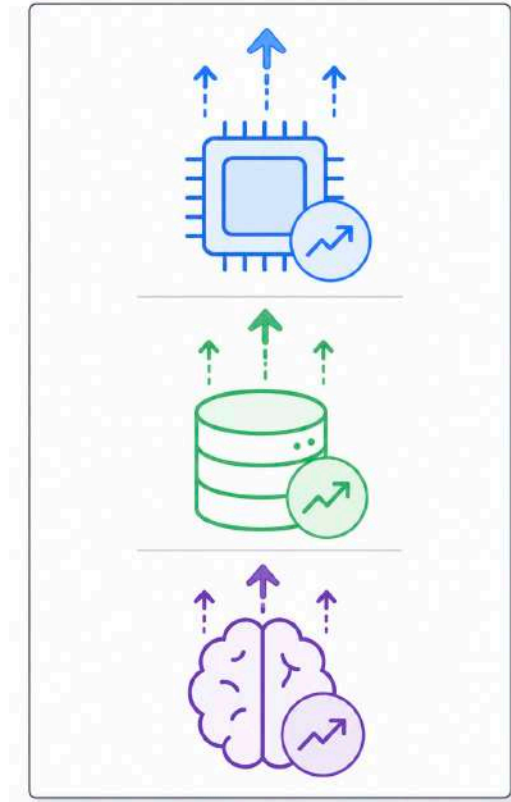
More Data Made a Big Difference

The Opportunity: The internet created enormous amounts of digital data

- **Examples:** Images, text, audio and video

Why It Matters: More training data helps neural networks learn better patterns

The Result: Larger datasets enabled more accurate and capable models





What Enabled the Deep Neural Network Revolution

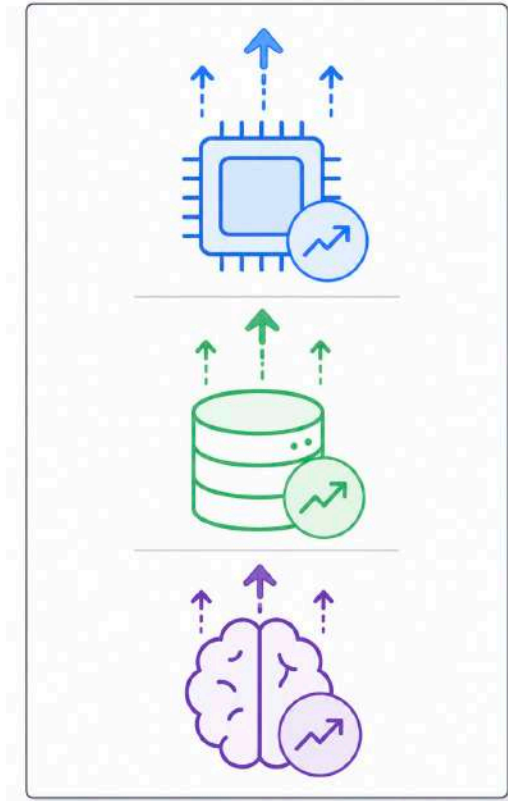
Better Compute: GPUs and TPUs made training faster

Better Software: Frameworks like TensorFlow and PyTorch simplified development

More Data: Large datasets became widely available

Better Techniques: Architectures and training methods improved

Together: These advances enabled the modern deep learning revolution





in28minutes

Going Deeper with Generative AI

Section Overview - Going Deeper with Generative AI



Focus: What will we learn?

- Why do LLMs use tokens instead of words?
- How do LLMs predict the next token?
- How are Generative AI models trained?
- What is Self-Supervised Learning?
- What is Supervised Fine-Tuning?
- What is Reinforcement Learning from Human Feedback (RLHF)?

Foundation: Understand how modern Generative AI models work and learn





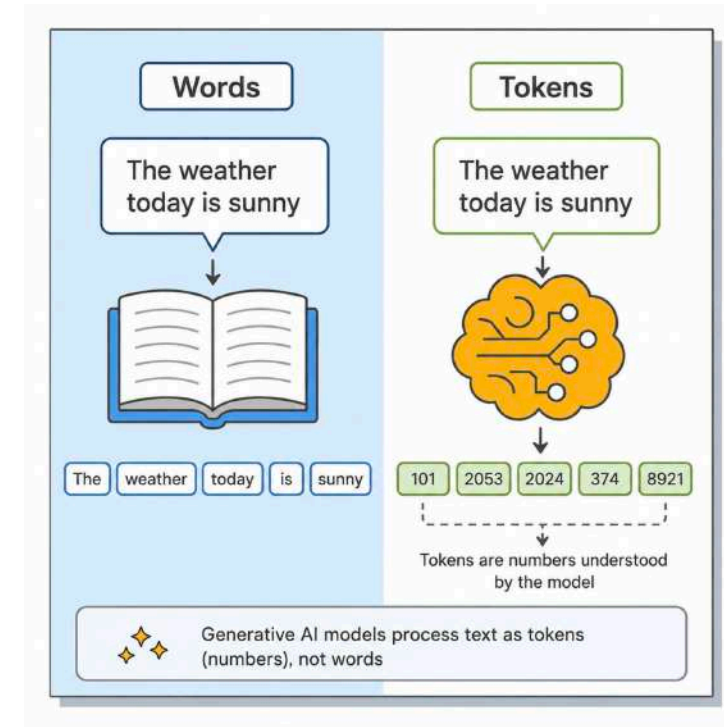
Large Language Models - Uses Tokens instead of Words

Token: A unit of text processed by the model

- Can be a word, part of a word, punctuation mark, or number

Why Tokens?

- **Efficient:** Whole words => large vocabulary
 - Tokens allow common words and word-parts to be reused
 - Examples: act, re + act, act + ing, view, re + view, view + ing
- **Handle Unseen Words:** Uncommon or new words can be split into smaller known tokens
 - Example: unfriendliness = un + friendli + ness
- **Support Diverse Text:** Tokens can represent different languages, code, numbers, symbols, and emojis





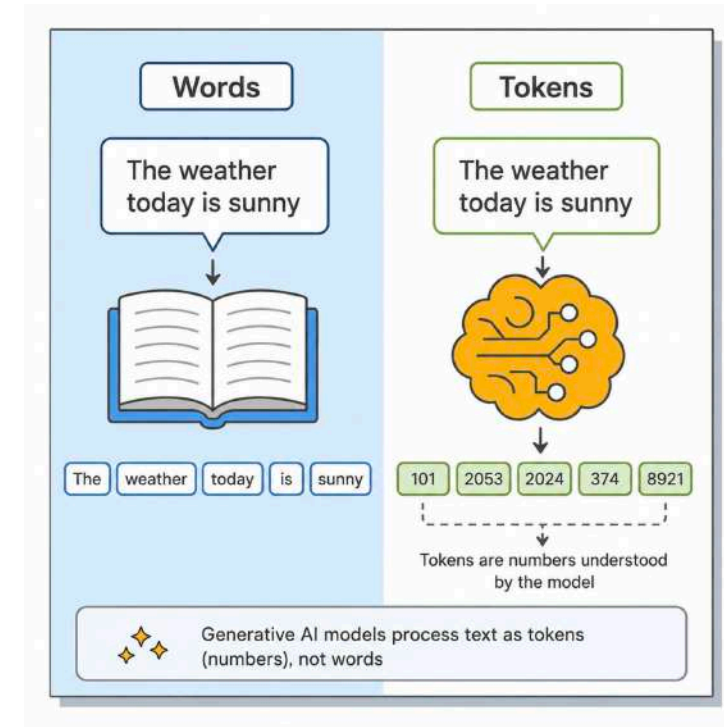
Key Step In Large Language Models - Decide Next Token [1/2]

Key Idea: Text generation is based on predicting the next token

During Training: Model learns probabilities for what could come next

- **Sequence:** "The cat sat on the"
- **Possible Next Tokens:** "mat": 0.4, "table": 0.2, "chair": 0.2, "moon": 0.1, ..

During Generation: Model might choose the highest probable token





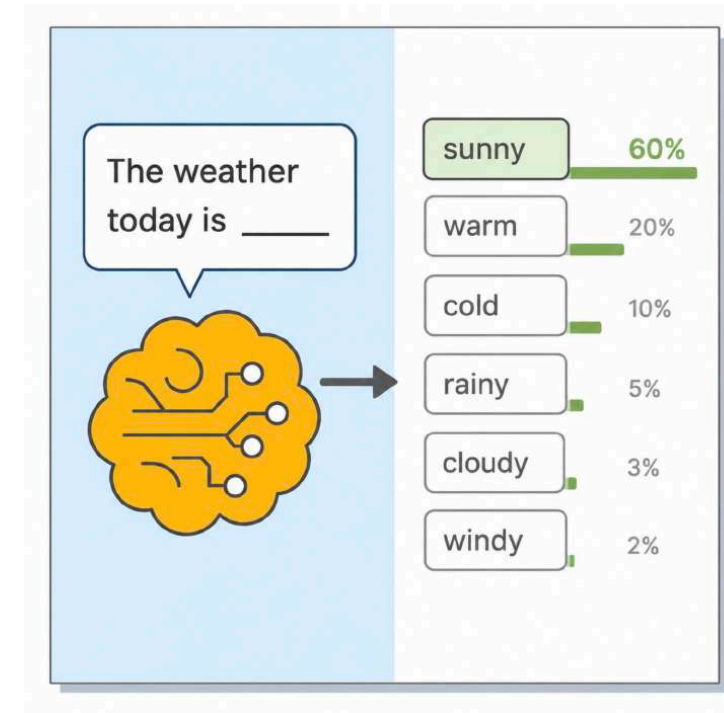
Key Step In Large Language Models - Decide Next Token [2/2]

During Generation: Model might choose the highest probable token

- The
- The weather
- The weather today
- The weather today is
- The weather today is sunny

Remember: The same approach is used to generate even large pieces of text

Temperature: Can influence the choice of next token





Generative AI - Uses Self Supervised Learning [1/2]

Self-Supervised Learning: Learn directly from training data

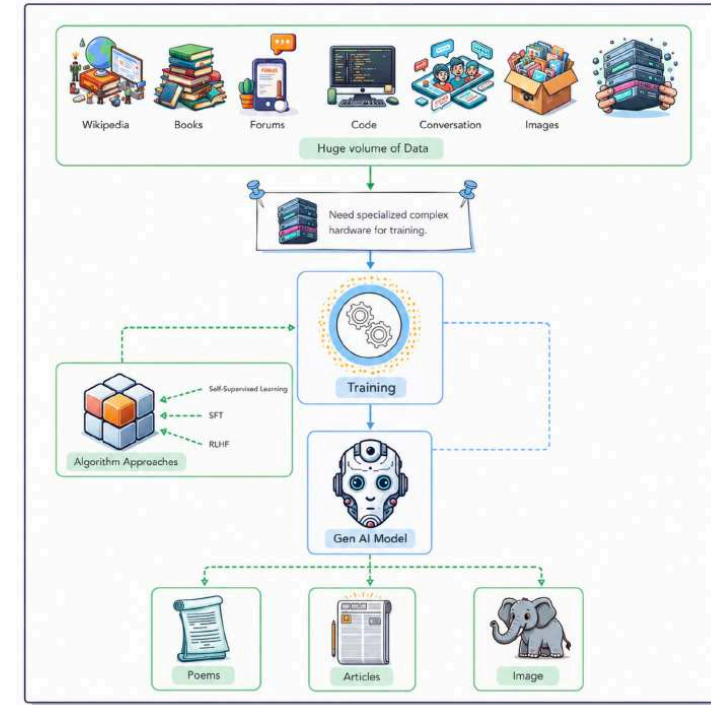
- No manual labels or annotations required

Training Data: "The capital of France is Paris"

- Use sequence: "The capital of France is"
- Try to predict the next word: "Paris"

Key Advantage: Can learn from huge volumes of text already available

- Does not need newly labeled training data





Generative AI - Uses Self Supervised Learning [2/2]

1 - Predict: Model tries to predict the next word based on preceding words

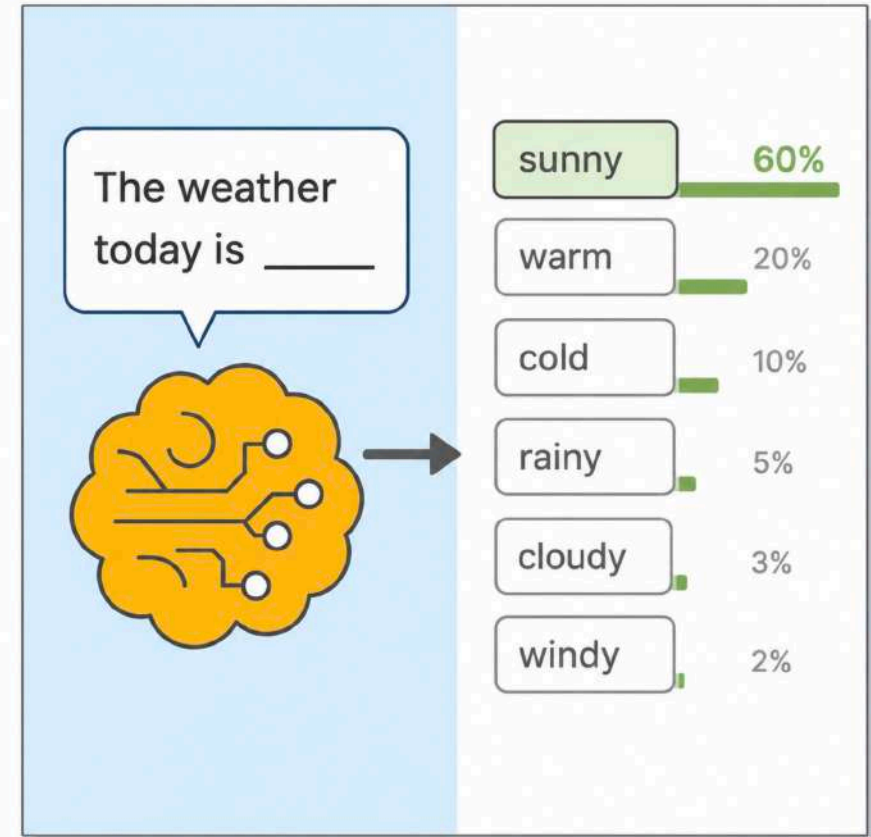
- Example: "The capital of France is"
- Model predicts the missing word

2 - Compare and Learn: Prediction is compared with the actual word

- Model learns from the error and adjusts its internal weights

3 - Repeat: Process is repeated across the entire training dataset

- Billions or trillions of examples can be used during training





Generative AI Text - Uses Supervised Fine-Tuning [1/2]

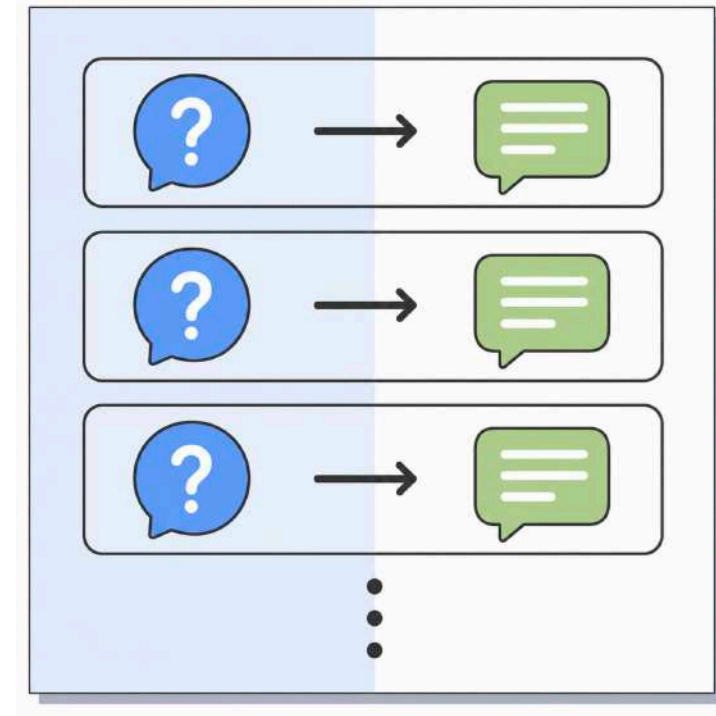
After Pre-Training: Model is good at predicting the next token

- **Example:** "My favorite sport is ...".
- Possible next tokens: basketball, soccer, cricket.

The Challenge: Next-token prediction does not guarantee Q&A behavior

- **Question:** How do we teach the model to answer questions?

Solution: Supervised Fine-Tuning (SFT)





Generative AI Text - Uses Supervised Fine-Tuning [2/2]

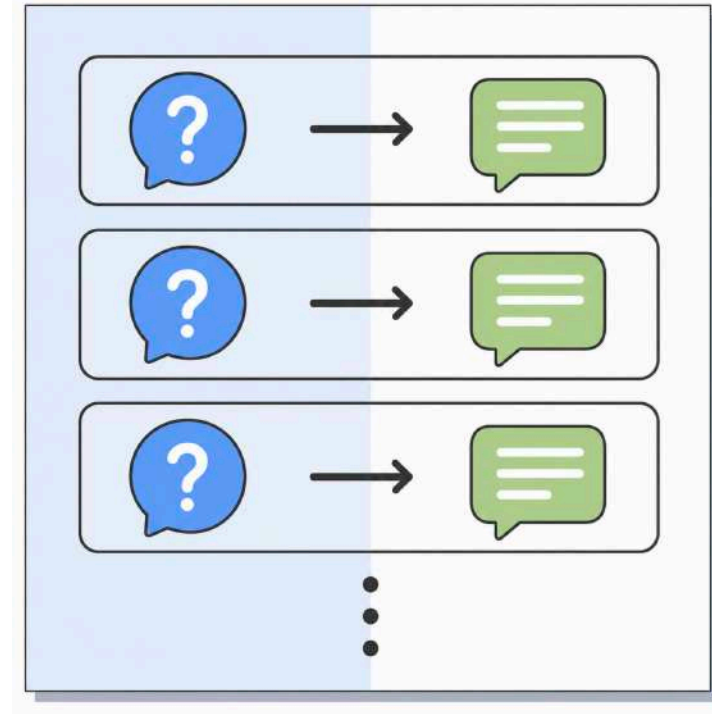
How It Works:

- Train using **Prompt + Ideal Response** examples
- Model learns patterns for answering questions and following instructions

The Result: Model also learns to respond in the expected way

- Answer questions
- Follow instructions
- Generate useful responses

Key Discovery: A relatively small, high-quality dataset can significantly improve behavior





Generative AI Text - Uses RLHF

The Challenge: Models do not inherently know which responses humans prefer

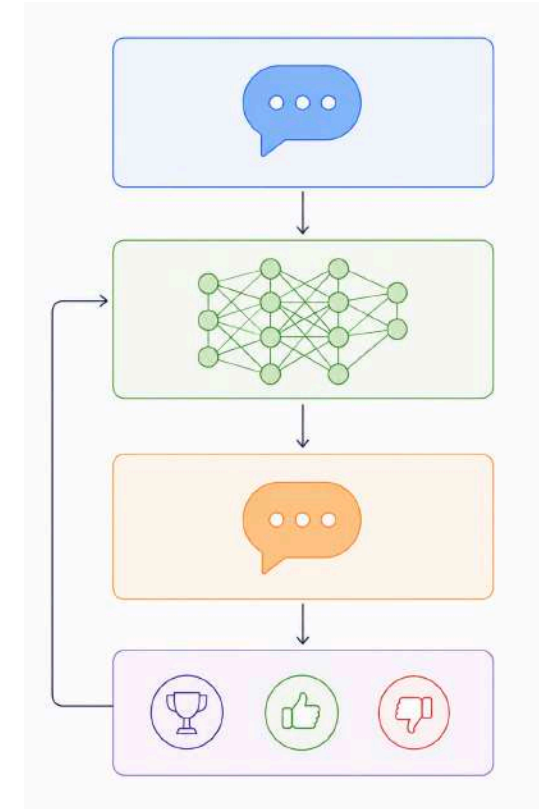
- Responses may be unhelpful, unsafe, biased, or inappropriate

Question: How can we better align model behavior with human preferences?

Solution: Reinforcement Learning from Human Feedback (RLHF)

High-Level Steps:

- **Step 1:** Learn human preferences using a Reward Model
- **Step 2:** Tune the Generative AI model using the Reward Model





Generative AI Text - RLHF - Step 1

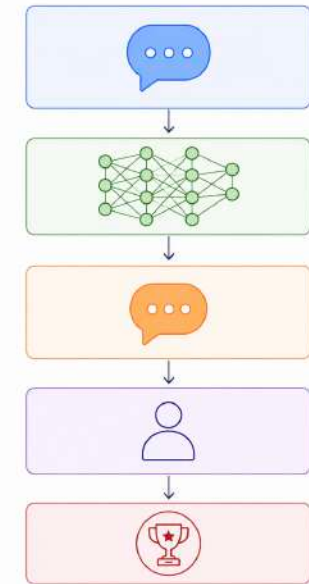
Goal: Train a Reward Model to recognize preferred responses

How?

- **1:** Generate multiple responses for the same prompt
- **2:** Human evaluators rank the responses
- **3:** Train the Reward Model using these preferences
- Repeat across many prompts and responses

Result: Reward Model learns to assign higher scores to responses humans tend to prefer

- These scores are called **rewards**





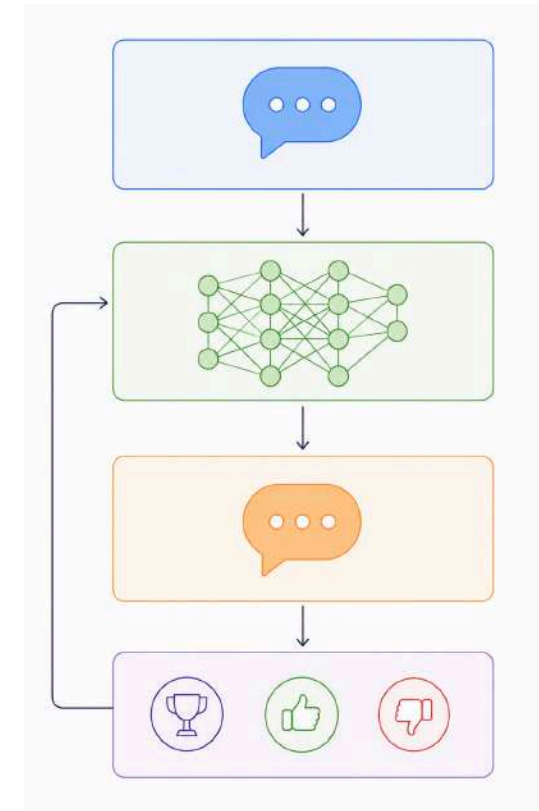
Generative AI Text - RLHF - Step 2

Goal: Tune the Generative AI model using the Reward Model

How?

- 1: Generate a response for a prompt
- 2: Reward Model evaluates the response
- 3: Adjust the Generative AI model to increase expected reward
- Repeat across many training examples

Result: Model behavior becomes better aligned with learned human preferences





in28minutes

Journey to Generative AI

Embeddings to Transformer

Section Overview - Journey to Generative AI



Focus: What will we learn?

- What are embeddings and why do we need them?
- Why does language understanding need context?
- How do Recurrent Neural Networks process sequences?
- What problem does attention solve?
- Why do we need Multi-Head Attention?
- What is a Transformer?
- Why were Transformers a breakthrough?

Journey: Embeddings → RNNs → Attention → Transformers





What are Embeddings?

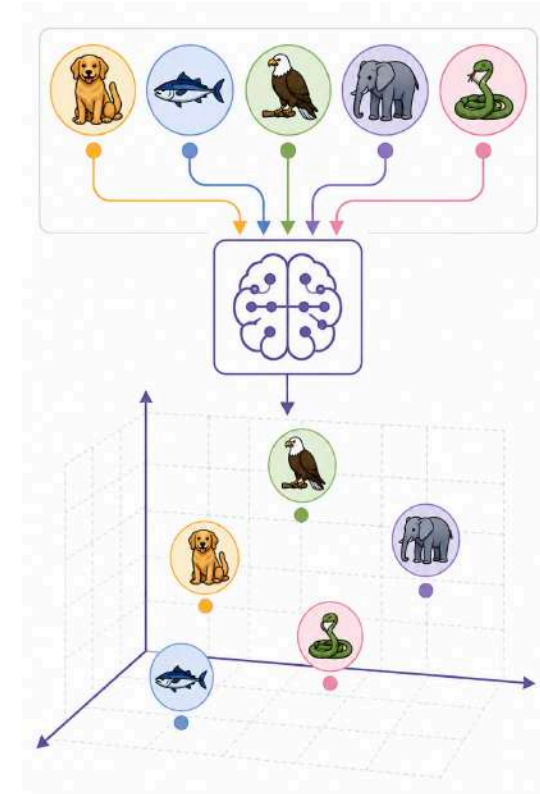
Simple Idea: Embeddings convert words or tokens into numbers that a neural network can work with

- Example: "cat" → [0.12, -0.45, 0.78, ...]

Vector: This list of numbers is a vector - each number represents a different dimension

- Simple dimension analogy for animals: [Habitat: water, land, or trees], [Diet: carnivore, herbivore, or omnivore], [Size: small, medium, or large], [Movement: flying, running, swimming, etc.]

Generative AI: Uses embeddings with hundreds or thousands of dimensions





What Can We Do with Embeddings?

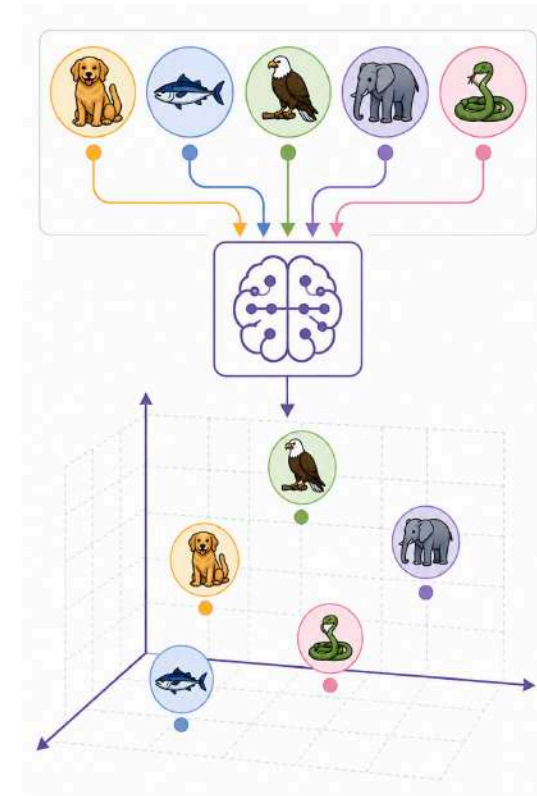
Similarity: Compare how closely related items are

- "AWS" may be closer to "Azure" than to "football"

Recommendations: Find similar products, users, movies, courses, or documents

Clustering: Automatically group similar items

Outlier Detection: Find items different from the rest





How are Embeddings Learned?

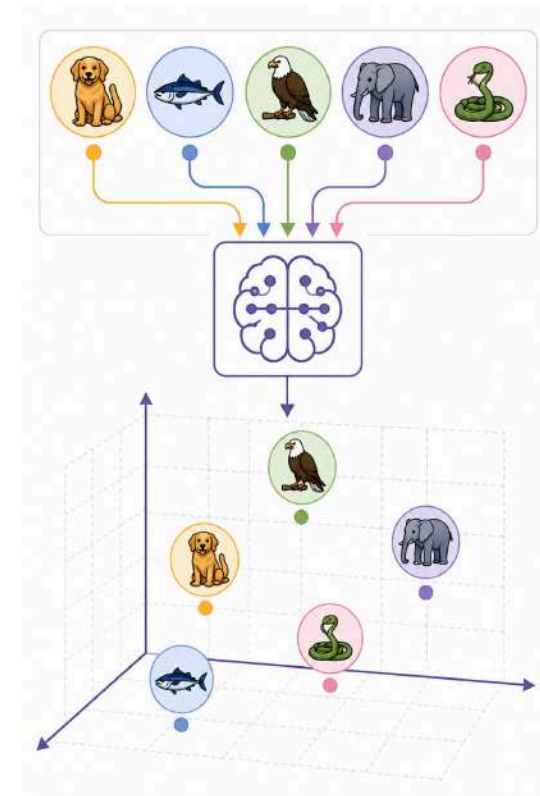
Step 1 - Start: Each token begins with an initial vector of numbers

- "cat" → [0.12, -0.45, 0.78, ...]

Step 2 - Train: The neural network processes huge amounts of training data

Step 3 - Learn: The numbers are repeatedly adjusted during training

Result: Tokens appearing in similar contexts develop useful relationships





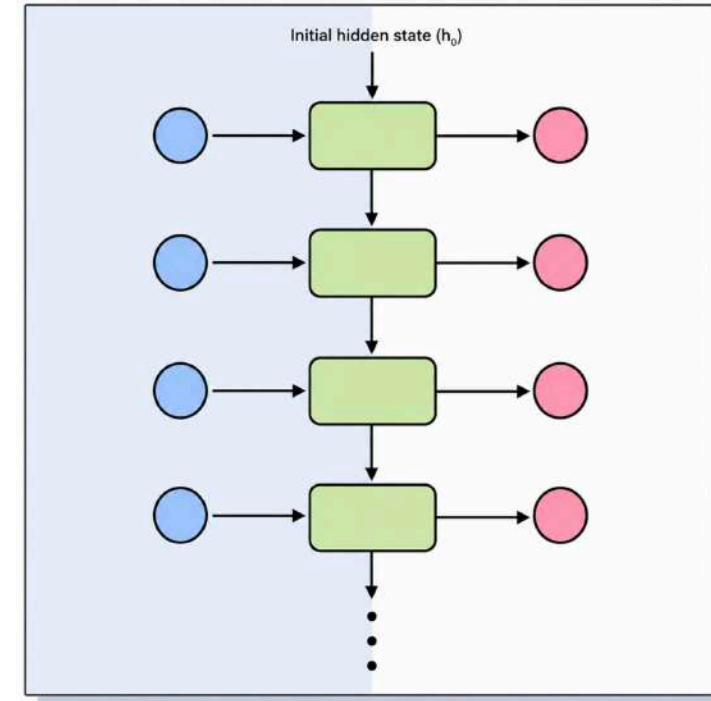
Language Needs Context

Individual Tokens: Embeddings provide useful information about each token

- **Language Understanding:** Also requires understanding relationships between tokens
- *Example:* "The animal didn't cross the street because it was tired" (What does "it" refer to?)
- **Challenge:** The model needs information from surrounding and earlier tokens

Early Solution: Recurrent Neural Networks

- Process tokens one after another
- Carry information from previous tokens forward





What is a Recurrent Neural Network?

Goal: Process sequential data such as text

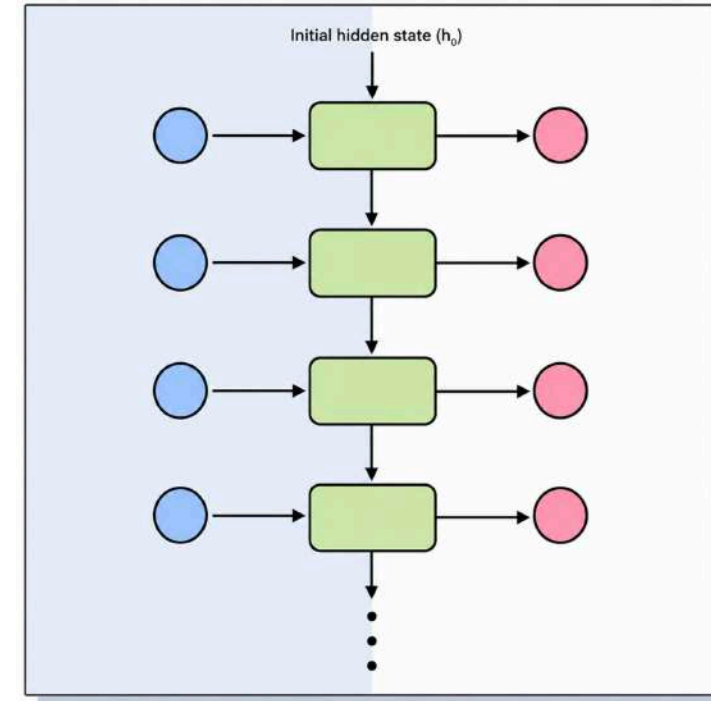
Key Idea: Process one token at a time

- First token → Second token → Third token → ...

Memory: Information from earlier tokens is carried forward

Example:

- "The cat is very ..."
- Information about "cat" is carried forward while processing later words





How Does an RNN Work?

Step 1 - Read Token: Read the current token

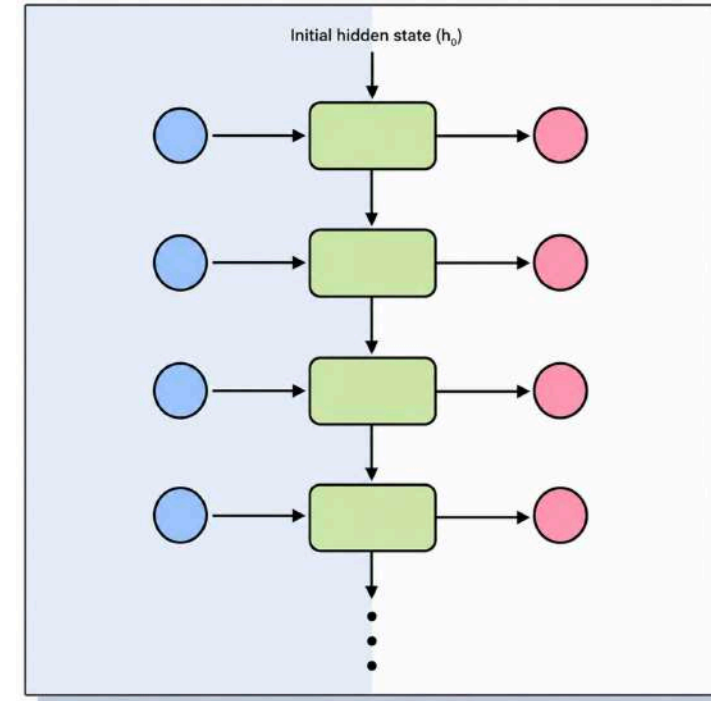
Step 2 - Read Previous Memory: Use information carried from earlier tokens

Step 3 - Combine: Combine the current token with previous information

Step 4 - Update Memory: Create a new hidden state

Step 5 - Move Forward: Pass the hidden state to the next token

Repeat: Continue till all tokens are processed



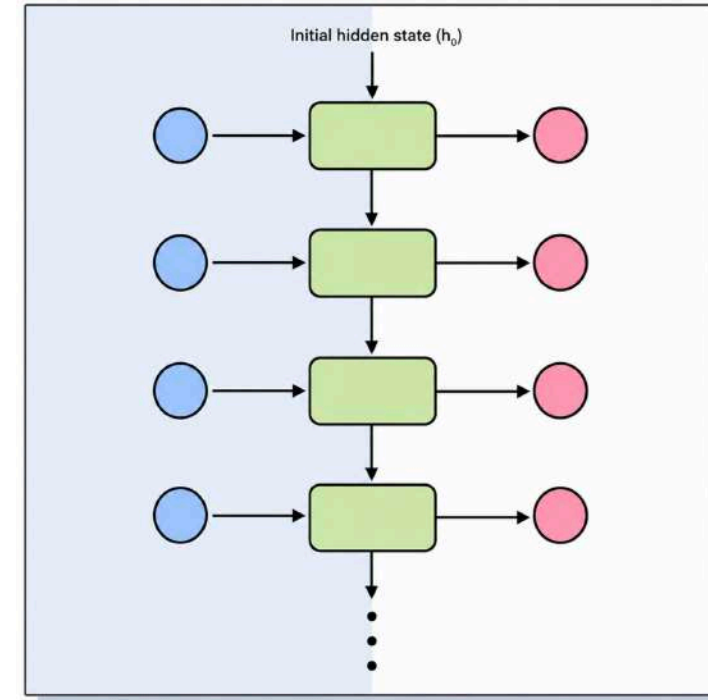


Hidden State - Memory of an RNN

Hidden State: Internal memory carried from one step to the next

- **Purpose:** Store useful information about previously processed tokens
- **Example:**
 - "Ranga lives in Hyderabad. He teaches..."
 - When processing "He", earlier information about "Ranga" can help

Limitation: Difficulty preserving important information across very long sequences





What is the Problem with RNNs?

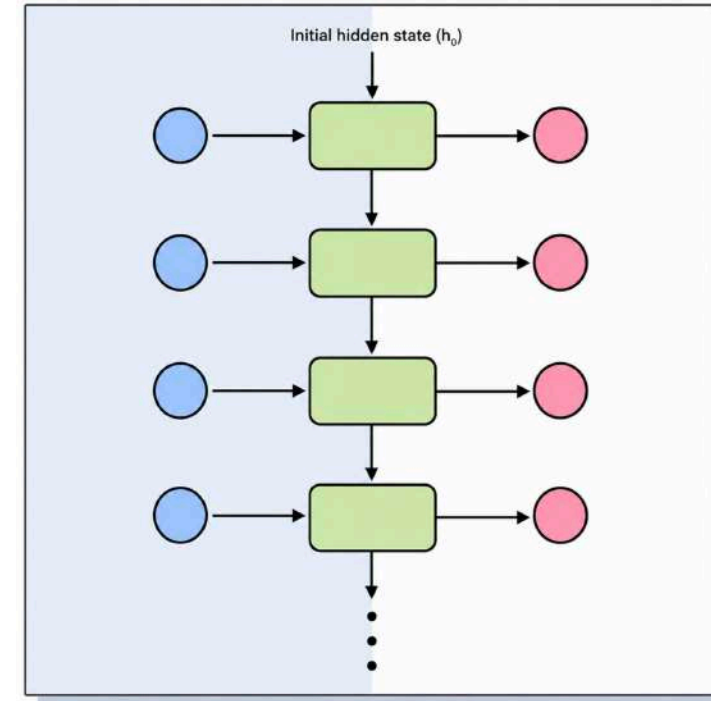
Long-Term Memory: Information from much earlier tokens can be difficult to preserve

- **Long Sequences:** The farther information has to travel, the harder it can be to use effectively

Sequential Processing: Tokens must be processed one after another

- Token 2 waits for Token 1, Token 3 waits for Token 2
- **Training Challenge:** Sequential processing limits parallelism

Question: Can a token directly look at the most relevant tokens?





What is Attention?

Goal: Determine which other tokens are important for understanding current token

For Each Token:

- Compare it with other tokens
- Calculate attention scores
- Give more weight to relevant tokens

Result: Each token gets a context-aware representation

Attention Matrix
(Rows = Word paying attention, Columns = Word being attended to)

Word (Query)		The	cat	sat	on	the	mat
The	→	0.40	0.45	0.05	0.02	0.03	0.05
cat	→	0.10	0.60	0.20	0.02	0.01	0.07
sat	→	0.02	0.38	0.40	0.10	0.02	0.08
on	→	0.01	0.04	0.15	0.30	0.10	0.40
the	→	0.02	0.03	0.05	0.10	0.40	0.40
mat	→	0.03	0.07	0.10	0.25	0.15	0.40



Attention - Simple Example

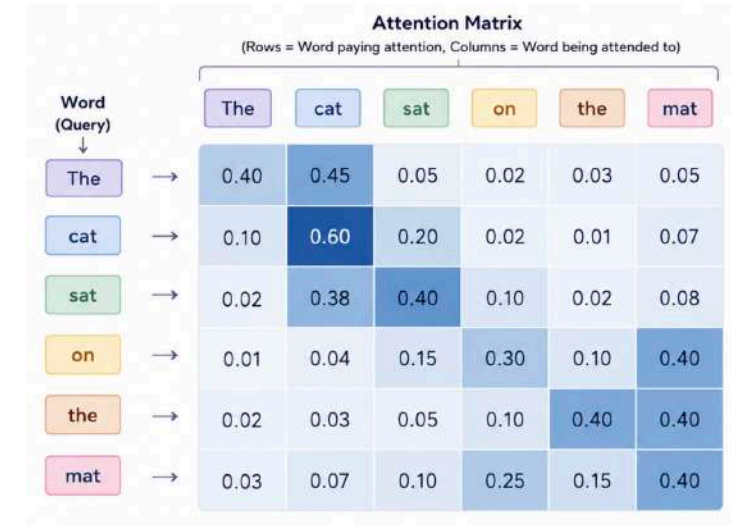
Sentence: "The cat sat on the mat"

Focus on "sat":

- "cat" → High attention
- "the" → Lower attention
- "mat" → Some attention
- Why?: "cat" tells us who performed the action

Focus on "on":

- "mat" → High attention





Attention Changed the Game

Attention: Allows a token to directly focus on other relevant tokens

Example: "The animal didn't cross the street because it was tired"

- **Understanding "it":** "it" can pay strong attention to "animal"
 - Less attention can be given to less relevant tokens

Benefit: Relevant relationships can be identified even when tokens are far apart

Attention Matrix
(Rows = Word paying attention, Columns = Word being attended to)

Word (Query)	The	cat	sat	on	the	mat
The	0.40	0.45	0.05	0.02	0.03	0.05
cat	0.10	0.60	0.20	0.02	0.01	0.07
sat	0.02	0.38	0.40	0.10	0.02	0.08
on	0.01	0.04	0.15	0.30	0.10	0.40
the	0.02	0.03	0.05	0.10	0.40	0.40
mat	0.03	0.07	0.10	0.25	0.15	0.40



RNN vs Attention

RNN: Information passes step by step through the sequence

- Token 1 → Token 2 → Token 3 → .. → Token 10
- **RNN Challenge:** Long-distance information can be difficult to preserve

Attention: Tokens can directly focus on other relevant tokens

- Token 10 can directly focus on Token 1
- **Attention Benefit:** Direct connections between related tokens
- **Result:** Better understanding of relationships across a sequence

Attention Matrix
(Rows = Word paying attention, Columns = Word being attended to)

Word (Query)	The	cat	sat	on	the	mat
The	0.40	0.45	0.05	0.02	0.03	0.05
cat	0.10	0.60	0.20	0.02	0.01	0.07
sat	0.02	0.38	0.40	0.10	0.02	0.08
on	0.01	0.04	0.15	0.30	0.10	0.40
the	0.02	0.03	0.05	0.10	0.40	0.40
mat	0.03	0.07	0.10	0.25	0.15	0.40



Why Multi-Head Attention?

Challenge: A sentence can contain many different relationships between tokens

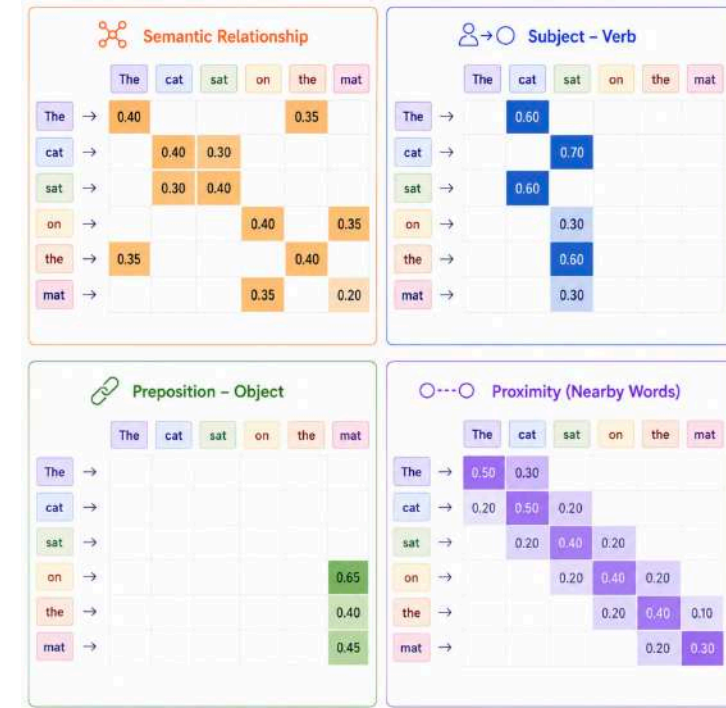
- **Single Attention Head:** May focus on only some of these relationships

Solution: Use multiple attention heads

- **Each Head:** Learns to focus on different relationships

Illustrative Example: “*The cat sat on the mat*”

- **Head 1 - Proximity:** “sat” ↔ “on”
- **Head 2 - Subject and Action:** “cat” ↔ “sat”
- **Head 3 - Preposition and Object:** “on” ↔ “mat”
- ...





Multi-Head Attention – Why Multiple Heads?

Different Perspectives: Each attention head processes the same tokens from a different perspective

Different Relationships: Heads can learn different patterns between tokens

Parallel Attention: Multiple relationships can be captured at the same time

Combined Output: Information from all heads is combined

Result: Richer contextual representation of each token





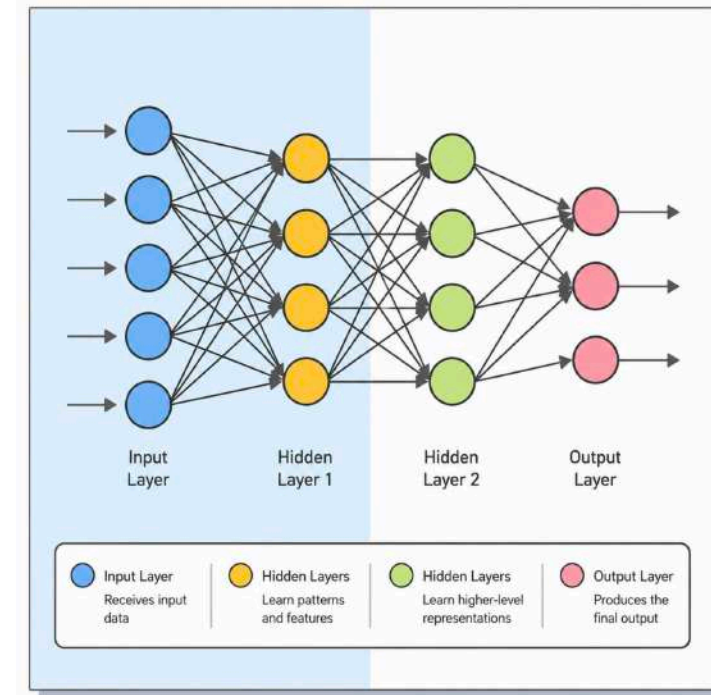
From Attention to Transformers

RNNs: Process tokens one after another and carry information forward

Attention: Allows tokens to directly focus on other relevant tokens

- **Key Question:** What if we build a neural network mainly around attention?

Transformer: Use attention as a core mechanism



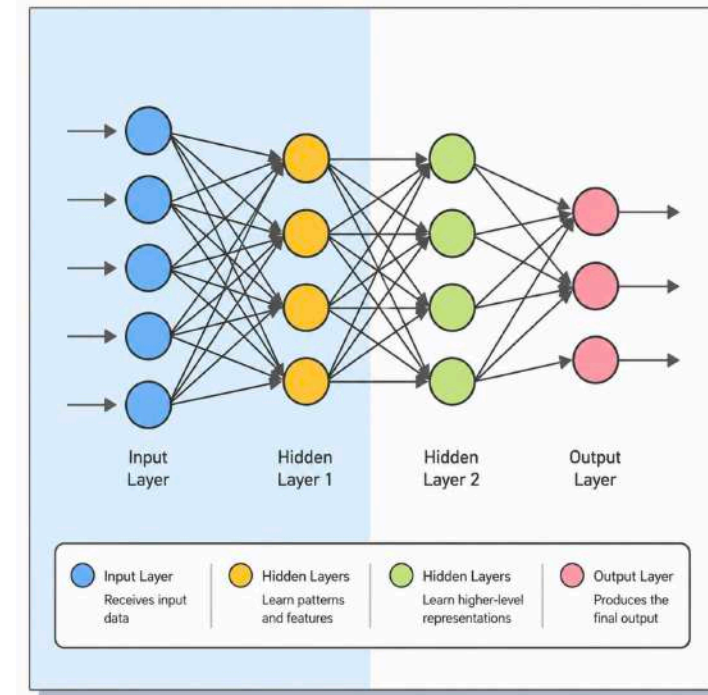


What is a Transformer?

Transformer: Neural network architecture designed to process sequences using attention

Core Building Blocks:

- **Embeddings:** Represent tokens as vectors
- **Multi-Head Attention:** Understand relationships between tokens
- **Neural Network Layers:** Process and refine the representations





Why Were Transformers a Breakthrough?

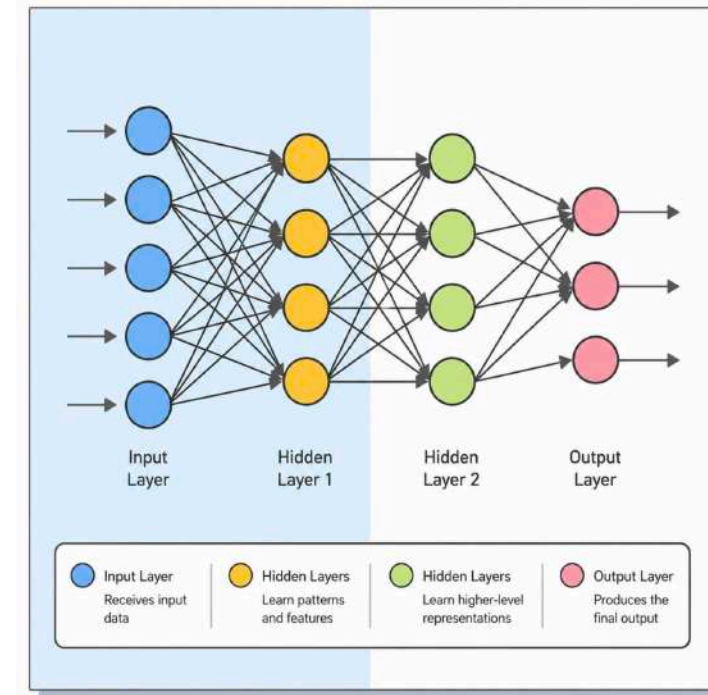
Better Context: Attention allows each token to directly consider other relevant tokens

Parallel Processing: Many tokens can be processed in parallel during training

- **Scalable:** Works efficiently with massive datasets and compute

Flexible: Can be used for text, images, audio, and multimodal data

Foundation: Became the architecture behind many modern Generative AI models





in28minutes

Getting Started with AI Agents



Why Do We Need AI Agents?

Models are great at understanding, reasoning, and generating outputs

- **Real-world tasks** often require more than generating a response:
 - **Decide** what needs to happen next
 - **Interact** with external systems
 - **Take Actions** based on results
 - **Complete Tasks** involving multiple steps

Question: How do we go one level above models and focus on achieving a specific goal?

- **Answer:** AI Agents





What Are AI Agents?

AI Agent: A system that uses AI models to pursue a goal

- **Observe:** Understand user input and context
- **Reason:** Determine what needs to be done
- **Use Tools:** Access data, invoke API and interact with other applications

Agentic AI: Moves beyond one-off prompts

- Uses models to pursue goals (Can adapt across multiple steps)

Key Idea: Move beyond generating responses to getting things done





AI Agents - An Example

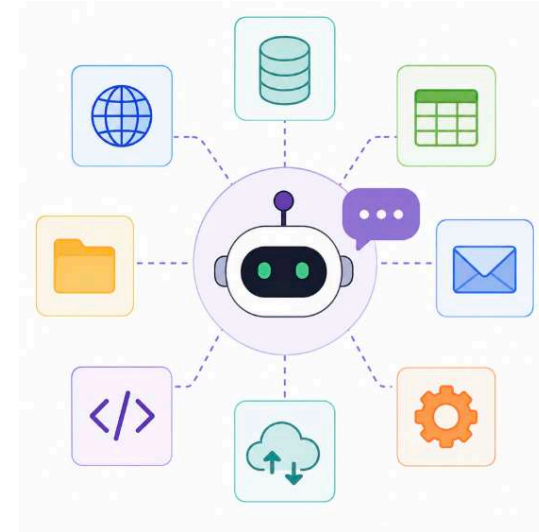
User Request: "Summarize report and send to team"

A model can:

- **Generate:** Summarize the report

An agent can:

- **Understand:** Identify the user's goal
- **Generate:** Create the summary using a model
- **Find Information:** Identify the team members
- **Use Tools:** Connect to email
 - **Take Action:** Send the summary
- **Result:** The task is completed





Use Cases for AI Agents [1/2]

Customer Agents: Assist customers and perform customer-facing tasks

- **Example:** Answer account questions and help resolve issues

Employee Agents: Help employees complete internal tasks

- **Example:** Answer HR questions and submit leave requests

Creative Agents: Assist with content creation workflows

- **Example:** Create campaign copy and visuals





Use Cases for AI Agents [2/2]

Data Agents: Analyze data and generate insights

- **Example:** Analyze sales data and summarize important trends

Coding Agents: Assist with software development tasks

- **Example:** Generate code, review, and fix issues

Security Agents: Assist with detecting and responding to threats

- **Example:** Investigate suspicious activity and initiate remediation





Agent Pattern: Conversational Agents

Goal: Interact with users through natural language

Flow:

- **1: Receive Request:** User requests an action
- **2: Understand:** Agent understands the request
- **3: Use Tools:** Fetch information or perform actions
- **4: Respond:** Return the result to the user

Examples:

- **Healthcare:** "Book an appointment with Dr. XYZ tomorrow"
- **Retail:** "Where is my order?"





Agent Pattern: Workflow Agents

Goal: Execute multi-step processes to achieve a business goal

- **1: Trigger:** User or system initiates a task
- **2: Determine Steps:** Determine what to do
- **3: Execute:** Use tools and external systems
- **4: Adapt:** Determine subsequent steps
- **5: Complete:** Deliver the final result

Examples:

- **HR On-boarding Agent:** Create account → assign equipment → schedule orientation
- **IT Operations Agent:** Diagnose issue → restart service → create incident → notify team





Getting Started with AI Agent Architecture [1/2]

Persona: Defines the agent's role, behavior, and communication style

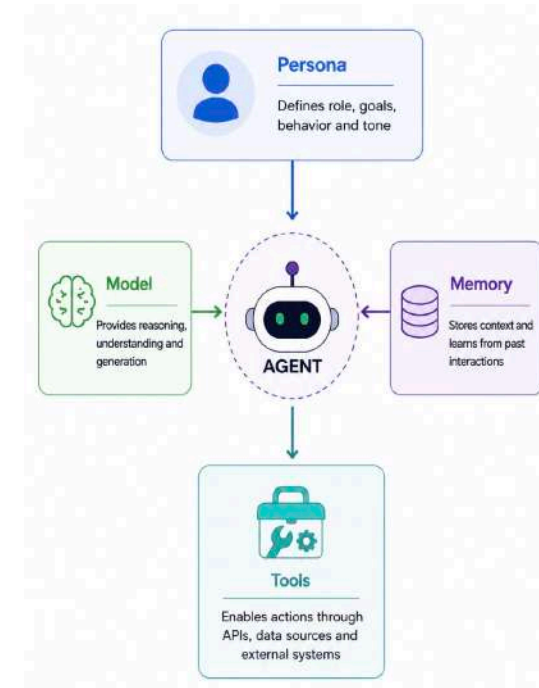
- **Defined Using:** Agent instructions or system prompts

Model (LLM): Provides the core intelligence for the agent

- **Understand:** Comprehend user requests and instructions
- **Reason:** Determine what needs to be done
- **Generate:** Create responses

Memory: Provides context across interactions

- **Short-Term:** Current conversation and task context
- **Long-Term:** Historical information and past interactions





Getting Started with AI Agent Architecture [2/2]

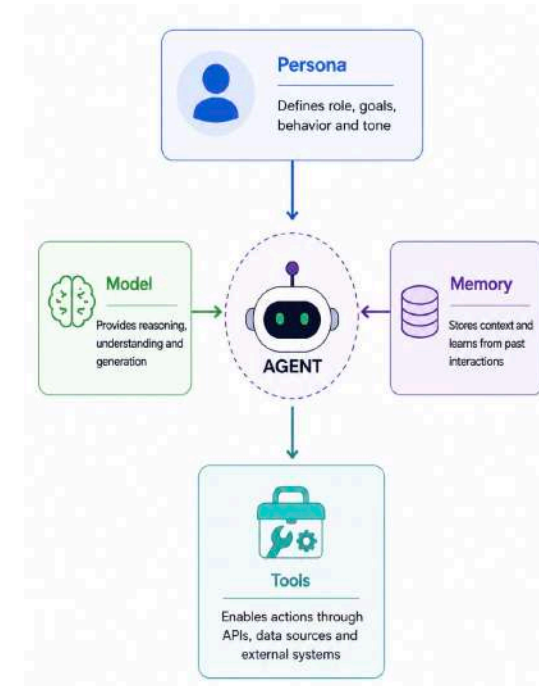
Tools: Extend what an agent can do

Knowledge Tools: Access external information

- **Examples:** Search engines, documents, databases, etc.

Action Tools: Perform tasks in external systems

- **APIs:** Connect to services such as translation or travel booking
- **Functions:** Execute application logic such as `calculate_xyz()`
- **Integrations:** Connect to calendars, email, payment systems, etc





in28minutes

Getting Started with FM Customization



Getting Started with FM Customization

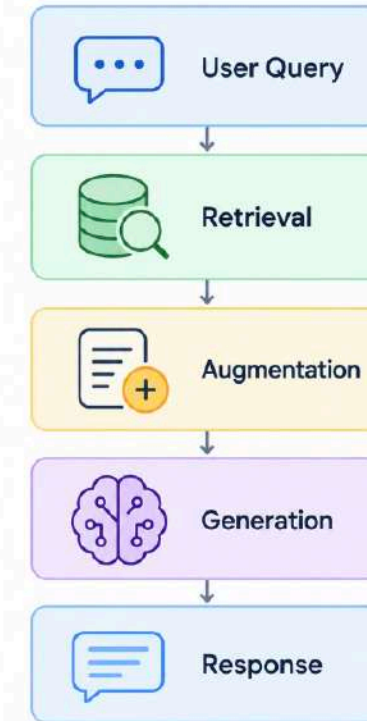
Sometimes foundation models need further customization

The model needs access to data → **RAG**

The model needs to behave differently for a specific use case → **Fine-tuning**

The model is too large or expensive for a use case → **Model Distillation**

Let's get started!





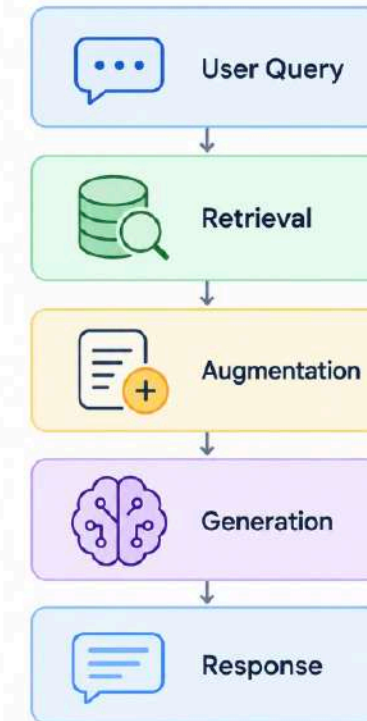
What is Groundedness?

Groundedness: How well is a response supported by trusted sources?

- Can the claims in the response be traced back to those sources?

Why It Matters:

- Improves accuracy and trust
- Reduces unsupported or fabricated claims (hallucinations)





What is RAG?

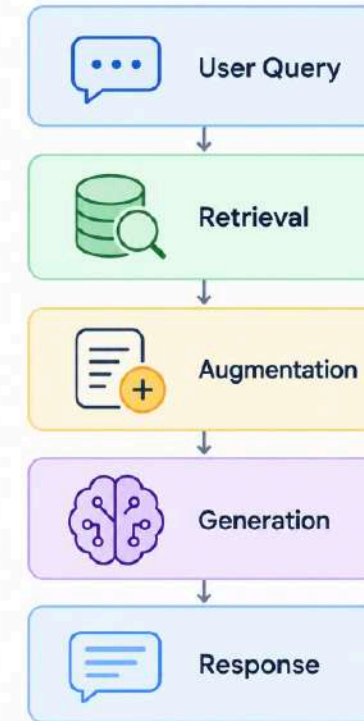
Retrieval: Fetch relevant information

Augmentation: Add information to the prompt

Generation: Model responds using prompt + augmented information

Example: A chatbot pulls related information from internal knowledge base to answer IT queries

- Retrieve relevant information
- Add to prompt (augment)
- Generate a grounded, accurate response





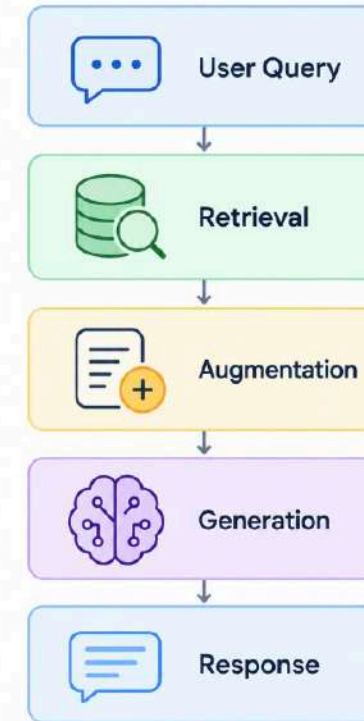
Why Use RAG?

Improves Accuracy: Grounds answers in relevant information

Use Recent Data: Use recent information that was unknown during model training

Improves Trust: Provide responses including citations and sources

Reduces Hallucinations: Get reliable responses all the time





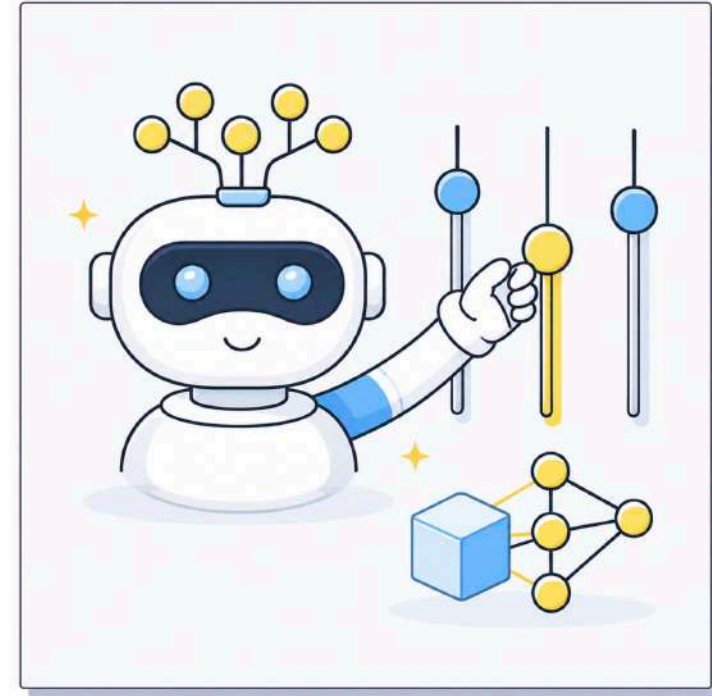
What is Fine-Tuning?

Problem: Foundation model is generic

Solution: Further train the model to adapt it to your specific needs

- **Medical:** Adapt to medical terminology
- **Brand:** Adapt to your company's tone and style
- **Code:** Adapt to specific coding style and standards

Example: Fine-tune a model using customer support examples to produce responses in your enterprise style





What is Model Distillation?

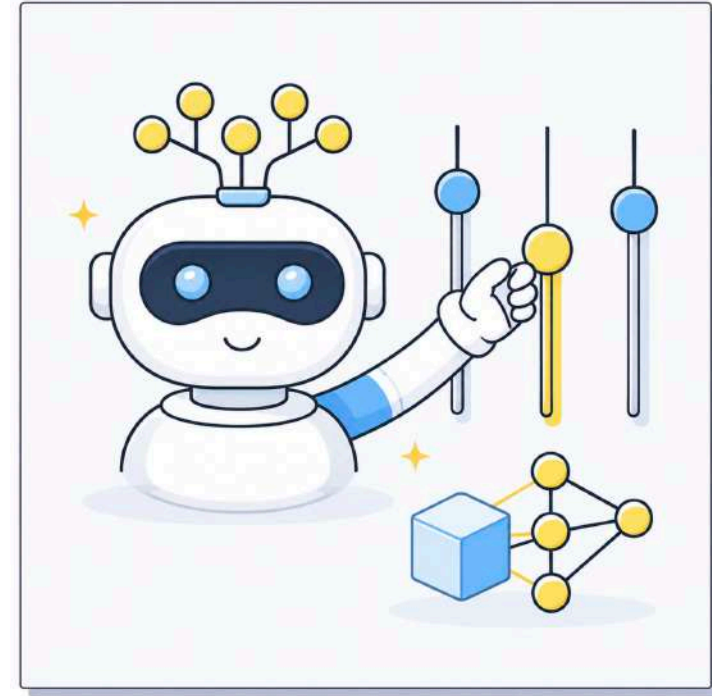
Problem: Large, powerful models can be expensive and slow for your use case

Solution: Train a smaller model to learn from the outputs of a larger model

- **Teacher Model:** Larger, more capable model
- **Student Model:** Smaller, more efficient model

Example: Use outputs from a large model to train a smaller model for customer support

Benefits: Lower cost to run and lower latency while retaining useful capabilities





Foundation Models - Reviewing Key Terminology

Topic	What Does It Mean?
Pre-training	Train a foundation model on large amounts of data (initial training to create the model)
Fine-tuning	Tuning a model for a specific task or behavior (customize it to understand business-specific language)
In-context learning	Guide the model using instructions and examples in the prompt (provide examples of the expected JSON format)
RAG	Retrieve relevant information and add to model's context (use information from your organization's documents)
Model distillation	Train a smaller model to learn from a larger model (create a smaller, more efficient model that is faster)



FM Customization - A Few Scenarios

Scenario	Recommendation	Why?
A support chatbot needs answers from your company's latest product documentation	RAG	Retrieve relevant information. Use it as context along with the prompt.
A model needs to consistently respond using your organization's specialized style	Fine-tuning	Adapt the model's behavior using your organization specific examples
A powerful model works well, but it is too expensive and slow for your application	Model Distillation	Train a smaller model to reproduce useful capabilities of a larger model
You want responses in a specific JSON structure and you provide a few examples	In-context Learning	Including examples in the prompt without retraining the model
An HR agent needs to answer questions using the latest company policies	RAG	Retrieve relevant policies before generating the answer



in28minutes

Getting Started with OpenAI API



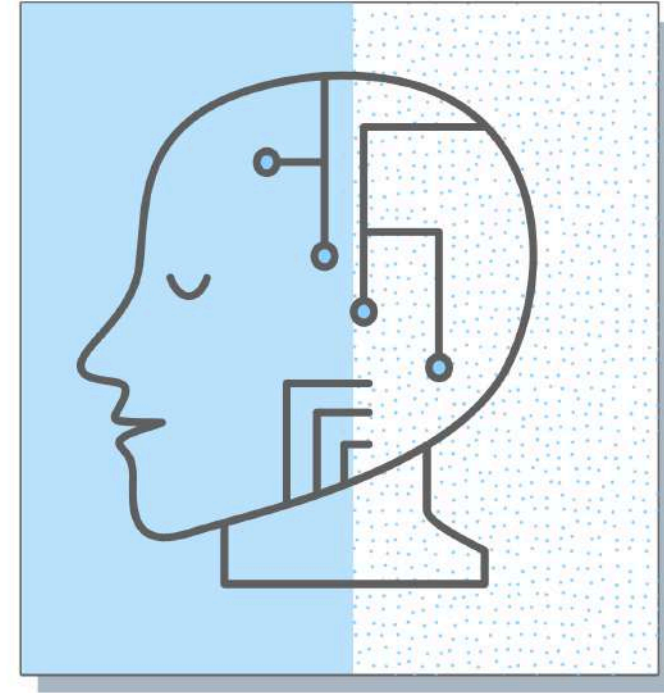
Let's Get Started with OpenAI API

Things You Will Need:

- Your email
- Your personal details (name, age, ..)
- Valid payment method: debit or credit card

Free Plan: Is NOT always available

Recommended: Add \$5 in credits for a smoother experience





in28minutes

Getting Started with OpenAI Platform

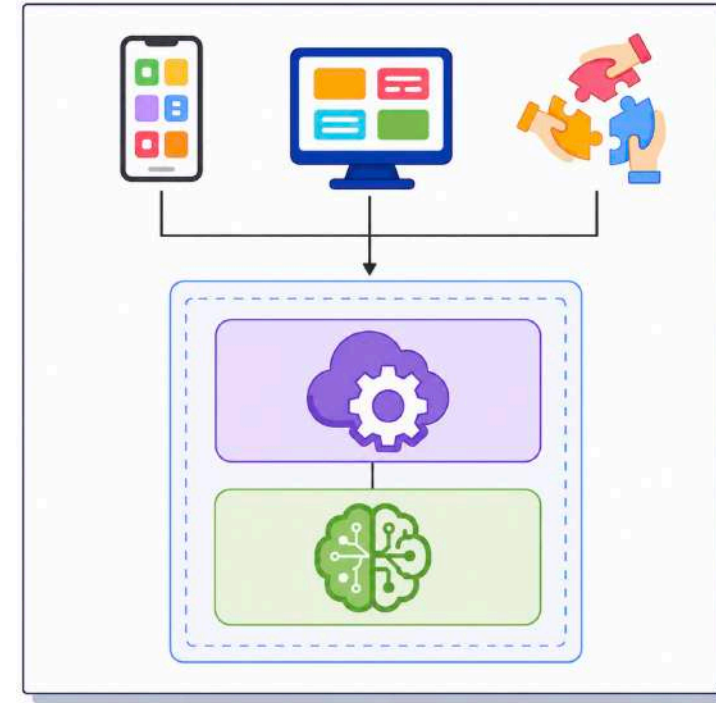


How to Integrate Generative AI into Your Apps?

OpenAI API: OpenAI provides an API around its models

Flow:

- **Request:** Your application sends a request to the API
 - Which model?
 - Prompt
 - Configuration (temperature etc.)
- **Processing:** API processes the request
- **Response:** Returns the response to your application





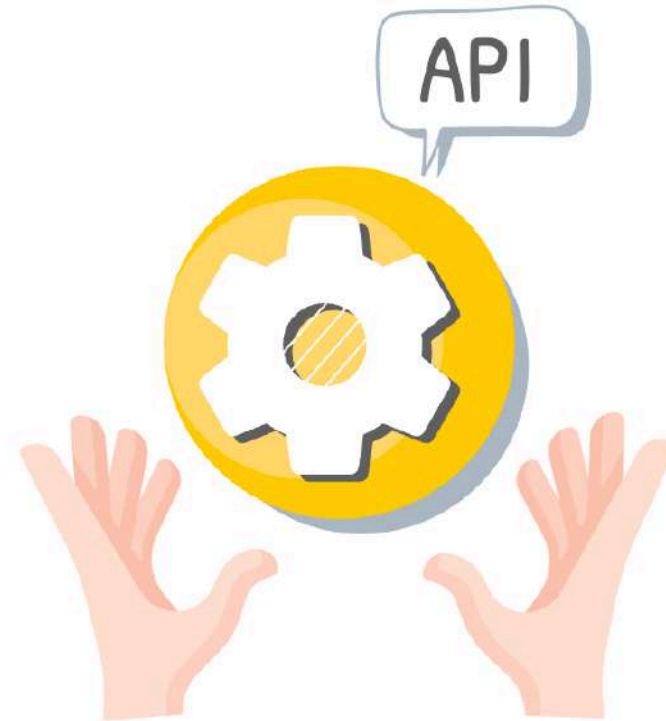
How To Understand OpenAI API?

API Platform: Web based platform to experiment with the OpenAI API

[*https://platform.openai.com*](https://platform.openai.com)

Next Steps:

- Create an OpenAI account
- Play with OpenAI API





in28minutes

Getting Started with Model Configuration



Constraining Response Size - Max output tokens

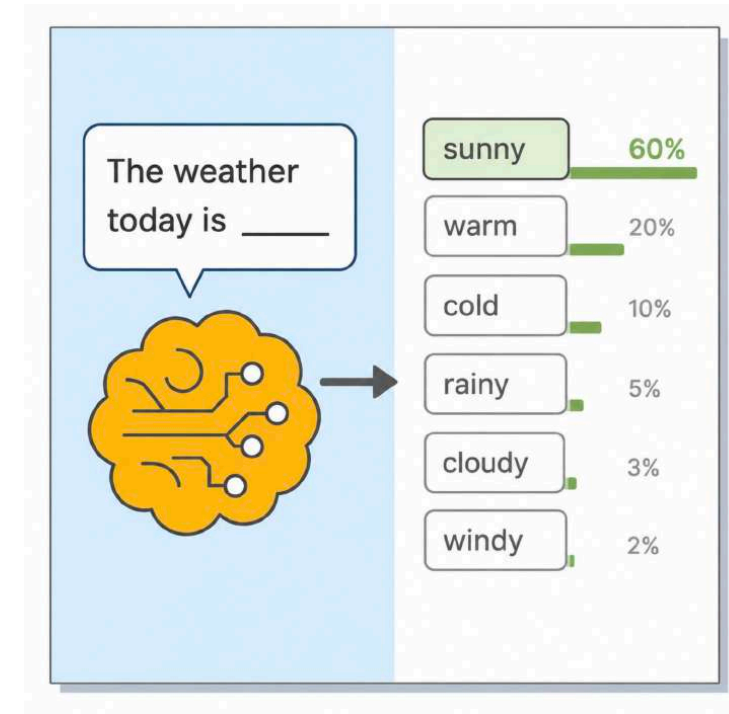
Ask for shorter responses: "List exactly two options" or "Write a 200-word summary"

OR configure **Output token limit**: How long do you want the response to be?

- Max output tokens: Sets an upper limit on the number of tokens the model can generate
- Rule of thumb for English: A token is approx. 4 characters. 100 tokens approximate to 60-80 words.

Why is this important?

- Shorter responses and
- Potentially lower cost (Less tokens consumed)





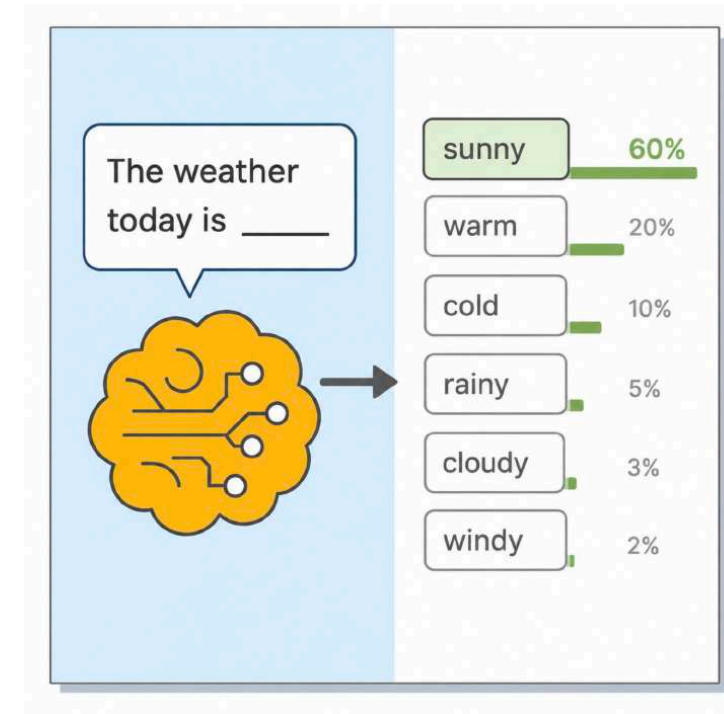
What is Top P

Top-P: Sets the cumulative probability threshold for token selection

How Does It Work?: Start with most probable token, include tokens until their cumulative probability reaches the Top-P value

Example: Tokens **sunny**, **warm**, and **cold** have probabilities of 0.6, 0.2, and 0.1:

- **Top-P = 0.9:** Model selects from **sunny** (0.6), **warm** (0.2), and **cold** (0.1)
- **Top-P = 0.8:** Model selects from **sunny** (0.6) and **warm** (0.2)



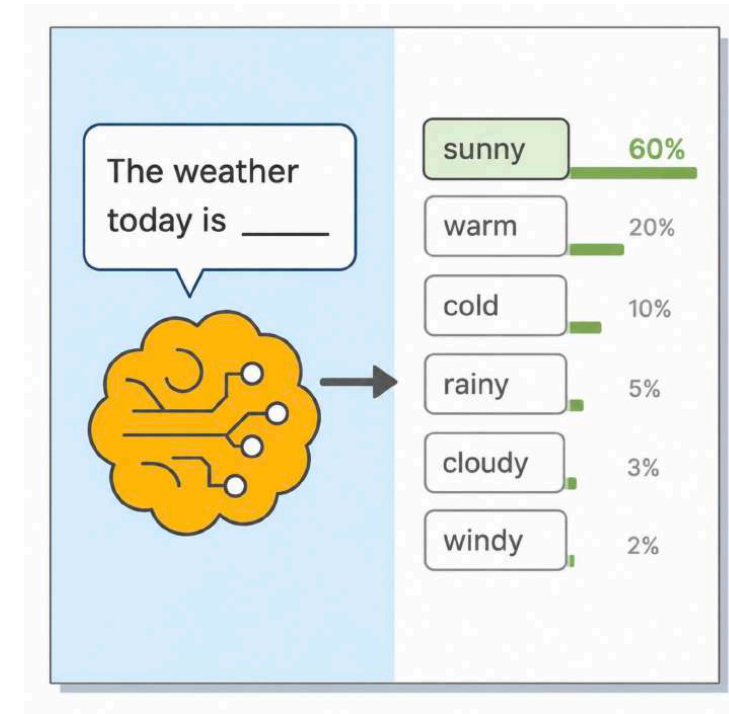


What is Temperature

A:40%, B:20%, C:10%, D:5%: Which of A, B, C, D should be chosen?

Temperature: Controls the degree of randomness in token selection

- Higher values => more randomness (more creative)
- Lower values => lesser randomness (more focused and predictable)
- Example Scenarios:
 - Find Capital City of India: use low value
 - Write a creative essay: use high value





Understanding Sampling Parameters - Scenarios

Scenario	Solution
Low or High Temperature: A student asks “What is 2 + 2?”	Low Temperature (You want a focused and predictable answer.)
Low or High Temperature: Writing a poem about "the color of silence"	High Temperature(Encourages imaginative and unexpected outputs.)
Low or High Top-P: Generating a formal email reply to a customer complaint	Low Top-P (Keeps output within a narrow, safe range)
Low or High Top-P: Brainstorming character names for a fantasy novel	High Top-P (Allows a wider sampling of less likely and more creative outputs.)

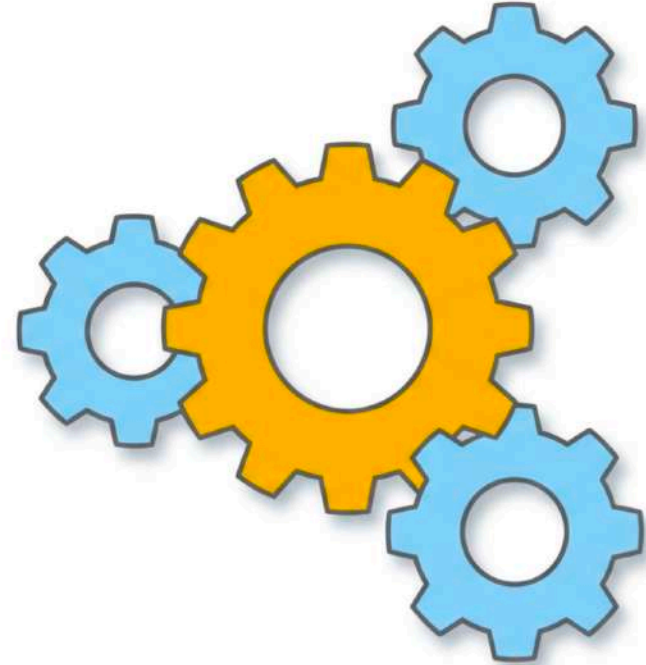


Need for System Instructions (or System Messages)

System instructions define the overall behavior of the AI model

- **Example:** You are an enterprise support assistant. Answer only from the supplied knowledge base and clearly state when information is unavailable
- Example things you can specify:
 - **Role:** Act as a cloud architect, tutor, or support agent
 - **Rules:** Do not reveal sensitive information
 - **Tone:** Use professional and beginner-friendly language
 - **Output format:** Respond using a table, JSON, or bullet points

System instructions have **higher** priority than user instructions





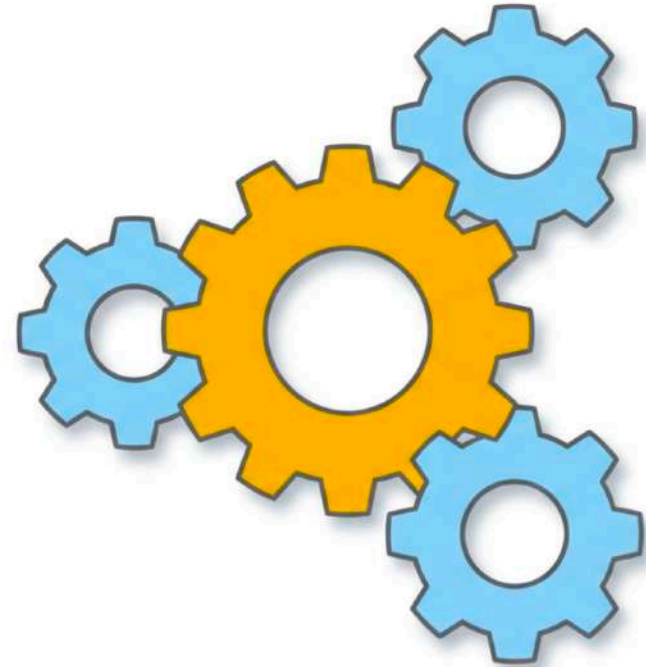
Need for Larger Context Windows

In traditional language models:

- Can they remember earlier parts of **long conversations** or **big documents**?
 - Not always
- Can they provide answers based on **entire textbooks** or **research papers**?
 - Not if the content exceeds the model's context window
- **Smaller context windows** limit what the model can "see" at once

What if we want the model to **understand, summarize, & reason** across Huge Docs?

- We need **larger context windows**



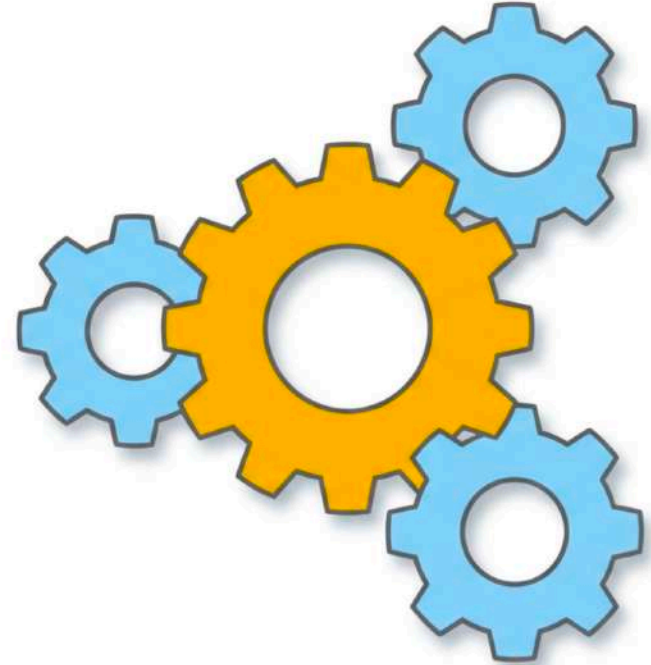


Getting Started with Context Windows [1/2]

Context window: Maximum amount of information a model can work with at one time

It may include:

- System instructions
- User prompts and model responses
- Conversation history
- Documents
- Tool results





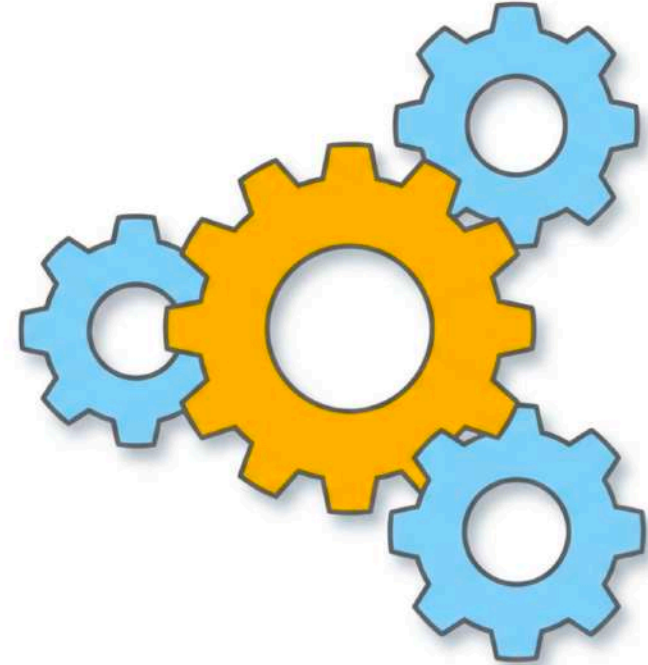
Getting Started with Context Windows [2/2]

With smaller context windows:

- Earlier parts of long conversations may be excluded
- Large documents may not fit into a single request
- Important information may be missed

Larger context windows help models:

- Analyze long reports and research papers
- Summarize multiple documents
- Maintain context across longer conversations
- Reason across information distributed across many pages





in28minutes

Getting Started with

Writing Better Prompts



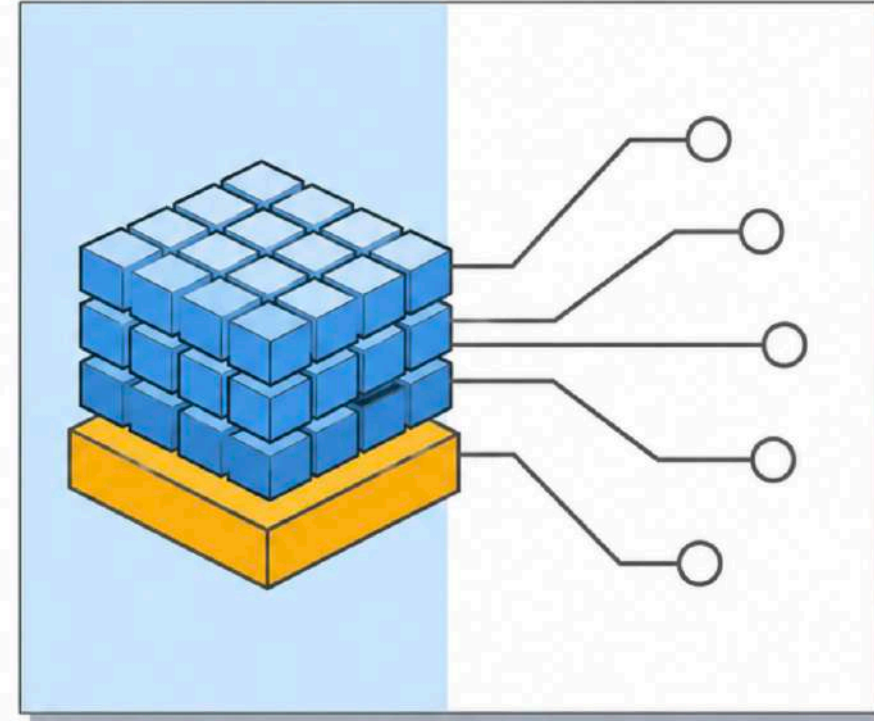
Prompting: Your Key to Great Gen AI Results [1/2]

Large Language Models: Generate responses by predicting the next word

The Way You Ask Matters: "Your prompt is the model's starting point for what “good” looks like"

Right Mindset: Treat your prompt like a "handoff" to a new intern

- Provide relevant context
- Give clear instructions
- Explain what a good response looks like

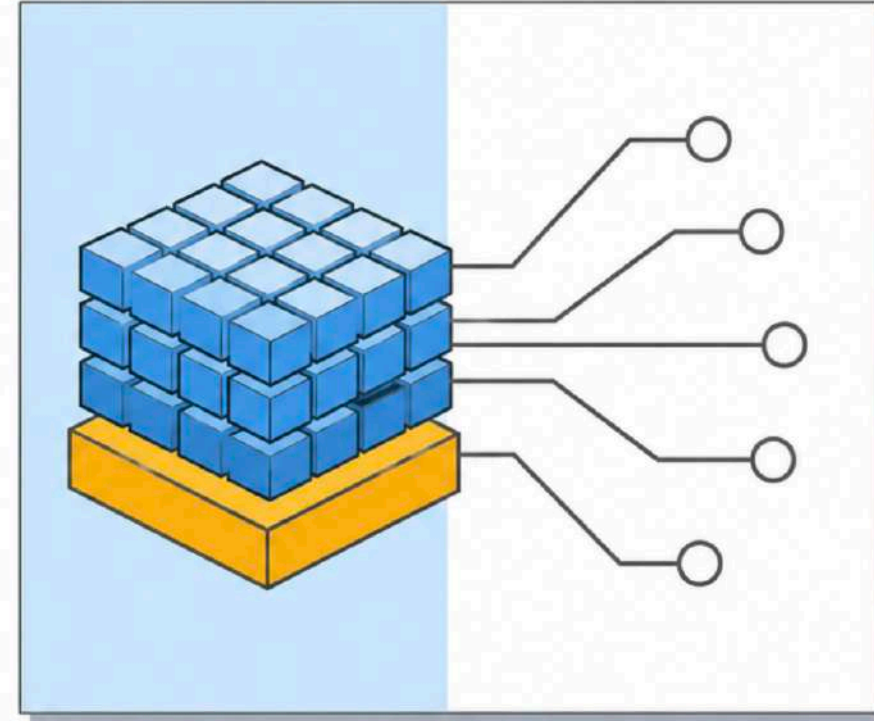




Prompting: Your Key to Great Gen AI Results [2/2]

Big Terms: Design and Engineering!

- **Prompt design:** "process of creating prompts, or natural language requests, that elicit accurate, high quality responses from a language model"
- **Prompt engineering:** "Art and science of designing and optimizing prompts to effectively interact with language models and to get responses that are accurate, concise, and relevant"



Better Prompts → Better Results

- **Key Question:** How do we write better prompts?

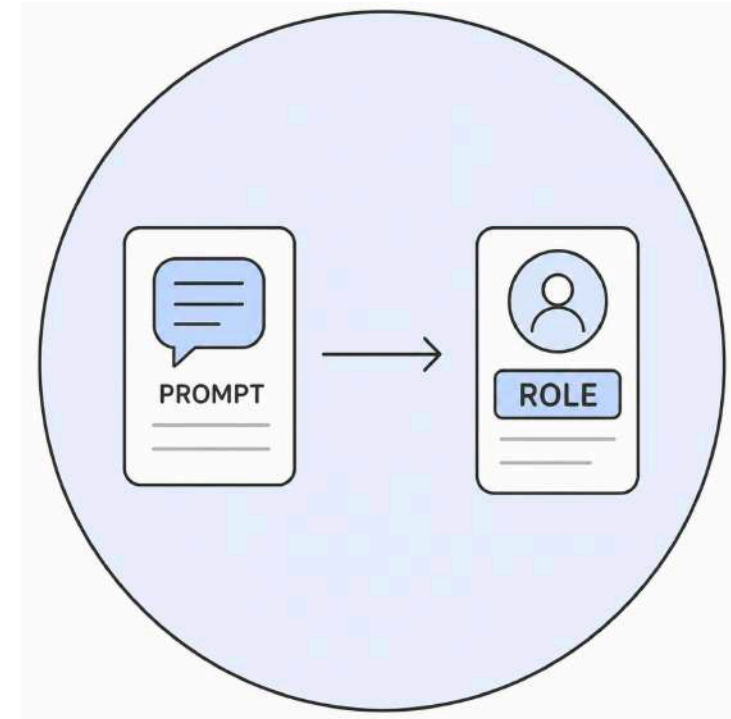


What is Role Prompting? [1/3]

Role Prompting: Assign the model a specific role or persona

Goal: Guide tone, style, & level of detail

Example Prompt: "You are an experienced travel advisor who specializes in helping first-time international travelers. You explain each step clearly and highlight common mistakes to avoid."





What is Role Prompting? [2/3]

Role	Prompt	Example Output
Without Role	Evaluate FreshSip Beverages as an investment.	Revenue has grown 18% annually, operating margins are healthy at 18%, and the company has little debt. Its 22% market share and extensive distribution network provide a strong competitive position. However, at a P/E ratio of 42, the stock trades at a significant valuation premium.
With Role	You are an experienced long-term investor investing in durable high-quality businesses. Evaluate FreshSip Beverages as an investment.	The first question is whether FreshSip's competitive advantage is durable. A brand that allows the company to maintain market share despite lower-priced competitors, combined with distribution across 600,000 outlets, may represent a meaningful economic moat. The business also produces strong free cash flow, requires little debt, and has demonstrated an ability to reinvest capital at attractive returns. Those are characteristics of a high-quality business. But at 42 times earnings, I would ask whether today's price leaves an adequate margin of safety.



What is Role Prompting? [3/3]

	Without Role	With Role
Prompt	Evaluate FreshSip Beverages as an investment.	You are an experienced long-term investor investing in durable high-quality businesses. Evaluate FreshSip Beverages as an investment.
Primary Focus	Financial metrics, Valuation	Business quality, durability
Time Horizon	Not clearly defined	Long-term ownership
Key Question	Is the stock attractive relative to its growth and peers?	Is this a wonderful business at a sensible price?



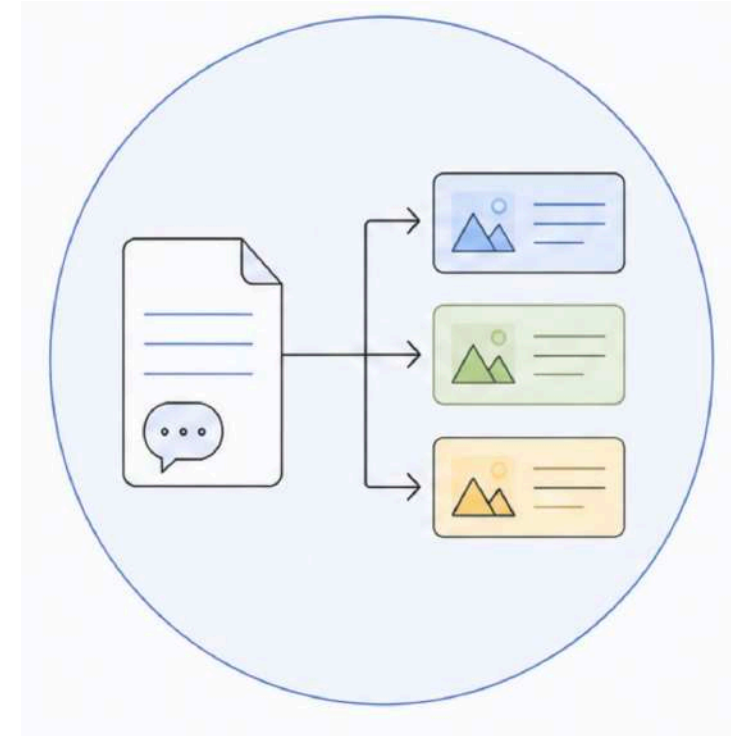
ZERO SHOT vs ONE SHOT - Example

EXAMPLE 1: ZERO SHOT

- Please choose the right answer:
- Question: Which of these is a programming language?
 - A) Docker B) Python

EXAMPLE 2: ONE SHOT

- Use the same answer format as the example.
- Please choose the right answer:
 - Question: Which of these is a container orchestration tool?
 - A) Docker B) Kubernetes
 - Answer: B is correct
 - Question: Which of these is a programming language?
 - A) Docker B) Python
 - Answer:





ZERO SHOT vs ONE SHOT vs FEW SHOT - Example

EXAMPLE 1: ZERO SHOT

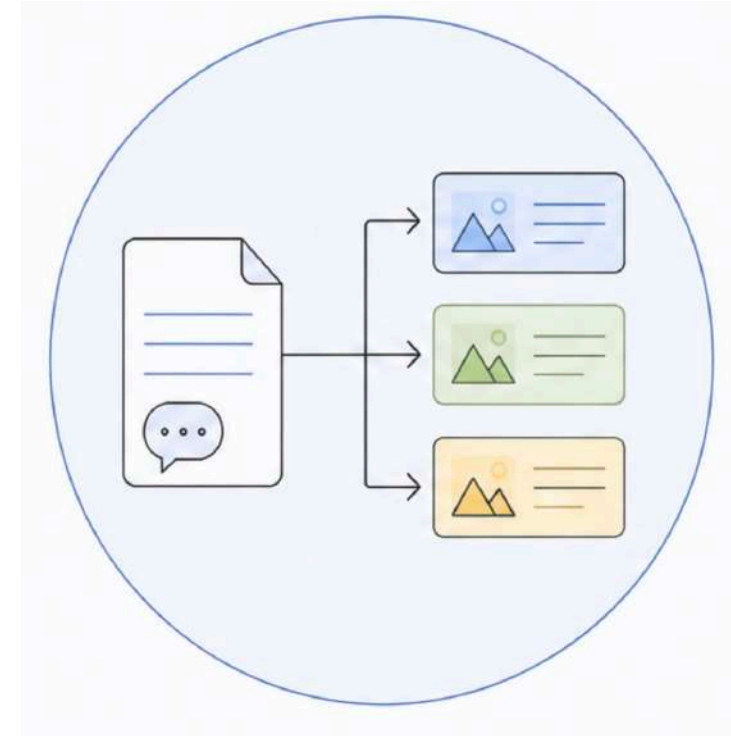
- For the given order, return a JSON object
 - **Order:** A pizza and a pepsi

EXAMPLE 2: ONE SHOT

- For the given order, return a JSON object
 - **Order:** A pizza and a pepsi, **Output:** {"pizza": 1, "pepsi": 1}
 - **Order:** A burger and a soda, **Output:**

Example 3: FEW SHOT

- For the given order, return a JSON object
 - **Order:** A pizza and a pepsi, **Output:** {"pizza": 1, "pepsi": 1}
 - **Order:** A burger and 2 sodas, **Output:** {"burger": 1, "soda": 2}
 - **Order:** A burger, A pizza and 2 sodas, **Output:**





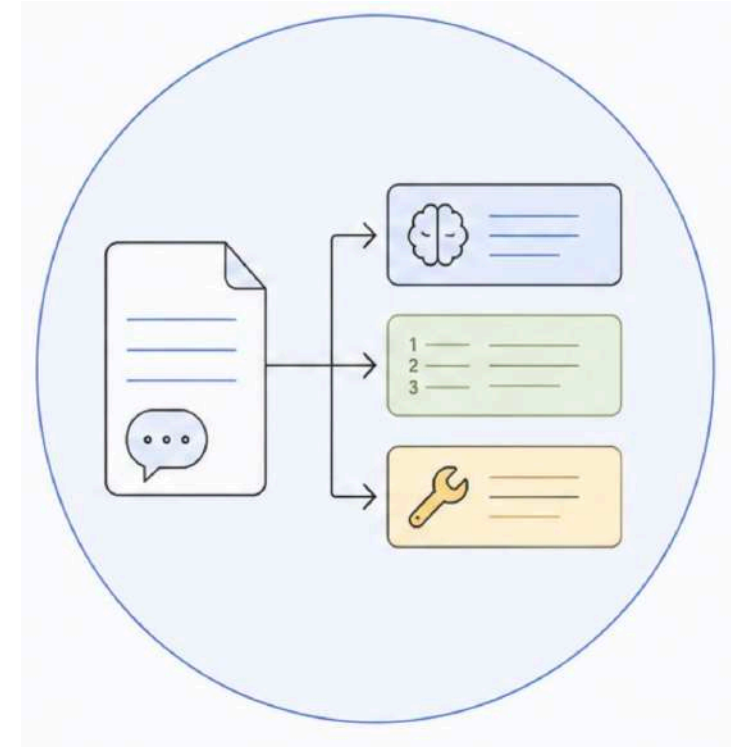
Chain-of-Thought Prompting

CoT: Encourage structured reasoning for complex problems

■ **Examples:**

- “If a pencil costs ₹10 and a notebook costs ₹30, how much for 3 pencils and 2 notebooks? **Show the calculation.**”
- “A train travels 60 km in 1.5 hours. What is its average speed? **Show the calculation.**”
- “Why does it rain more in coastal regions? **Explain step-by-step.**”
- “Explain why binary search is faster than linear search **step-by-step.**”
- “This Python function isn't working. **Debug it step-by-step.**”

Benefits: Can improve performance on some complex reasoning tasks





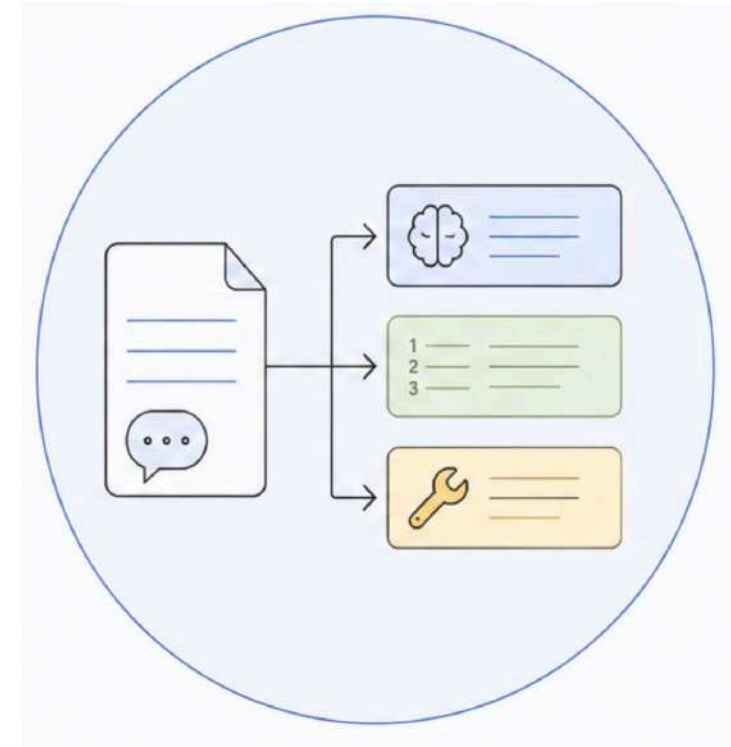
Why Use Prompt Chaining?

You ask a foundation model a complex question:

- Will one prompt always give the best result?
- **Not always**

What can you do?

- **Solution:** Use Prompt Chaining (break a complex task into multiple connected prompts)
- **Each Prompt:** Focuses on one specific step
- **Connection:** Output from one prompt can become input to the next





Prompt Chaining: Examples

Goal	Prompt Chain
Draft a customer email campaign	1: Summarize the new product features 2: Write an email using the summary 3: Polish it for a professional tone
Plan a 3-day trip to Paris	1: List top attractions 2: Organize them into a 3-day itinerary 3: Add nearby restaurant recommendations
Prepare an interview guide	1: Identify key skills for a Data Analyst 2: Generate questions for each skill 3: Add tips for answering each question

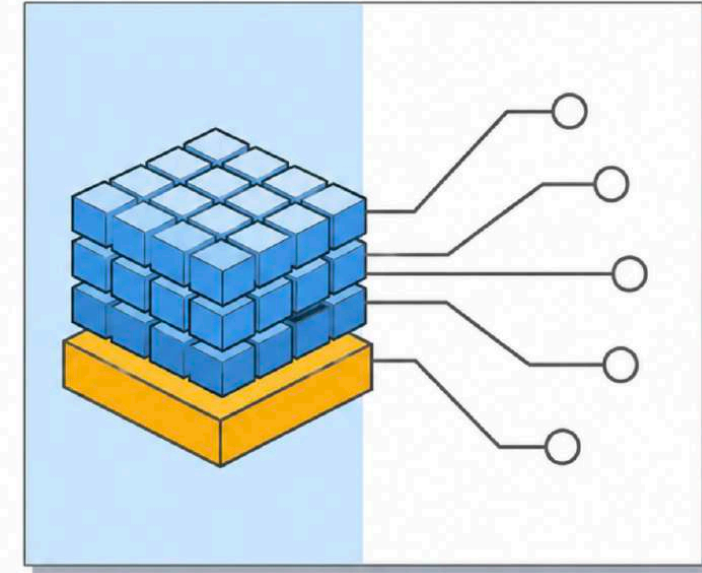


Add a Final Review

Before Finalizing: Ask the model to review its response

Ask the model: To check for

- **Completeness:** Cover all requested points
- **Sources:** Cite reliable sources when available
- **Accuracy:** Avoid unsupported or invented facts
- **Uncertainty:** Clearly label assumptions or uncertainty
- **Example:** "Final Review: Check for accuracy and completeness. Do not invent missing information. Clearly label uncertainty and cite reliable sources for important factual claims."





What is Meta-Prompting?

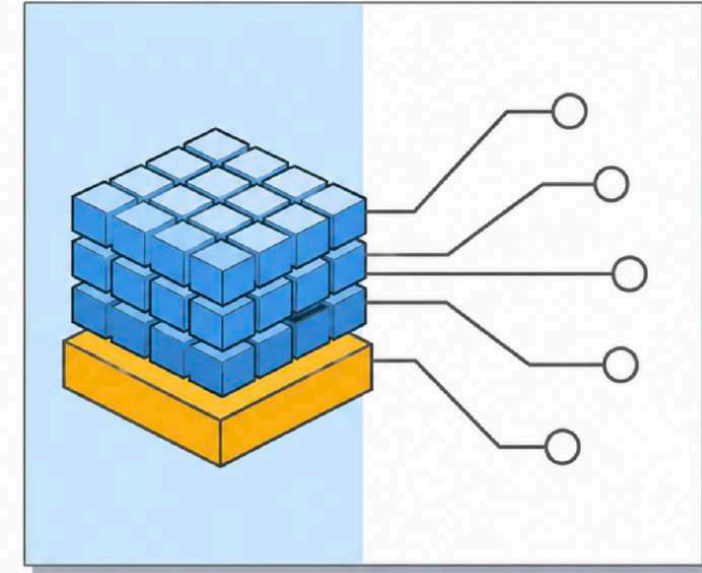
Meta-Prompting: Ask the model to help create or improve a prompt

- "Let's build a better prompt. Ask me a few questions first. Then create a final prompt with sections for Context, Instructions, and Output Format. Here's my draft prompt - `YOUR_DRAFT_PROMPT`"

Best For:

- Complex tasks
- When you want a reusable prompt

Not Needed: For simple, straightforward tasks





Prompting Techniques: A few scenarios

Scenario	Recommendation	Why?
A customer service chatbot needs to handle complaints empathetically and provide refund information	Role Prompting	Assigning the model the role of a polite and helpful support agent ensures consistent tone and behavior
You're solving a math word problem and want the AI to show every step clearly	Chain-of-Thought Prompting	Encourages the model to think step-by-step and provide detailed reasoning
You're developing an interview bot that behaves like a career counselor giving personalized advice	Role Prompting	Setting the persona helps tailor tone and guidance for the scenario
You want the model to convert customer orders into JSON using a specific structure	Few-Shot Prompting	Providing a few input-output examples shows the model the exact format and pattern you expect



Simple Prompt Framework - RTF

Role: Define a persona for the model

Task: Clearly state what you want the model to do

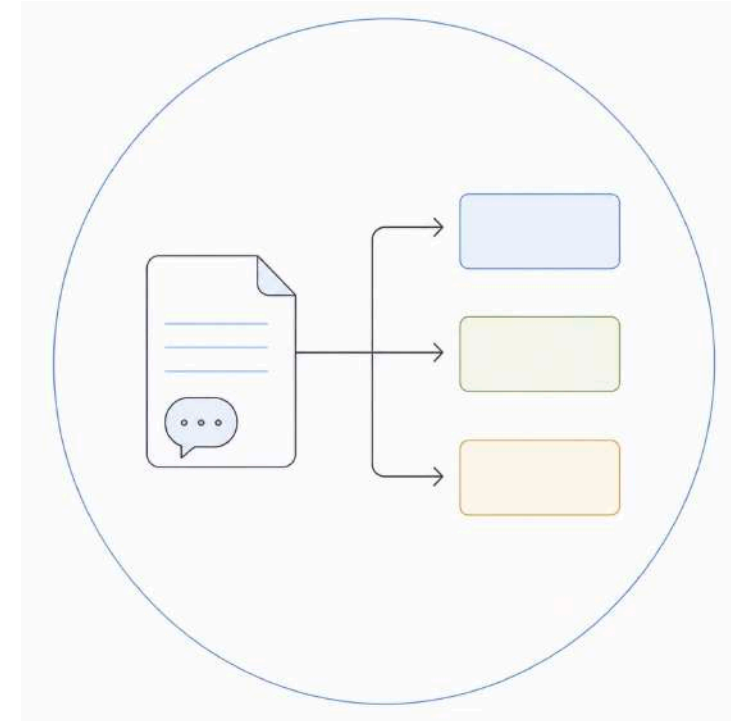
Format: Provide instructions on the desired format for the response

- Example: a summary, pros and cons, a debate, or a step-by-step explanation

Basic Prompt: "Evaluate FreshSip"

▪ **Improved RTF Prompt:**

- **Role:** You are an experienced investment analyst
- **Task:** Evaluate FreshSip Beverages as a long-term investment
- **Format:** Create a detailed report with sections on ..





Simple Prompt Framework - CTF

Context: Provide relevant background info

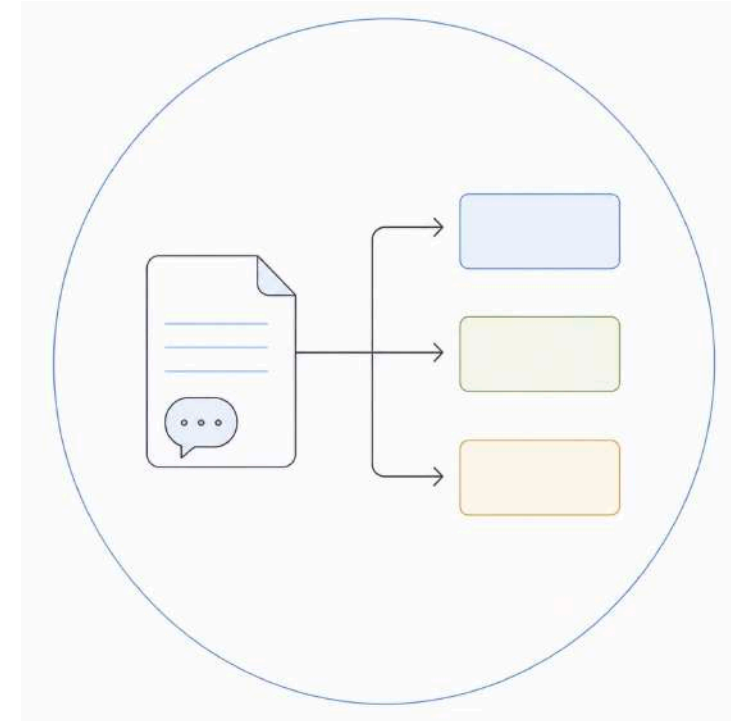
Task: Clearly state what you want the model to do

Format: Provide instructions on the desired format for the response

- Example: a summary, pros and cons, a debate, or a step-by-step explanation.

Basic Prompt: "Evaluate FreshSip"

- **Improved CTF Prompt:**
 - **Context:** Demand for packaged fruit drinks is growing rapidly
 - **Task:** Evaluate FreshSip Beverages as a long-term investment
 - **Format:** Create a detailed report with sections on ..





Prompt Framework - RASCEF

Role: Define the persona

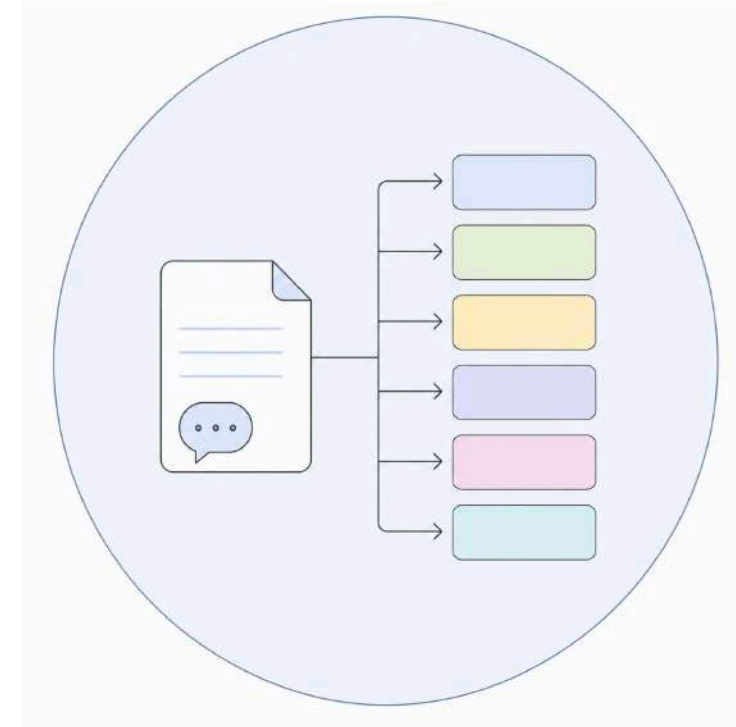
Action: Clearly state the problem

Steps: Outline the sequential or logical steps

Context: Provide relevant background info

Examples: Offer illustrative examples

Format: Provide instructions on the desired format for the response



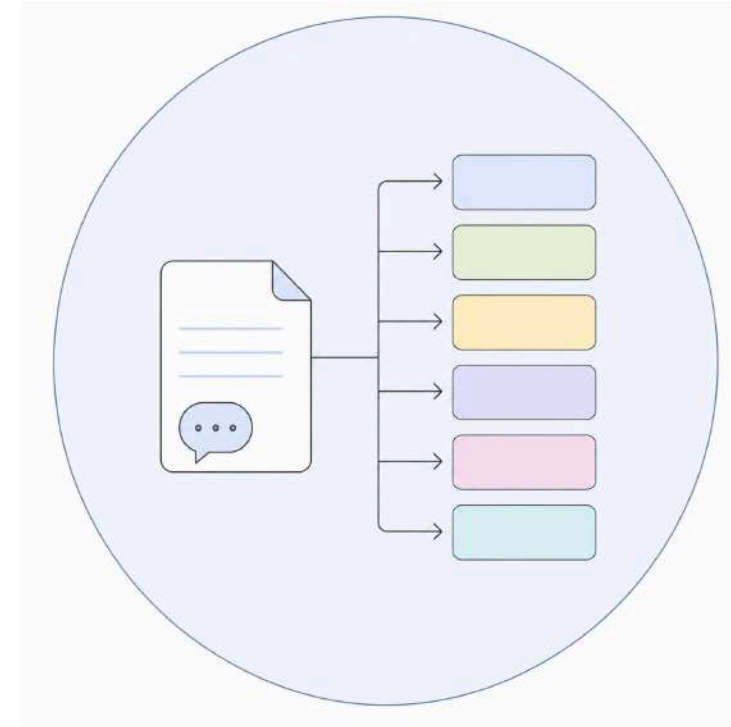


Prompt Framework - RASCEF Example

Basic Prompt: "Evaluate FreshSip"

Improved RASCEF Prompt:

- **Role:** You are an experienced investment analyst
- **Action:** Evaluate FreshSip Beverages as a long-term investment
- **Steps:** Assess growth, competitive advantage, financial strength, and valuation
- **Context:** Demand for packaged fruit drinks is growing rapidly
- **Examples:** Provide examples from earlier investment research as a reference
- **Format:** Create a detailed report with sections on ..





in28minutes

Getting Started with Choosing a Model



Understanding Model Parameters & Size

Model Parameters: Numerical values learned during training

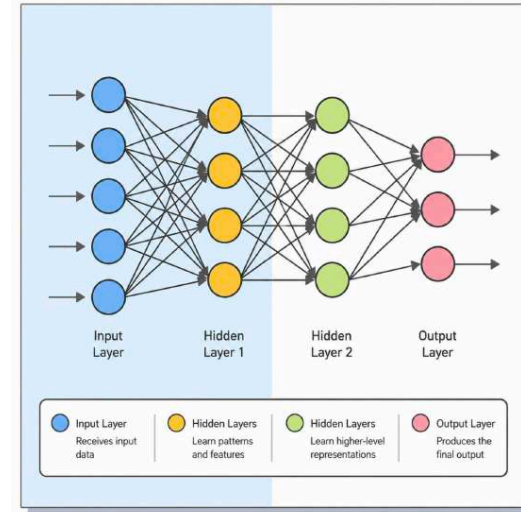
- **Weights:** Parameters that control the strength of connections in neural network

Model Size: Described by number of parameters

- **8B:** About 8 billion parameters
- **70B:** About 70 billion parameters

Are Bigger Models Better?: Not always

- **Trade-Offs:** Larger models => More hardware, latency, and cost
- **Key Idea:** Choose the smallest model that meets your task needs





Comparing Large and Small Language Models

Feature	Large Language Models	Small Language Models
Training Data	Trained on vast datasets	Trained on focused or domain-specific datasets
Conversational Scope	Handles a wide range of contexts and topics	Optimized for specific topics or use cases
Model Size	Very large (billions of parameters)	Compact and efficient
Deployment	Requires powerful infrastructure	Easy to deploy even on local devices
Fine-Tuning Cost	Expensive and resource-intensive	Faster and cheaper to fine-tune



How Do You Choose a Model? - Start with the Use Case

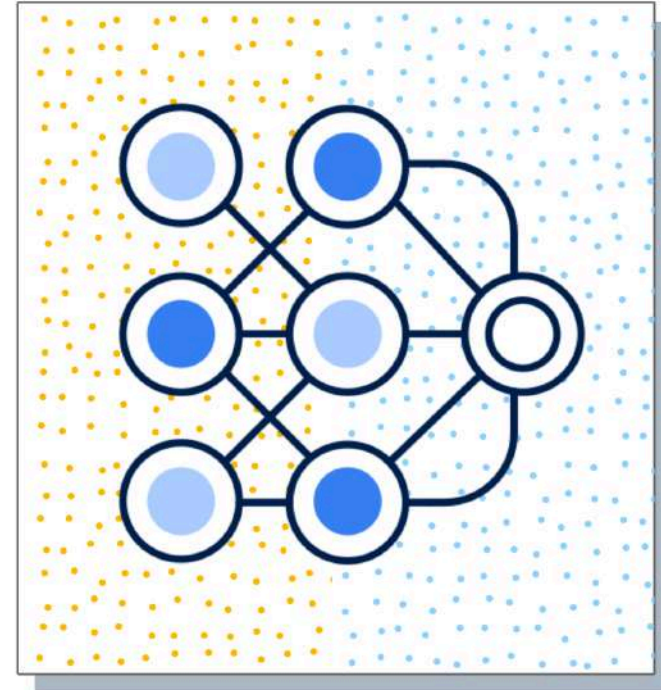
Task: Chat, summarization, coding, reasoning, image generation, etc

Modality: Text, image, audio, video, or multimodal

Context Requirements: How much information must it process?

Performance Requirements: Latency, Concurrency etc.

Benchmarks: Use relevant evaluations to shortlist models





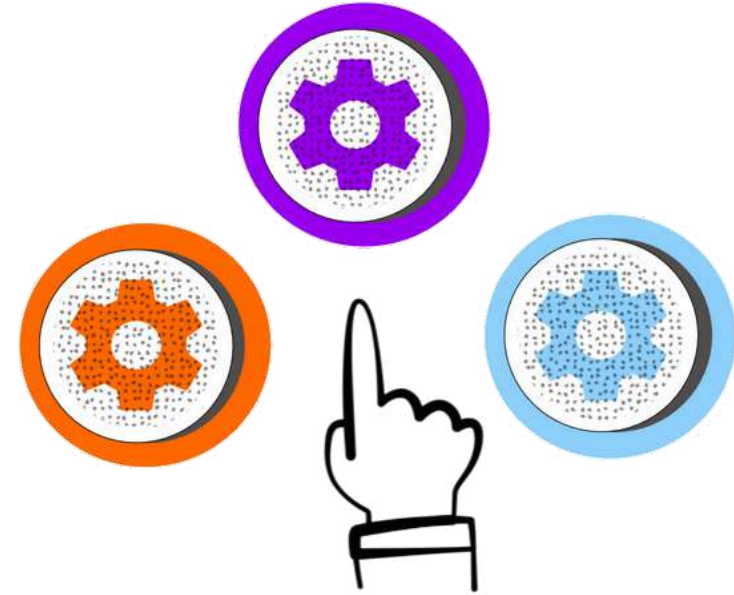
How Do You Choose a Model? - Compare Models

Performance: Is latency and concurrency support acceptable?

Cost: What will production usage cost?

Model Size: Is a smaller model sufficient?

Context Window: Does it support your input and output requirements?





How Do You Choose a Model? - Review Constraints

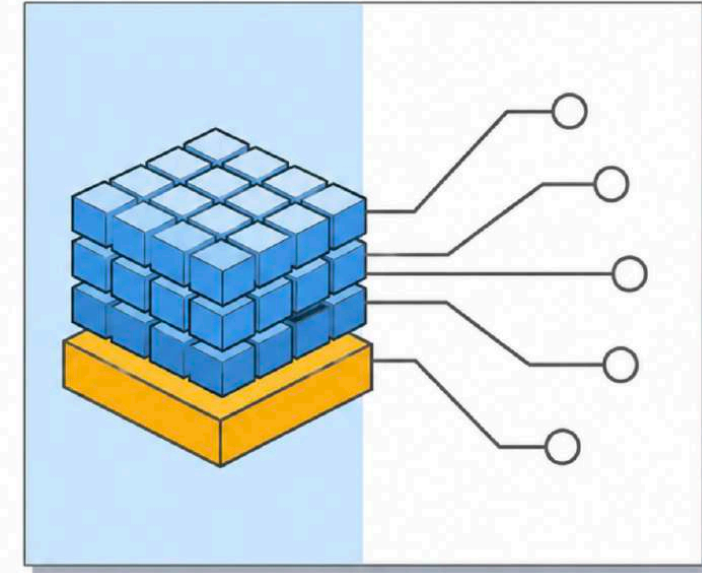
Availability: Is the model available in required regions?

Security & Compliance: Does it satisfy organizational requirements?

Safety: Does it meet your responsible AI requirements?

Licensing: Are there restrictions on usage or deployment?

Hosting: Serverless, dedicated, or self-hosted





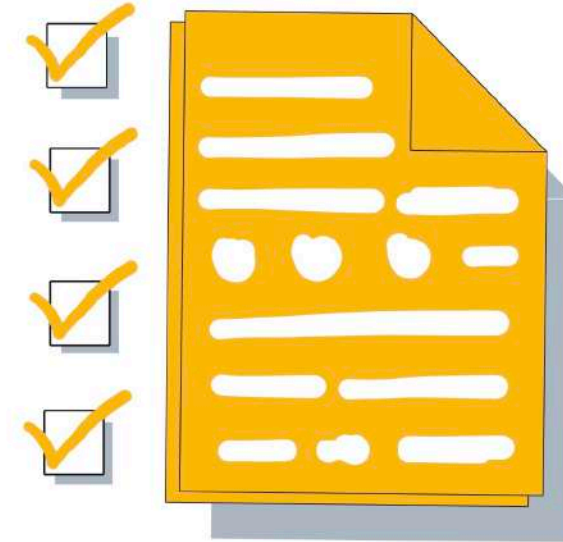
How Do You Choose a Model? - Test Before You Choose

Your Data Matters Most: Test with representative real-world data

Evaluate: Quality, Groundedness, Safety, Latency, and Cost

Final Choice: Select the model that best balances your requirements

Remember: There isn't a single model that is right for every use case





in28minutes

Getting Started with Data Challenges



Importance of Data

Data is fundamental to AI

- Used to train, evaluate, and ground AI models

Bad Data = Bad Outcomes

- Poor, incomplete, or biased data can lead to inaccurate or unfair results

Key Challenges:

- Can we get access to right data?
- How to ensure that the data is high quality?
- How to ensure that the data is representative and fair?
- How to protect sensitive data?





Obtaining the Right Data - Data Accessibility

Availability: Data may exist but be locked away

- *Ex: Health data hidden due to privacy regulations*

Cost: High-quality data is not free

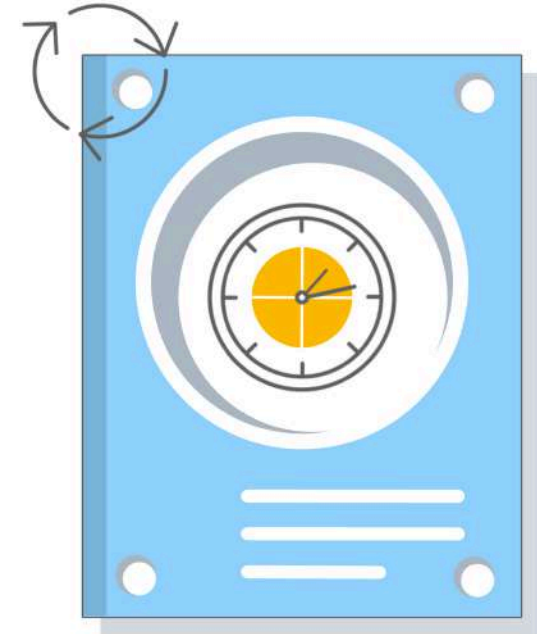
- *Ex: Labeling support emails can cost time and money*

Format: Raw data may need cleanup

- *Ex: Handwritten forms must be digitized*

Usability: Noisy or messy data reduces model effectiveness

- *Ex: Social media slang must be cleaned before analysis*





Ensuring Data Quality

Accuracy: Wrong labels = wrong learning

- *Ex: A cat labeled as a "dog" will confuse the model*

Completeness: Too little data = weak insights

- *Ex: One day weather history isn't enough for forecasting*

Representativeness: Missing groups = biased results

- *Ex: Ignoring teens in shopping data skews recommendations*

Consistency: Mixed formats = model confusion

- *Ex: "NYC" vs "New York" vs "NewYork"*





Understanding Bias and Fairness

Bias: Patterns in data that can lead to unfair predictions

- *Ex: Historical hiring data favors one group over another*

Fairness: Ensure AI outcomes are fair across relevant groups

How?

- **Audit Data:** Check for underrepresented groups
- **Use Representative Data:** Include diverse examples
- **Test Across Groups:** Compare model performance across relevant groups
- **Human Oversight:** Humans should review important decisions





Protecting Sensitive Data

Data Minimization: Collect and use only the data you need

Access Control: Restrict access to sensitive data

Anonymization: Remove identifying information

- Identity cannot reasonably be recovered

Pseudonymization: Replace identifying information with artificial identifiers

- Identity can potentially be restored using separately stored information





in28minutes

Understanding AI Systems

Key Risks



Is AI Perfect?

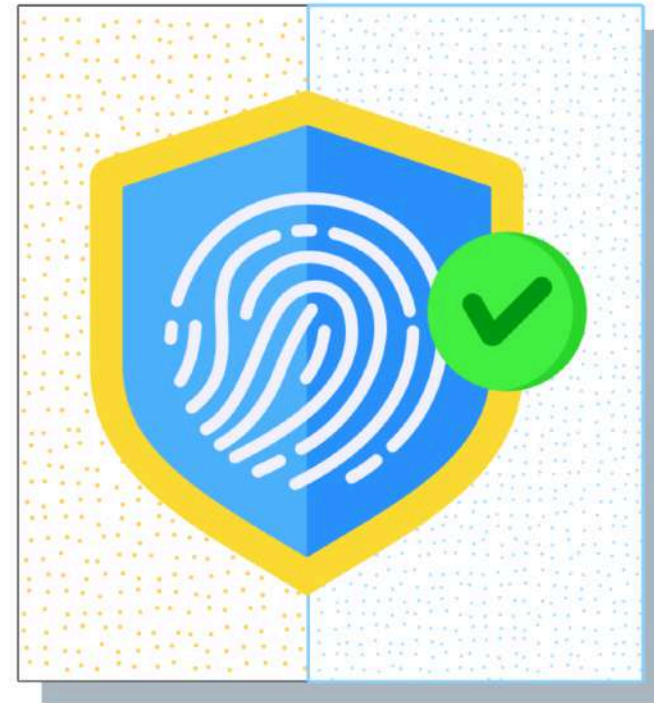
Can AI models sometimes produce **incorrect, harmful, or biased responses**?

- Yes

Can AI models be **misused or manipulated**?

- Yes

Next Up: Let's look at key risks with AI systems





AI Risks - Reliability

Risk	Simple Explanation	Example
Hallucinations	A model can present false or invented information as fact	It cites an academic paper that does not exist
Knowledge Cutoff	A model's built-in knowledge may not include information after its training cutoff	It cannot describe a product launched after its training cutoff
Edge Cases	A model may struggle with unusual situations	A support chatbot treats an empty package as a successful delivery because tracking says "Delivered"
Nondeterminism	The same prompt can produce different responses across runs	Given the same report and prompt, a model highlights different findings in two summaries



AI Risks - Data and Fairness

Risk	Simple Explanation	Example
Data Dependency	Poor-quality or incomplete training data can lead to poor results	A support assistant gives incorrect refund advice because its training data is missing information about international orders
Bias	A model may treat some groups unfairly because of patterns in its data	A resume screening tool ranks equally qualified women lower than men



AI Risks - Transparency and Safety

Risk	Simple Explanation	Example
Limited Interpretability	It can be difficult to understand why a model produced a particular answer	A model ranks one job applicant above another, but the reasons behind the ranking are unclear
Harmful Content	A model may generate content that could harm people	A chatbot suggests mixing household cleaners that are dangerous to combine



AI Risks - Security

Risk	Simple Explanation	Example
Poisoning	An attacker deliberately adds or changes data to influence an AI system's responses	An attacker changes a support knowledge base to list a fraudulent payment account
Prompt Injection	An attacker places instructions in a prompt or external content to make an AI system act against its intended instructions	A retrieved document tells the assistant to ignore the user's question and ask for their password
Jailbreaking	A user tries to bypass a model's safety rules	A user requests dangerous instructions "as a fictional story" to try to bypass safety rules
Tool Abuse	An AI agent uses a connected tool to take an unauthorized or harmful action	Asked to find old files, an agent deletes them without approval



AI Risks - Scenarios

Scenario	Risk
An AI-generated article cites a research paper that does not exist	Hallucinations
A chatbot cannot answer questions about a product launched after its training cutoff	Knowledge Cutoff
Given the same report and prompt, a model highlights different findings in two summaries	Nondeterminism
A resume screening tool consistently ranks equally qualified women lower than men	Bias
A model ranks one job applicant above another, but the basis for its ranking is unclear	Limited Interpretability
A user requests dangerous instructions “as a fictional story” to try to bypass safety rules	Jailbreaking



in28minutes

Getting Started with Secure AI



Why Secure AI at Every Stage?

End-to-End Security: AI systems are only as secure as their weakest point

Across the Lifecycle: Security must span data, development, deployment, and operations

Continuous Process: Plan → Secure → Monitor → Repeat

Next: Let's explore how to secure each stage of the AI lifecycle





Secure AI - At Each Stage [1/2]

Secure AI: Security matters throughout AI lifecycle

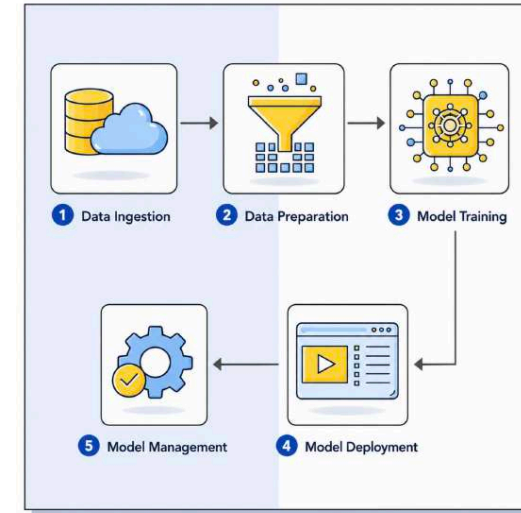
1: Data Ingestion: Secure how data enters the system

2: Data Preparation: Protect and validate data before use

3: Model Training: Secure training data, compute, and model artifacts

4: Model Deployment: Protect endpoints, access, and runtime environments

5: Model Management: Monitor, govern, and update models after deployment





Secure AI - At Each Stage [2/2]

Stage	Key Security Actions
Data Ingestion	Restrict who can upload or modify data. Use trusted sources. Log ingestion activity.
Data Preparation	Mask or anonymize sensitive information. Encrypt data. Validate and clean data before use.
Model Training	Secure the training environment. Restrict access to training data and model configuration. Protect model artifacts from theft or tampering.
Model Deployment	Control who can access the model. Secure APIs with authentication, authorization, and rate limits. Monitor usage.
Model Management	Monitor performance, drift, bias, and security events. Use alerts and update models when needed.



Secure AI – Match the Action to the Stage

Action	Stage
Log every file that gets uploaded to the training folder	Data Ingestion
Prevent unauthorized access to a pricing model's API	Model Deployment
Alert team when prediction accuracy suddenly drops	Model Management
Train models in an isolated environment to avoid external access	Model Training
Replace email addresses with masked IDs before feeding into model	Data Preparation
Detect bias over time and update model configuration	Model Management



in28minutes

Getting Started with ML Operations



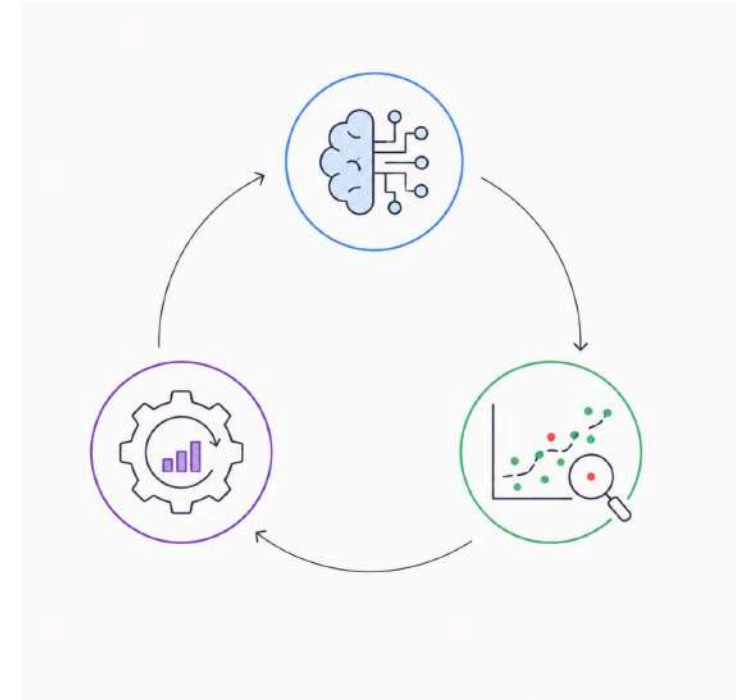
What Happens After Deployment?

Models serve predictions in production

- Data can change over time
- Model performance can decline

Models need continuous monitoring

- Track model performance using metrics and KPIs
- Detect data and performance drift
- Collect feedback and new data
- Retrain and deploy new model versions when needed
- Apply security updates





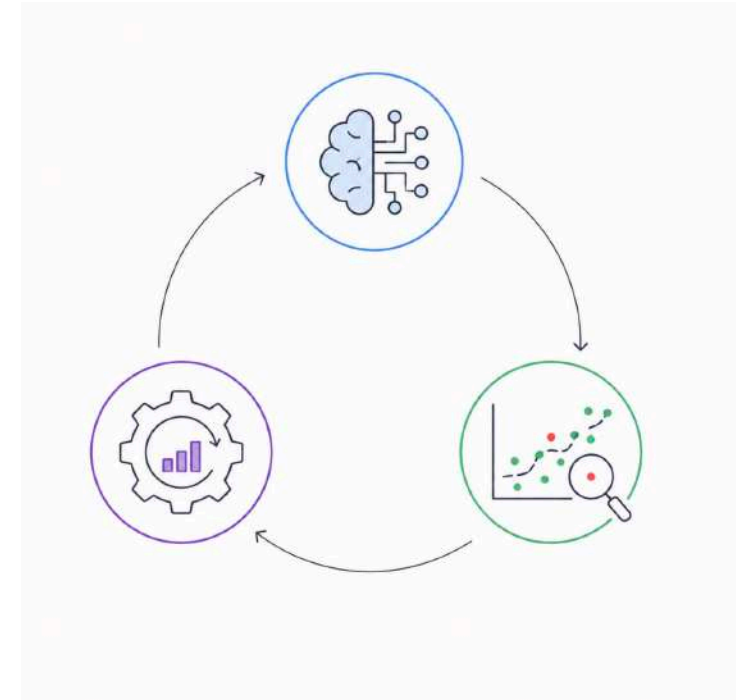
What is MLOps? [1/2]

MLOps: Practices that bring together Machine Learning development (Dev) and operations (Ops)

- Standardize and automate processes across the ML lifecycle

Key Principles:

- **1: Version Control:** Track code, data, and model versions
- **2: Automation:** Automate training, testing, and deployment

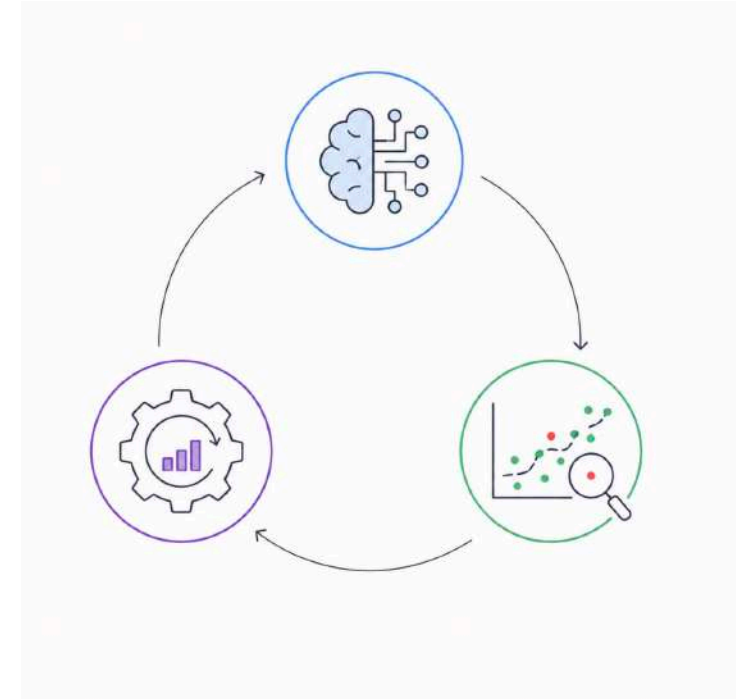




What is MLOps? [2/2]

Key Principles:

- **3: Continuous Monitoring:** Monitor data and model performance
- **4: Continuous Training:** Retrain models when needed
- **5: Continuous Integration:** Validate and test changes
- **6: Continuous Delivery:** Reliably deploy model versions
- **7: Model Governance:** Track, approve, audit, and manage models





in28minutes

WOW!

Congratulations

You Made It!



Congratulations on reaching the end of the course!

Be proud of your progress: You have taken another step forward

Thank you for choosing to learn with in28minutes



Keep Learning With Us

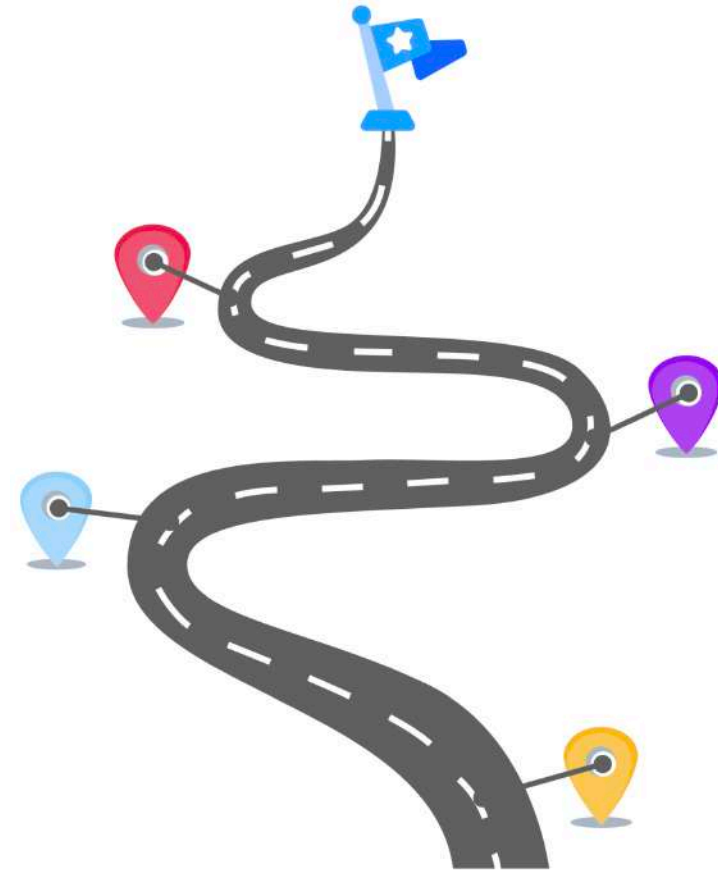


Share your story in the course Q&A or on LinkedIn. Tag **in28minutes** and **Ranga Karanam**.

Recommend the course to a friend who might find it useful.

Leave a Review: Your feedback helps future learners.

Explore our roadmaps when you are ready for your next step.





01  Generative AI Certifications

02  Cloud Certifications

03  Interview Guides

04  Programming

 **in28minutes**
From Cloud to Generative AI - Learn,
Build, and Evolve with in28minutes

